MediCore

Documentación completa del proyecto

Proyecto 2, curso 048 Aseguramiento de la Calidad, UMG

26 documentos. Generado el 2026-11-06.

Índice

#	Documento	Página del PDF
1	MediCore	3
2	MediCore · Alcance, pantallas y requisitos	8
3	MediCore · Plan de trabajo por entregas	11
4	MediCore · Matriz inicial de trazabilidad	13
5	MediCore · Registro de hallazgos y verificación manual	21
6	MediCore · App móvil del paciente (.NET MAUI, Android)	38
7	MediCore · Contenedores e infraestructura de QA (entrega 8)	43
8	MediCore · Pruebas de rendimiento (k6)	48
9	MediCore · Prueba de usabilidad	50
10	MediCore · Matriz de riesgos (confidencialidad, integridad y disponibilidad)	52
11	MediCore · Plan de pruebas	54
12	MediCore · Casos de prueba y evidencias	60
13	MediCore · Registro de defectos	64
14	MediCore · Informe final	68
15	MediCore · Guion de defensa práctica	73
16	MediCore · Reporte de seguridad y accesibilidad	77
17	Evidencia de la entrega 6: laboratorio y notificaciones asíncronas	80
18	Entrega 6 · comandos y resultados de las pruebas	82
19	Entrega 7 · resultado de los flujos Maestro	84
20	Entrega 7 · comandos y resultados verificados	85
21	Entrega 8 · flujos Maestro contra el ambiente QA en contenedores	87
22	Entrega 8 · verificación local (contenedores e infraestructura)	88
23	Terraform y Ansible ejecutados sin Azure	91
24	Maestro · repetición con el código final	93
25	Evidencia de resiliencia en QA	94
26	Respaldos automáticos y simulacro de restauración	95

MediCore

Plataforma hospitalaria: web del personal (Razor Pages), API (ASP.NET Core) y app móvil del paciente (.NET MAUI, Android). Proyecto 2 del Proyecto Final de QA. **Estado: aplicación, pruebas y pipeline completos en QA local.** Propuesta actual: NO-GO para producción y GO condicionado para QA (informe final).

Requisitos

Herramienta	Para qué	Versión
.NET SDK	compilar y ejecutar (hoy)	10.0.x
Git	control de versiones	cualquiera reciente
Visual Studio 2022 17.14+ o 2026	IDE (soporta `.slnx`)	opcional
Docker Desktop	ambiente QA, pruebas y pipeline	necesario
Workload MAUI + Android SDK	app móvil y Maestro	opcional
Terraform, Ansible	infraestructura	opcional; la validación y la simulación corren en contenedores

Ejecutar

Primera vez (dependencias y base de datos):

```
cp .env.example .env                # cambia las contraseñas; .env no se versiona
docker compose up -d                # PostgreSQL (puerto 5433), Redis y RabbitMQ (panel en
http://localhost:15672)
dotnet tool restore                 # instala dotnet-ef (herramienta local del repositorio)
dotnet user-secrets set "ConnectionStrings:Postgres"
"Host=localhost;Port=5433;Database=medicore;Username=medicore;Password=<POSTGRES_PASSWORD de tu .env>" -
-project src/MediCore.Api
```

Aplicar migraciones (`database update` lee la cadena de la variable de entorno; ejemplo en PowerShell):

```
$env:ConnectionStrings__Postgres =
"Host=localhost;Port=5433;Database=medicore;Username=medicore;Password=<POSTGRES_PASSWORD>"
dotnet ef database update --project src/MediCore.Infrastructure --startup-project src/MediCore.Api
```

Secretos locales de la API (inventa valores propios; nunca se versionan):

```
dotnet user-secrets set "ConnectionStrings:Redis" "127.0.0.1:6379,password=<REDIS_PASSWORD de tu .env>"
--project src/MediCore.Api
dotnet user-secrets set "ConnectionStrings:RabbitMq" "amqp://medicore:<RABBITMQ_DEFAULT_PASS de tu
.env>@127.0.0.1:5672/" --project src/MediCore.Api
dotnet user-secrets set "Jwt:SigningKey" "<texto aleatorio de 32 caracteres o más>" --project
src/MediCore.Api
dotnet user-secrets set "Seed:DevPassword" "<contraseña de desarrollo para los usuarios ficticios>" --
project src/MediCore.Api
```

Usa `127.0.0.1` y no `localhost` para Redis: Docker publica el puerto solo en IPv4 y el cliente puede intentar IPv6 primero.

Al arrancar la API en Development se aplican las migraciones y, si la tabla de usuarios está vacía, se crean usuarios ficticios con la contraseña de `Seed:DevPassword`: `elena.vargas@example.com` (Recepción), `lucia.mendez@example.com` (Médico), `daniela.reyes@example.com` (Laboratorio), `admin@example.com` (Administrador) y `ana.garcia@example.com` (Paciente, solo móvil). También se crean 4 pacientes y 3 médicos ficticios del diseño (Ana García queda vinculada a su cuenta y la Dra. Lucía Méndez a la suya). Cada médico recibe un horario de lunes a viernes de 08:00 a 12:00 y la Dra. Méndez cuatro citas el próximo día hábil, como en la agenda del diseño.

El puerto 5433 evita choques con un PostgreSQL local en 5432 (cámbialo con `POSTGRES_PORT`).

Después:

Abre `MediCore.slnx` en Visual Studio, o desde la terminal en la raíz (la web necesita la API en marcha para iniciar sesión):

```
dotnet build MediCore.slnx
dotnet test MediCore.slnx             # las de integración levantan PostgreSQL y Redis con
Testcontainers (requiere Docker)
dotnet run --project src/MediCore.Api --launch-profile http   # http://localhost:5011/api/health  (y
/api/health/ready comprueba PostgreSQL)
dotnet run --project src/MediCore.Web --launch-profile http   # http://localhost:5298
```

Con la API en Development, el contrato OpenAPI está en `http://localhost:5011/openapi/v1.json`.

Expediente, consultas y recetas

- El médico abre el expediente desde su agenda o desde "Pacientes autorizados". Cada consulta pertenece a una cita (una consulta por cita; de hoy o anteriores) y cada receta a una consulta, con 1 a 10 medicamentos.
- **Expediente compartido:** un médico con relación de atención válida con el paciente (una cita no cancelada o una consulta registrada) lee las consultas y recetas que escribieron **otros médicos**, identificadas por autor. **Nadie modifica registros de otro profesional** (solo el autor crea recetas sobre su consulta; las consultas y recetas no se editan ni se borran). Sin relación válida no hay acceso. Recepción, Administrador y Laboratorio no acceden a la información clínica.
- El paciente lee únicamente sus propias recetas, de cualquiera de sus médicos (`GET /api/prescriptions/me`, para la app móvil).
- La pestaña Laboratorio del expediente lista los resultados del paciente (ver la sección siguiente).
- La auditoría y las bitácoras registran quién accedió a qué recurso y cuándo, nunca el contenido clínico.

Laboratorio y notificaciones (entrega 6)

- **Laboratorio** registra resultados (WEB-17: paciente, tipo de examen, fecha y de 1 a 10 parámetros con valor, unidad y rango), los lista con el estado de la notificación (WEB-16) y ve su detalle (WEB-18). Los resultados no se editan ni se borran: una corrección es un resultado nuevo.
- **Quién ve qué:** Laboratorio, todos los resultados; el **Médico**, los de pacientes con quienes tiene una relación válida de atención (desde la pestaña Laboratorio del expediente); el **Paciente**, solo los suyos (`GET /api/labs/results/me`, para la app móvil). **Recepción y Administrador no tienen acceso clínico.** Un acceso ajeno devuelve 404 (no confirma que exista) y queda auditado; la auditoría y las bitácoras no guardan tipo de examen, valores ni observaciones.
- **Notificación asíncrona (RabbitMQ real, outbox transaccional):** al guardar un resultado se guarda en la **misma transacción de PostgreSQL** el evento `LabResultReady` (solo identificadores). Un proceso en segundo plano lo publica en RabbitMQ y un consumidor crea la notificación interna del paciente. Si RabbitMQ no está disponible, el resultado queda guardado (estado "Pendiente"), el evento espera con reintentos crecientes y se publica cuando el broker vuelve. El consumidor es idempotente (un mensaje reentregado no duplica la notificación), reintenta los fallos con espera y, agotados los intentos, deja el mensaje en la cola de mensajes fallidos (`medicore.notifications.dlq`). En WEB-18, Laboratorio puede **reintentar la notificación** si sigue pendiente.
- **API para la futura app móvil:** `GET /api/notifications` (paginado, `unreadOnly`), `GET /api/notifications/unread-count` y `POST /api/notifications/{id}/read`. No hay portal web del paciente ni se envían correos.
- **Configuración:** `ConnectionStrings:RabbitMq` en user-secrets (arriba) y la sección `Messaging` de `appsettings.json` (intervalos, intentos). `Messaging:Enabled=false` apaga publicador y consumidor.
- **Evidencia reproducible:** `python scripts/evidencia_laboratorio.py` (con la API en marcha y `MEDICORE_PASSWORD`) ejecuta los escenarios con el broker real, incluido detener y volver a iniciar el servicio `rabbitmq` de compose, y escribe `documentacion/evidencias/entrega-6-laboratorio.md`.

Pruebas en equipos con Smart App Control

Windows con Smart App Control en modo activo bloquea la carga de los `.dll` sin firma que compila el proyecto (migraciones, pruebas, API). Sin desactivarlo, `scripts/dotnet-linux.sh` y `scripts/dev-stack-linux.sh` ejecutan el mismo código en contenedores Linux del SDK oficial (se descarga `mcr.microsoft.com/dotnet/sdk:10.0`; la caché de NuGet vive en el volumen nuevo `medicore-nuget`). Los volúmenes y la red de compose (`medicore_*`) no se modifican. Detalle en `CLAUDE.md`.

App móvil del paciente (entrega 7)

App Android con .NET MAUI (`src/MediCore.Mobile`) sobre la API real: inicio y cierre de sesión, próxima cita, mis citas y detalle, reserva en tres pasos, cancelación, resultados, recetas y notificaciones (MOB-01 a MOB-12). La lógica está en `src/MediCore.Mobile.Core` y se prueba sin Android. **Cómo compilar, instalar, conectar con la API en Docker y ejecutar Maestro: documentacion/05-app-movil.md.** La app se compila con `scripts/android-build-linux.sh` (contenedor Linux) y se prueba en un emulador con Maestro; las herramientas (SDK de Android, emulador, Maestro) ya están instaladas en este equipo. Pasos exactos en el documento.

Contenedores y ambiente QA (entrega 8)

El sistema completo corre en contenedores (API, web, proceso de mensajería, PostgreSQL, Redis, RabbitMQ y proxy inverso). **Guía, arquitectura, costos estimados de Azure y procedimientos de crear, modificar y destruir: documentacion/06-infraestructura.md.**

```
scripts/qa.sh init      # .env.qa con contraseñas aleatorias (no se versiona)
scripts/qa.sh up        # construye, migra y levanta; solo el proxy publica puertos
(http://127.0.0.1:8080)
scripts/qa.sh seed      # usuarios y datos ficticios
scripts/qa.sh smoke     # pruebas de humo de web y API
scripts/qa.sh down      # conserva los volúmenes; "destroy" los borra (pide confirmación)
scripts/qa.sh backups   # lista los respaldos automáticos; también: backup, export, import, restore
```

La infraestructura de Azure (Terraform y Ansible) se **ejecutó sin costo, no en Azure**: `scripts/iac-simulacion.sh` usa un proveedor simulado y un Ubuntu desechable. No se creó ningún recurso real; el plan y su costo esperan aprobación (sección 3 del documento). La app Android es un APK, no un servicio del Compose.

Agenda y citas

- Las horas son del reloj de la clínica (`Clinic:TimeZone`, por defecto `America/Guatemala`); se puede reservar desde hoy hasta `Scheduling:MaxDaysAhead` días (90). Las citas duran 30 minutos.
- La disponibilidad se cachea en Redis (`Cache:AvailabilityTtl`, 2 min) y se invalida al reservar, cancelar o cambiar el horario del médico.
- Para reintentar sin duplicar, `POST /api/appointments` acepta el encabezado `Idempotency-Key` (8 a 64 caracteres, letras, números, `-` y `_`). La web lo genera en cada formulario de reserva.
- Una cita no se puede solapar con otra activa del mismo médico ni del mismo paciente: lo garantiza PostgreSQL (extensión `btree_gist`, que incluye la imagen oficial de Docker; en Azure Database for PostgreSQL debe habilitarse en la lista de extensiones permitidas).

Estructura

```
MediCore.slnx
docker-compose.yml      dependencias locales
src/MediCore.Api        Web API
src/MediCore.Contracts  contratos compartidos API/Web/móvil (DTO, roles)
src/MediCore.Domain     entidades y reglas (sin dependencias)
src/MediCore.Infrastructure  EF Core, PostgreSQL, Redis, migraciones
src/MediCore.Web        Razor Pages del personal
tests/MediCore.Api.Tests   integración (Testcontainers)
tests/MediCore.UnitTests   reglas de dominio
documentacion/          alcance, plan de trabajo, trazabilidad
Docs/  Referencias/     documento del proyecto y diseño aprobado (solo lectura)
```

Estado de las pruebas

Suite	Resultado
xUnit	304/304 (48 unitarias, 50 móviles, 206 de integración)
Insomnia	65/65 comprobaciones en 61 solicitudes
Cypress en Chrome	116/116, incluidas 14 pantallas con axe
Maestro	8/8 con el código final, en emulador contra QA
k6	Load, Stress y Spike dentro de umbrales, en equipo local
SonarQube	Puerta aprobada, 0 bugs, 74,2 % de cobertura
ZAP	0 alertas altas en web y API con 5 roles; 40 de 45 operaciones alcanzadas
Fortify	Sin ejecutar: falta licencia
Usabilidad	Definida, sin aplicar a personas

Documentos de entrega

Documento	Contenido
Plan de pruebas	Las 20 secciones exigidas
Matriz de riesgos	Confidencialidad, integridad y disponibilidad
Casos de prueba	Casos críticos con su evidencia
Registro de defectos	Pasos, severidad, prioridad y evidencia
Informe final	Métricas y decisión GO / NO-GO
Guion de defensa	Demostraciones en vivo
Seguridad y accesibilidad	Hallazgos y su tratamiento

Pipeline

Jenkins y SonarQube corren en contenedores:

```
docker compose -p medicore-ci --env-file ci/.env.ci -f ci/docker-compose.ci.yml up -d --build
```

Jenkins queda en `http://127.0.0.1:8090` y SonarQube en `http://127.0.0.1:9000`. `ci/.env.ci` no se versiona y define `JENKINS_ADMIN_PASSWORD`, `SONAR_ADMIN_PASSWORD`, `SONAR_TOKEN` y `MEDICORE_BRANCH`. Para crear la puerta de calidad, usa `scripts/sonar-setup.sh`.

Cada corrida borra los datos de QA y los vuelve a sembrar. Solo afecta los volúmenes `medicore-qa_*`.

Seguridad

No se guardan secretos en el repositorio. Copia `.env.example` a `.env` (ignorado por Git) cuando una entrega lo requiera. Todos los datos de ejemplo son ficticios.

MediCore · Alcance, pantallas y requisitos

Fuente: `Docs/Proyecto Final QA.docx` (Proyecto 2 + secciones generales 1–9, 11, 12) y `Referencias/medicore/`. El documento se leyó completo, incluidas las tablas (extraídas del XML del .docx). No se implementan los Proyectos 1, 3 y 4.

Alcance

Dentro: gestión de pacientes y médicos; agenda y reserva de citas; consultas; recetas; resultados de laboratorio; notificaciones al paciente; app móvil Android del paciente; autenticación, autorización por rol y por recurso, auditoría.

Fuera: portal web del paciente (cancelado en la revisión del diseño), cualquier módulo no listado, microservicios.

Roles (del prototipo)

Recepción, Médico, Laboratorio, Administrador (web del personal) y Paciente (móvil). El rol sale del usuario autenticado; la barra Web/Móvil + rol del prototipo **no** se copia.

Inventario de pantallas

Web del personal (WEB)

ID	Pantalla	Roles	Notas
WEB-01	Inicio de sesión	todos	Sin credenciales precargadas en el producto final
WEB-02	Agenda de citas	Recepción, Médico (WEB-11)	Calendario, filtro por médico, horarios 08:00–11:30 cada 30 min
WEB-03	Reservar cita	Recepción	Paciente, médico, fecha, horario; error si el horario está ocupado
WEB-04	Detalle de cita	Recepción	Cancelar cita (con confirmación)
WEB-05	Gestión de pacientes	Recepción, Administrador, Médico (solo autorizados)	Búsqueda por nombre/identificación
WEB-06	Registrar / editar paciente	Recepción, Administrador	Identificación única
WEB-07	Gestión de médicos	Administrador	
WEB-08	Registrar / editar médico	Administrador	
WEB-09	Disponibilidad médica	Administrador	Días, inicio/fin; fin > inicio
WEB-10	Usuarios y roles	Administrador	Asignar rol

ID	Pantalla	Roles	Notas
WEB-11	Mi agenda	Médico	Vista del día; abre expediente
WEB-12	Expediente del paciente	Médico	Pestañas Consultas / Recetas / Laboratorio
WEB-13	Registrar consulta	Médico	Guardar / guardar y crear receta
WEB-14	Crear receta	Médico	
WEB-15	Detalle de receta	Médico	
WEB-16	Resultados de laboratorio	Laboratorio	Estado de notificación Enviada/Pendiente
WEB-17	Registrar resultado	Laboratorio	El resultado se guarda aunque la notificación quede pendiente
WEB-18	Detalle de resultado	Laboratorio, Médico	Reintentar notificación

App móvil del paciente (MOB)

ID	Pantalla
MOB-01	Inicio de sesión
MOB-02	Inicio (próxima cita, accesos)
MOB-03	Mis citas
MOB-04	Seleccionar médico (paso 1/3)
MOB-05	Fecha y horario (paso 2/3)
MOB-06	Confirmar cita (paso 3/3)
MOB-07	Detalle de cita (cancelar)
MOB-08	Mis resultados
MOB-09	Detalle de resultado
MOB-10	Mis recetas
MOB-11	Detalle de receta
MOB-12	Notificaciones

Requisitos obligatorios (resumen; IDs en la matriz de trazabilidad)

Funcionales (Proyecto 2): pacientes y médicos, agenda/citas, consultas, recetas, laboratorio, notificaciones, app móvil.

API mínima de referencia: `GET /api/doctors`, `GET /api/doctors/{id}/availability`, `POST /api/appointments`, `GET /api/patients/{id}`, `POST /api/labs/results`, `GET /api/prescriptions/{id}`.

Redis: cache de disponibilidad médica, catálogos, sesiones y consultas frecuentes; probar datos vencidos, TTL, invalidación y recuperación ante indisponibilidad.

RabbitMQ: eventos `AppointmentCreated`, `AppointmentCancelled`, `LabResultReady`, `PrescriptionCreated`, `PatientNotification`; productor, consumidor, reintentos y manejo de errores (DLQ o equivalente); una caída del broker no debe perder información clínica ya almacenada.

Escenario crítico: alta demanda de citas mientras los médicos consultan agendas y expedientes → sin doble reserva ni colisiones de horario, consistencia, y comportamiento correcto cuando RabbitMQ se interrumpe después de registrar un resultado.

Seguridad: RBAC clínico/administrativo, autorización a nivel de recurso (IDOR), exposición de datos sensibles, sesiones y autenticación, seguridad de API, trazabilidad y auditoría.

Rendimiento: pacientes pidiendo citas + profesionales consultando; p95, tasa de error, throughput, estabilidad; umbrales declarados antes de ejecutar; Load, Stress y Spike.

Móvil: solicitar/consultar citas, ver resultados y recetas, notificaciones; Maestro con 5–8 flujos (incluye al menos un negativo).

Generales: GitHub (ramas, PR, Issues, Releases, protección de `main`); Jenkins + Jenkinsfile; Docker/Compose; Azure + Terraform; Ansible; API REST documentada; Insomnia (positivas, negativas, autorización); SonarQube + Quality Gate; Fortify, Dependency-Check, npm audit; ZAP; Cypress (smoke, flujo crítico, regresión, ≥1 negativo); JMeter o k6; usabilidad (≥5 participantes) + Lighthouse/axe; registro de defectos; secretos fuera del repo; ambiente reproducible desde el repo.

Pipeline mínimo (18 etapas): checkout, dependencias, lint, unitarias, SonarQube, npm audit, Dependency-Check, Fortify, Docker build, deploy QA, pruebas API, Cypress, Maestro, ZAP, rendimiento, Quality Gate, aprobación, deploy final. Prueba crítica fallida, vulnerabilidad crítica no aceptada o Quality Gate rechazado bloquean el deploy final.

Entregables documentales: plan de pruebas (20 secciones obligatorias, sección 7 del documento), matriz de riesgos (C/I/D), casos y evidencias, registro de defectos, informe final con métricas y GO / NO-GO, presentación (10–15 min) y demo.

Observaciones sobre el documento y el diseño

- El documento no fija el modelo de sesiones/JWT; se decide en la entrega de autenticación (ver CLAUDE.pdf).
- El prototipo permite que Recepción/Administrador gestionen pacientes y que el Médico vea "Pacientes autorizados": la regla de "autorizado" (relación médico–paciente por cita/consulta) hay que definirla; no está en el documento. Propuesta pendiente de validar.
- El prototipo trae un rol "Paciente" en Usuarios y roles, usado por la app móvil.
- Hallazgo del prototipo: la reserva/cancelación y disponibilidad usan solo el día del mes (sin mes/año) y datos en memoria; no es modelo válido de concurrencia.
- Numeración del documento: salta de la sección 9 a la 11 (no existe la 10). No afecta al alcance.

MediCore · Plan de trabajo por entregas

Cada entrega es pequeña, termina compilando, con pruebas verdes y la matriz de trazabilidad actualizada. Un flujo a la vez, comparado contra el diseño de referencia.

#	Entrega	Contenido	Estado
0	Preparación	Solución, API y web iniciales, CLAUDE.md, README, plan y trazabilidad, prueba de humo	**Hecha**
1	Base técnica	Capas del monolito modular (dominio/aplicación/infraestructura por módulo), EF Core + PostgreSQL con migraciones, `docker-compose` (Postgres, Redis, RabbitMQ), `.env.example`, configuración y secretos locales (user-secrets), manejo global de errores, Swagger/OpenAPI	**Hecha** (PR #1 fusionado)
2	Identidad y acceso	Usuarios, roles, login web (cookie) y API (JWT corto + refresh), sesiones en Redis, políticas por rol y por recurso, auditoría base, WEB-01, MOB-01 (API), usuarios y roles (WEB-10)	**Hecha** (PR #2 fusionado)
3	Pacientes y médicos	CRUD, búsqueda, `GET /api/patients/{id}`, `GET /api/doctors`, WEB-05…08, auditoría de accesos a datos clínicos	**Hecha** (PR #3 fusionado)
4	Agenda y citas	Disponibilidad (WEB-09), `GET /api/doctors/{id}/availability`, `POST /api/appointments`, cancelación, **anti doble reserva** (restricción única en BD + concurrencia optimista/transacción), cache Redis de disponibilidad con invalidación, WEB-02/03/04/11	**Hecha** (incluida en main con el PR #4)
5	Consultas y recetas	Expediente, consultas, recetas, `GET /api/prescriptions/{id}`, WEB-12…15, autorización por recurso	**Hecha** (PR #4 fusionado en main; incluye la entrega 4 y el expediente compartido)
6	Laboratorio y notificaciones asíncronas	`POST /api/labs/results` y consulta/listado, WEB-16…18, autorización por rol y por recurso, **RabbitMQ real**: evento `LabResultReady` con outbox transaccional, consumidor idempotente con reintentos y DLQ, API de notificaciones del paciente	Implementada en la rama `feature/entrega-6-laboratorio-notificaciones` (pendiente de PR y fusión)
7	Mensajería y notificaciones (resto)	Generalizar lo hecho en la entrega 6 (outbox, publicador, consumidor con reintentos, DLQ e idempotencia) a los eventos restantes: `AppointmentCreated`, `AppointmentCancelled`, `PrescriptionCreated` y `PatientNotification`; reintento de notificación	**Hecha**
8	Web del personal completa	Pulido contra el diseño, accesibilidad, responsive, español	**Hecha**: 116/116 en Cypress, 14 pantallas con axe
9	App móvil (trabajada como "entrega 7" de la numeración de sesiones)	.NET MAUI Android (MOB-01…12) sobre la API real, flujos Maestro y documentación de conexión con Docker	Implementada en la rama `feature/entrega-7-app-movil`; compilada a APK de desarrollo y **probada en emulador (Maestro 8/8)**; pendiente de PR y fusión
10	Pruebas	Unitarias, integración (BD/Redis/RabbitMQ con Testcontainers), API (Insomnia), concurrencia de agenda, Cypress, Maestro	**Hecha**
11	DevSecOps	Jenkinsfile (18 etapas), SonarQube, Dependency-Check, npm audit, Fortify, ZAP, Dockerfiles	**Hecha**: 18 etapas y bloqueo demostrado. Fortify sin licencia

#	Entrega	Contenido	Estado
12	Cloud (preparación = "entrega 8" de la numeración de sesiones)	Terraform (Azure), Ansible, aprovisionar/modificar/destruir, ambiente QA	Preparado y verificado en local. Azure es el último paso
13	Rendimiento y calidad	k6/JMeter con umbrales previos, Lighthouse/axe, prueba de usabilidad	k6 y axe hechos. Usabilidad pendiente de aplicar
14	Cierre	Plan de pruebas (20 secciones), matriz de riesgos, casos/evidencias, defectos, informe GO / NO-GO, presentación	**Hecha**: propuesta NO-GO para producción, GO condicionado para QA

Decisión aceptada (entregas 3–4): el médico accede a pacientes con quienes tiene una cita no cancelada **o una consulta registrada**; la regla se valida en la API (`PatientAuthorizationHandler`). Cancelar una cita solo retira el acceso si no queda otra relación válida (otra cita con ese médico o una consulta). La parte de consultas se implementó y se prueba en la entrega 5 (la prueba que estaba omitida ya está activa).

Pendientes (ver `13-informe-final.md`): usabilidad con personas, Fortify y despliegue en Azure.

Decisión aceptada (entrega 5, ajuste): **expediente compartido entre médicos autorizados.** Un médico con relación de atención válida lee las consultas y recetas que registraron otros médicos; nadie modifica registros de otro profesional; sin relación válida no hay acceso; los demás roles mantienen sus restricciones. Detalle en `CLAUDE.md` y en `04-registro-de-hallazgos.md`.

Decisión aceptada (entrega 6): el **evento** `LabResultReady` y la infraestructura de mensajería (outbox, publicador, consumidor con reintentos, DLQ e idempotencia) se adelantaron a la entrega 6 porque el flujo de laboratorio los necesita. La entrega 7 conserva el resto: los eventos `AppointmentCreated`, `AppointmentCancelled`, `PrescriptionCreated` y `PatientNotification`, y la generalización del consumidor.

Decisión aceptada (entrega 8): el proceso de mensajería se separa en el proyecto `MediCore.Worker` (mismo código de Infrastructure) para desplegarlo como contenedor; en desarrollo sigue dentro de la API. **Siguen pendientes y NO están hechos:** Jenkins (Jenkinsfile de 18 etapas), SonarQube, Dependency-Check/npm audit/Fortify, OWASP ZAP, Insomnia, k6/JMeter, ampliar Cypress y los entregables de QA de la entrega 14.

Dependencias clave: 2 antes de 3–6; 4 y 6 usan los eventos de 7 (se publican vía outbox desde 4 y 6 y se consumen en 7); 9 requiere API estable (≥ entrega 6).

Los entregables de documentación (plan de pruebas, riesgos, casos) se van construyendo en cada entrega y se consolidan en la 14.

MediCore · Matriz inicial de trazabilidad

Estados: ✓ hecho y verificado · ◐ parcial · ○ pendiente. "Implementación" y "Pruebas" apuntan a rutas reales cuando existan; hoy solo hay lo de la entrega 0.

Entregables de pruebas obligatorios

Las 304 pruebas xUnit (48 unitarias, 50 de la lógica móvil y 206 de integración con PostgreSQL, Redis y RabbitMQ reales; ninguna omitida) son complementarias. El documento exige además estos entregables:

Entregable (documento, secciones 4, 8 y 12)	ID	Qué exige	Estado	Entrega
Cypress (web)	GEN-10	Smoke, flujo funcional crítico, regresión y al menos un escenario negativo	✓ **EJECUTADO** — 11 specs, 116/116 en Chrome contra QA (102 funcionales y 14 de accesibilidad). Electron y dispositivo físico sin repetir	10
Insomnia (API)	GEN-17	Colección con pruebas positivas, negativas, de autenticación, autorización y errores	✓ **EJECUTADO** — 61 solicitudes, 65/65 comprobaciones contra QA	10
Maestro (móvil)	GEN-11	Entre 5 y 8 flujos de la app, con al menos un escenario negativo	✓ **EJECUTADO** — `tests/maestro`: 8 flujos (login válido e inválido, reserva, consulta y cancelación, resultados, recetas, notificaciones, cierre de sesión; con negativos) **8/8 aprobados con el código final en emulador Android 35, contra QA** (`documentacion/evidencias/maestro-codigo-final.md`). Límite: sin dispositivo físico	7, 10
Rendimiento con k6 o JMeter	GEN-12, CRI-01, CRI-02	Load, Stress y Spike con umbrales declarados antes de ejecutar	✓ **EJECUTADO** — umbrales cumplidos, 0 solapes y 0 pendientes; medición local (`07-rendimiento.md`)	13

Cypress cubre ahora los recorridos web de Acceso, Recepción, Administrador, Médico y Laboratorio (ver fila GEN-10); **no cubre** el smoke del pipeline ni una auditoría de accesibilidad automatizada; el caso "RabbitMQ detenido" no es automatizable desde Cypress (se cubre con `MessagingTests` y `scripts/evidencia_laboratorio.py`) y la caída de la API se comprobó con `scripts/web_conexion.py`.

Lo que sí existe hoy y se reutilizará: pruebas de concurrencia por API (12 solicitudes al mismo horario, 24 pacientes por 4 horarios con lecturas simultáneas), que sirven como base del escenario pero **no cuentan como prueba de rendimiento** (no miden p95 ni throughput y no usan umbrales previos). La verificación manual en navegador está descrita en `04-registro-de-hallazgos.md` y tampoco sustituye a Cypress.

Funcionales

ID	Requisito (fuente)	Pantallas	Implementación	Pruebas previstas	Entrega	Estado
RF-01	Gestión de pacientes y médicos (P2)	WEB-05…08	`PatientsController`, `DoctorsController`, `Pages/Pacientes`, `Pages/Medicos`	`PatientsApiTests`, `DoctorsApiTests`, `PatientRulesTests`; Cypress pendiente	3	◐
RF-02	Agenda y reserva de citas (P2)	WEB-02, 03, 04, 09, 11; MOB-03…07 (app: `Pages/Appointments*`, `Doctors`, `Slots`, `Confirm`)	`AppointmentsController`, `DoctorSchedulesController`, `Pages/Agenda`, `Pages/MiAgenda`, `Pages/Medicos/Disponibilidad`	`AppointmentsApiTests`, `AvailabilityApiTests`, `SchedulingRulesTests`; Cypress: recorrido de Recepción (`recepcion-reserva.cy.js`, 8 pruebas); Maestro `tests/maestro`	4	✓
RF-03	Consultas médicas (P2)	WEB-12, 13	`ClinicalController` (consultas), `Pages/Pacientes/Expediente`, `RegistrarConsulta`; entidad `Consultation` (una por cita, con paciente, médico y cita)	`ClinicalApiTests`, `ResourceAuthorizationTests`; Cypress `medico-expediente.cy.js`	5	◐ (la lectura por el paciente no está en el diseño)
RF-04	Recetas (P2)	WEB-14, 15; MOB-10, 11 (app: `Pages/Prescription*`)	`ClinicalController` (recetas con 1 a 10 medicamentos), `Pages/Pacientes/CrearReceta`, `Receta`; `GET /api/prescriptions/me` para el móvil	`ClinicalApiTests`, `ResourceAuthorizationTests`; Cypress `medico-expediente.cy.js`; Maestro `tests/maestro`	5, 9	✓
RF-05	Registro/consulta de resultados de laboratorio (P2)	WEB-16…18; MOB-08, 09 (app: `Pages/LabResult*`)	`LabsController`, `LabResultAuthorizationHandler`, `Pages/Laboratorio`, pestaña Laboratorio de `Pages/Pacientes/Expediente`	`LaboratoryApiTests` (registro, validación, roles, IDOR, atomicidad, reintento, auditoría sin contenido), `LabResultAuthorizationTests`, `LabResultRequestValidationTests`; Cypress `laboratorio-resultados.cy.js`; Insomnia y Maestro pendientes	6	✓ (web y API; móvil pendiente)
RF-06	Notificaciones al paciente (P2)	WEB-18; MOB-12 (app: `Pages/Notifications Page`)	`NotificationsController` (`GET /api/notifications`, `unread-count`, `POST /{id}/read`), `Notification`, `NotificationConsumerService`	`LaboratoryApiTests` (notificaciones propias y ajenas), `MessagingTests` (notificación asíncrona real); Maestro `tests/maestro`	6	✓
RF-07	Aplicación móvil para pacientes (P2)	MOB-01…12	`src/MediCore.Mobile.Core` (cliente, sesión, ViewModels), `src/MediCore.Mobile` (vistas MAUI)	`MediCore.Mobile.Tests` (50), `MobileApiIntegrationTests` (17, API real); **Maestro 8/8 en emulador Android**	7	✓ (8/8 en emulador; sin dispositivo físico)

ID	Requisito (fuente)	Pantallas	Implementación	Pruebas previstas	Entrega	Estado
RF-08	Usuarios y roles (diseño)	WEB-10	`UsersController`, `Pages/Usuarios.cshtml`	`IdentityApiTests` (RBAC, cambio de rol); Cypress `administrador.cy.js` (cambio de rol y su efecto, cuenta propia, acceso por rol)	2	✓

API de referencia

ID	Endpoint	Implementación	Pruebas previstas	Entrega	Estado
API-00	`GET /api/health` y `/api/health/ready` (técnico)	`src/MediCore.Api/Controllers/HealthController.cs`	`BasicEndpointTests`, `PostgresIntegrationTests` (API↔PostgreSQL)	0–1	✓
API-A1	`POST /api/auth/login`, `/refresh`, `/logout`, `GET /api/auth/me` (base de MOB-01)	`AuthController`	`IdentityApiTests`	2	✓
API-A2	`GET /api/users`, `PUT /api/users/{id}/role`, `GET /api/audit` (administrador)	`UsersController`, `AuditController`	`IdentityApiTests`	2	✓
API-01	`GET /api/doctors` (+ `GET/POST/PUT /api/doctors/{id}`, administrador)	`DoctorsController`	`DoctorsApiTests`; Insomnia pendiente	3	✓
API-02	`GET /api/doctors/{id}/availability?date=` (+ `GET/PUT /api/doctors/{id}/schedule`)	`DoctorSchedulesController`, `AvailabilityService`	`AvailabilityApiTests`; Insomnia pendiente	4	✓
API-03	`POST /api/appointments` (+ `GET` agenda, `GET /{id}`, `GET /me`, `POST /{id}/cancel`) con `Idempotency-Key`	`AppointmentsController`	`AppointmentsApiTests` (concurrencia, idempotencia, cancelación, autorización, datos inválidos); Insomnia pendiente	4	✓
API-04	`GET /api/patients/{id}` (+ listado/búsqueda, `me`, alta y edición)	`PatientsController`, `PatientAuthorizationHandler`	`PatientsApiTests` (IDOR, roles, duplicados, concurrencia); Insomnia pendiente	3	✓
API-05	`POST /api/labs/results` (+ listado, detalle, `labs/patients`, `me`, `patients/{id}/lab-results`, reintento de notificación)	`LabsController`, `LabResultAuthorizationHandler`	`LaboratoryApiTests` (43 casos), `MessagingTests` (caída de RabbitMQ); evidencia `documentacion/evidencias/entrega-6-laboratorio.md`; Insomnia pendiente	6	✓
API-06	`GET /api/prescriptions/{id}` (+ crear, listar por paciente y `me`) y consultas	`ClinicalController`, `PrescriptionAuthorizationHandler`	`ClinicalApiTests` (autor y paciente dueño; terceros 404; roles 403); Insomnia pendiente	5	✓

Redis y RabbitMQ

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
RED-01	Cache de disponibilidad médica (TTL, invalidación al reservar/cancelar o cambiar el horario)	`Infrastructure/Scheduling/AvailabilityService.cs`, `CacheKeys.Availability`, `RemoveByPrefixAsync`	`AvailabilityApiTests`: MISS, HIT, TTL, invalidación, prefijo por médico	4	✓
RED-02	Cache de catálogos y consultas frecuentes	Catálogo de médicos: `Infrastructure/Caching/CacheService.cs`, TTL 5 min, invalida al crear/editar	`DoctorsApiTests`: MISS, HIT, TTL, invalidación	3–4	◐ (falta disponibilidad: entrega 4)
RED-03	Sesiones en Redis	`Infrastructure/Identity/SessionStore.cs`	`IdentityApiTests`: logout, refresh, reutilización, Redis caído	2	✓
RED-04	Recuperación si Redis no está disponible (degradar a BD)	El cache falla abierto y consulta PostgreSQL (catálogo y disponibilidad); las sesiones fallan cerrado (RED-03)	`DoctorsApiTests` y `AvailabilityApiTests` (consulta y reserva con Redis inalcanzable), `IdentityApiTests`	3–4	✓
MQ-01	Eventos AppointmentCreated / AppointmentCancelled	`AppointmentsController` → outbox en la misma transacción → `PatientEventHandler`	`PatientEventsTests` (RabbitMQ real: reserva, cancelación concurrente, broker detenido, duplicado)	7	✓
MQ-02	LabResultReady, PrescriptionCreated, PatientNotification	Los tres por outbox; cada notificación creada emite `PatientNotification` en su misma transacción y su consumo marca `DispatchedAtUtc`	`MessagingTests`, `PatientEventsTests`	6–7	✓
MQ-03	Consumidor con reintentos, DLQ e idempotencia	`NotificationConsumerService`, `LabResultReadyHandler`, topología en `Messaging/Topology.cs` (reintento con espera por mensaje, DLQ), unicidad en BD de `Notification`	`MessagingTests`: fallo dos veces y luego procesa, fallo permanente → DLQ y recuperación con "reintentar", mensaje ilegible o inexistente → DLQ directa, mensaje reentregado y evento republicado sin duplicar	6–7	✓ (para `LabResultReady`; se generaliza en la entrega 7)
MQ-04	Caída del broker no pierde datos clínicos (outbox)	Resultado + evento en una sola transacción (`LabsController.Create`); `OutboxPublisherService` con confirmaciones, `FOR UPDATE SKIP LOCKED` y espera creciente; migración `LaboratorioYNotificaciones`	`MessagingTests.Con_el_broker_detenido…` (Testcontainers) y `scripts/evidencia_laboratorio.py` (contra el RabbitMQ de compose: detener, registrar, recuperar), `LaboratoryApiTests.Resultado_y_evento_se_guardan_juntos_o_ninguno`	6	✓

Escenario crítico y seguridad

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
CRI-01	Sin doble reserva ni colisión de horario bajo alta demanda	Restricciones de exclusión en PostgreSQL (médico y paciente) + candados de asesoría por horario + reintento ante deadlock	`AppointmentsApiTests`: 12 solicitudes al mismo horario → 1 reserva; 24 pacientes por 4 horarios + lecturas; mismo paciente con 6 médicos; reenvíos simultáneos con la misma clave. Carga con k6 pendiente (entrega 13)	4, 13	◐
CRI-02	Consistencia con lecturas concurrentes de agenda/expediente	`tests/k6/medicore.js`, `scripts/perf-integridad.sh`	k6 mixto: 0 errores inesperados y 0 solapes	13	✓
CRI-03	RabbitMQ interrumpido después de registrar un resultado	Ver MQ-04	`MessagingTests.Con_el_broker_detenido…`, evidencia `documentacion/evidencias/entrega-6-laboratorio.md` (sección B)	6	✓
SEC-01	RBAC por rol clínico/administrativo	`[Authorize(Roles)]`, política fallback, `Contracts/Roles.cs`	`IdentityApiTests.Solo_el_administrador…`, `BasicEndpointTests`	2	◐ (roles clínicos con sus módulos)
SEC-02	Autorización a nivel de recurso (IDOR)	Pacientes, citas, recetas, consultas y resultados de laboratorio: `PatientAuthorizationHandler`, `AppointmentAuthorizationHandler`, `PrescriptionAuthorizationHandler`, `LabResultAuthorizationHandler` y `ICareRelationship`. **Laboratorio:** Laboratorio lee todos los resultados, el médico solo con relación válida, el paciente solo los suyos, Recepción y Administrador nunca (403); ajeno → 404 + `clinical.access_denied`. **Expediente compartido:** el médico con relación de atención válida (cita no cancelada o consulta registrada) lee los registros de otros médicos; sin relación, 404; nadie modifica lo de otro (solo el autor crea recetas sobre su consulta; no hay edición ni borrado); 404 y auditoría ante intento ajeno	`PatientsApiTests`, `AppointmentsApiTests`, `ClinicalApiTests` (`Expediente_compartido_*`), `ResourceAuthorizationTests` (unitarias, sin BD); Cypress `medico-expediente.cy.js` (colega)	3–6	◐ (falta laboratorio: entrega 6)

ID	Requisito	Implementación	Pruebas previstas	Entrega	Estado
SEC-03	Exposición de datos sensibles (respuestas, logs, errores)	Información clínica solo para quien corresponde; la auditoría y las bitácoras no contienen motivo, notas, diagnóstico ni medicamentos; el paciente no ve contacto de médicos; ProblemDetails	`ClinicalApiTests.Ni_la_auditoria_ni_las_bitacoras_contienen_el_contenido_clinico` y `LaboratoryApiTests.Ni_la_auditoria_ni_las_bitacoras_contienen_el_contenido_del_resultado` (con control del capturador); el evento del outbox lleva solo identificadores, `DoctorsApiTests`; revisión ZAP pendiente	3–6	◐
SEC-04	Sesiones y autenticación (expiración, bloqueo, logout)	`AuthController`, JWT, `User` (bloqueo)	`IdentityApiTests` (expiración, manipulación, bloqueo, logout), `UserTests`; Cypress negativo pendiente	2	✓
SEC-05	Seguridad de API (validación, rate limit, CORS, cabeceras)	ProblemDetails, validación de entrada, rate limit en login/refresh, CSP y cabeceras en la web	`IdentityApiTests.Login_se_limita_por_ip`; Insomnia y ZAP pendientes	1–2	◐
SEC-06	Trazabilidad y auditoría de accesos y cambios clínicos	`audit_entries`, `IAuditLogger`, `GET /api/audit`	`IdentityApiTests`; pacientes y médicos (`PatientsApiTests`); citas: creadas, reenviadas, conflictos, canceladas, leídas, agenda consultada, accesos denegados, horario cambiado (`AppointmentsApiTests`, `AvailabilityApiTests`)	2–6	◐
SEC-07	Secretos fuera del repositorio	`.gitignore`, `.env.example`	Revisión + escaneo en pipeline	0	◐

Generales / DevSecOps / documentos

ID	Requisito	Implementación	Entrega	Estado
GEN-01	Repo GitHub: ramas, PR, Issues, Releases, protección de `main`	Remoto en GitHub, ramas `feature/*` y PR #1 fusionado; falta protección de `main`, Issues y Releases	0, 11	◐
GEN-02	Jenkinsfile con las 18 etapas + bloqueo por Quality Gate	`Jenkinsfile`, `ci/`, `scripts/quality-gate.py`; 18 etapas ejecutadas (corrida 5) y bloqueo demostrado (corrida 6), `reports/jenkins/`	11	✓
GEN-03	Dockerfiles + docker-compose	`docker/{api,web,worker,migrator}.Dockerfile`, `docker-compose.yml` (dependencias) + `docker-compose.qa.yml` (aplicación, proxy `deploy/caddy/Caddyfile`), `scripts/qa.sh`; verificado: ambiente levantado, humo, persistencia (`documentacion/evidencias/entrega-8-verificacion.md`)	1, 8	✓ (local)
GEN-04	Azure + Terraform (aprovisionar, modificar, destruir)	`infrastructure/terraform/{bootstrap,azure}` (estado remoto protegido, VM, red, NSG, IP, discos). Ciclo aprovisionar, modificar y destruir ejecutado con proveedor simulado (`documentacion/evidencias/iac-sin-costo.md`). **Sin despliegue real**: el equipo decidió no usar recursos de pago	8, 12	◐
GEN-05	Ansible (SO, Docker, reverse proxy)	`infrastructure/ansible` (roles `base`, `docker`, `medicore`; proxy Caddy en el Compose). Aplicado sin fallas sobre un Ubuntu 24.04 desechable, con la aplicación funcionando (`documentacion/evidencias/iac-sin-costo.md`). **Sin servidor en Azure**	8, 12	◐
GEN-06	API REST documentada (OpenAPI)	`AddOpenApi` + `/openapi/v1.json` en Development (sin esquema Bearer ni comentarios aún)	1	◐
GEN-07	SonarQube + Quality Gate	Puerta "MediCore" aprobada; rechazó código nuevo en las corridas 2, 7 y 8, y aprobó la 9; `reports/sonarqube/`	11	✓
GEN-08	Fortify, Dependency-Check, npm audit	Dependency-Check, npm audit y NuGet ejecutados. **Fortify sin ejecutar: falta licencia** (`scripts/sec-fortify.sh`)	11	◐
GEN-09	OWASP ZAP sobre QA	Web y API con 5 roles: 0 alertas altas, 40 de 45 operaciones alcanzadas; `reports/zap/`	11	✓
GEN-10	Cypress (smoke, crítico, regresión, negativo)	`tests/cypress/e2e/*.cy.js` (10 specs; ver fila superior y `tests/cypress/README.md`)	10	◐

ID	Requisito	Implementación	Entrega	Estado
GEN-11	Maestro (5–8 flujos)	`tests/maestro/*.yaml` (8 flujos), `scripts/maestro_run.py` — 8/8 con el código final en emulador, contra QA	7, 10	✓
GEN-12	k6/JMeter: Load, Stress, Spike con umbrales previos	`tests/k6/medicore.js`, `documentacion/07-rendimiento.md`, `reports/k6/`; umbrales cumplidos, solo en equipo local	13	✓
GEN-13	Usabilidad (≥5 participantes) + Lighthouse/axe	axe: 14 pantallas, 0 infracciones. **Usabilidad definida en `08-usabilidad.md`, sin aplicar a personas**	13	◐
GEN-14	Registro de defectos con evidencia	`documentacion/12-registro-de-defectos.md`, plantilla `.github/ISSUE_TEMPLATE/defecto.yml`. 26 Issues creados en GitHub con `scripts/crear-issues.py` (23 cerrados y 3 abiertos, una vez cerrado H8-08)	continuo	◐
GEN-15	Plan de pruebas (20 secciones), riesgos C/I/D, casos, informe GO / NO-GO	`10-plan-de-pruebas.md`, `09-matriz-de-riesgos.md`, `11-casos-de-prueba.md`, `13-informe-final.md`	14	✓
GEN-16	Ambiente reproducible desde código y documentación	README + `documentacion/06-infraestructura.md` + `scripts/qa.sh` + Compose + migraciones por contenedor; reproducido localmente desde cero. Falta reproducirlo en Azure	1, 8, 12	◐
GEN-18	Respaldos y restauración (R-16, H8-08; requisito de continuidad)	`deploy/respaldos/respaldo.sh`, `scripts/simulacro-respaldo.sh`, `documentacion/evidencias/respaldos.md`; 14 tablas idénticas tras pérdida total	14	✓
GEN-17	Insomnia: colección +/−/authz	`tests/insomnia/medicore.insomnia.json`, `scripts/api-tests.sh`; 61 solicitudes, 65/65	10	✓

MediCore · Registro de hallazgos y verificación manual

Registro de los errores que aparecieron al construir cada entrega, su causa, la corrección y cómo se comprobó. Es el punto de partida del registro formal de defectos (entregable de la entrega 14: pasos, esperado, obtenido, severidad, prioridad y evidencia).

Tipos: **Producto** (defecto del sistema), **Prueba** (defecto de la prueba o de su dato), **Entorno** (herramienta o infraestructura local).

Entrega 4 · disponibilidad, agenda y citas

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H4-01	Producto	Con 12 reservas simultáneas al mismo horario, 11 respondían 500 en lugar de 409 (`40P01: deadlock detected`).	Las restricciones de exclusión de PostgreSQL insertan la fila y luego buscan conflictos; dos inserciones en conflicto se ven mutuamente sin confirmar y se bloquean entre sí.	La alta se serializa con `pg_advisory_xact_lock` por horario del médico y por paciente dentro de una transacción, y se reintenta (hasta 3 veces) ante 40P01/40001. Las restricciones siguen siendo la garantía final.	Las pruebas de concurrencia pasan: 1 reserva y 11 conflictos limpios. Repetidas 3 veces seguidas sin fallos.
H4-02	Producto	`GET /api/health` y las rutas protegidas respondían 500 en pruebas sin base de datos (3 pruebas fallaron).	`PatientAuthorizationHandler` recibía `MediCoreDbContext` por constructor y ASP.NET lo instancia en cada autorización; sin cadena de conexión, crear el contexto lanzaba excepción.	El manejador recibe `IServiceProvider` y obtiene el contexto solo cuando evalúa al médico.	`BasicEndpointTests` pasan; la API vuelve a arrancar y responder sin PostgreSQL configurado.
H4-03	Producto (web)	Toda página con `@section Scripts` (Agenda, Mi agenda, detalle de cita) daba 500: "sections defined but not rendered".	`_Shell.cshtml` es un layout anidado y no reenviaba la sección al layout base.	`_Shell` declara `@section Scripts` y la reenvía con `RenderSectionAsync`.	Agenda, Mi agenda y detalle cargan; comprobado en navegador.
H4-04	Producto (web)	En "Reservar cita", tras cambiar paciente o fecha, **Confirmar cita** no reservaba y la página quedaba igual.	El formulario no tenía `action`: después de "Actualizar horarios" la URL conservaba `?handler=Actualizar` y el envío volvía a ese manejador.	`action` explícito a `/Agenda/Reservar`.	Ver H4-05.
H4-05	Producto (web)	Con el `action` explícito, el envío daba una página vacía (HTTP 400).	Al declarar `action` a mano el *tag helper* no genera el token antifalsificación.	`asp-antiforgery="true"` en el formulario.	La reserva se completa y redirige al detalle de la cita; comprobado en navegador (envío con JavaScript, ver más abajo).

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H4-06	Prueba	`Administrador_configura_y_consulta_el_horario…` fallaba aunque los valores eran iguales.	Comparaba tuplas que contenían un arreglo (se comparan por referencia).	Se comparan por separado el arreglo de días y el resto.	Pasa.
H4-07	Prueba	`No_se_puede_cancelar_una_cita_que_ya_paso` lanzaba `DbUpdateException` al preparar el dato.	Se insertaba un `DateTime` con `Kind=Utc` en una columna `timestamp without time zone` (Npgsql lo rechaza).	El asistente de pruebas fuerza `DateTimeKind.Unspecified`.	Pasa.
H4-08	Prueba	Una aserción de `Alta_demanda…` estaba mal escrita (expresión sin sentido) y habría dado un resultado engañoso.	Error al redactar la prueba.	Detectada al revisarla antes de depender de ella; reescrita con conteos explícitos (4 creadas, 20 conflictos, 20 lecturas 200).	Pasa y es legible.
H4-09	Producto (datos de desarrollo)	Advertencia de EF Core `10103` (`First` sin `OrderBy`) al sembrar las citas de demostración.	Consulta sin orden determinista en `ClinicalDemoSeeder`.	`OrderBy` por correo normalizado.	Sin advertencia.
H4-10	Entorno	Al empezar, `/api/health/ready` respondía `unavailable`; los contenedores estaban detenidos.	Docker Desktop reiniciado; no es un defecto del sistema.	`docker compose up -d`.	PostgreSQL, Redis y RabbitMQ saludables.
H4-11	Entorno	Capturas y clics del panel del navegador fallaban a veces ("Screenshot timed out").	El panel no terminaba de dibujar con la ventana en segundo plano.	Se reintentó y, cuando no hubo forma de hacer el clic, se envió el formulario con JavaScript (ver más abajo).	Limita lo que cuenta como interacción real; ver detalle.

| H4-12 | Producto (web) | Con Cypress, escribir la fecha en el filtro de la agenda no la aplicaba (la URL conservaba la fecha anterior). | El envío automático escuchaba solo `change`. Cypress emite `input` y `change` solo al perder el foco; además, en navegadores de escritorio un campo de fecha emite `change` en cada edición válida mientras se teclea (por ejemplo, al escribir el primer dígito del año), lo que podría recargar la página a medias. Esto último es el comportamiento conocido del navegador: **no lo reproduje con un teclado físico**. | `autosubmit.js` y `reservar.js` escuchan `input` y `change` de los campos de fecha, esperan 700 ms a que termine la escritura y envían al instante al salir del campo. Las listas se siguen enviando al instante. | Cypress 8/8 en Electron y en Chrome. | | H4-13 | Prueba | Todas las pruebas fallaban en el `before` con "`Cypress.env()` was removed in Cypress version 16". | Cypress 16 eliminó `Cypress.env()`. | Valores no sensibles con `expose` y la contraseña con `cy.env()`. | Pasa. | | H4-14 | Prueba | Al usar `clear()` en la fecha, la agenda recargaba con la fecha vacía (la API toma "hoy"). | Borrar un campo de fecha lo cambia, y el envío automático lo envía. Comportamiento esperado,

no un defecto. | Las pruebas escriben la nueva fecha sin borrar antes. | Pasa. | | H4-15 | Prueba y producto (límite) | Elegir el paciente justo después de cambiar la fecha fallaba: "this element is `disabled`" (`--submitting form--`). | El navegador estaba enviando la actualización de horarios. En uso real, un cambio hecho en los pocos cientos de milisegundos de una recarga en curso se pierde. | La prueba espera cada recarga (`cy.intercept` + `cy.wait`). El límite del producto se acepta y queda documentado. | Pasa. | | H4-16 | Entorno | Cambiar los `.js` no tenía efecto y la prueba seguía fallando igual. | Los archivos estáticos se publican con huella en la compilación (`MapStaticAssets`): hay que recompilar y reiniciar la web. | Compilar antes de probar (anotado en `CLAUDE.md`). | Una ejecución perdida. | | H4-17 | Herramienta | Las capturas de Cypress salían recortadas y con barra de desplazamiento horizontal. | La ventana de Electron era menor que el *viewport* configurado (1440). | *Viewport* 1280×800. El desbordamiento horizontal se descartó midiendo `scrollWidth == clientWidth` en Agenda, Reservar y Pacientes, en Electron y en Chrome. | Capturas correctas. |

Hallazgo de diseño aceptado (no es un error): el calendario del prototipo tenía la leyenda "Seguimiento" (color azul) que no corresponde a ningún estado del sistema; se retiró y solo queda "Confirmada".

Verificación en navegador de la entrega 4: qué fue interacción real y qué fue JavaScript

Las pruebas automatizadas (129: 128 superadas y 1 omitida) no usan navegador. La revisión manual en el navegador integrado se hizo así; **no sustituye** a Cypress.

Interacción real (herramienta de navegador: teclas, clics, formularios y capturas)

Comprobación	Cómo
Inicio de sesión de Recepción (Elena)	Campos con la herramienta de formularios y envío con la **tecla Enter**.
Pantalla WEB-02 (agenda) con captura de pantalla	Navegación y captura; coincide visualmente con el diseño.
Cambio de médico en el filtro de la agenda	Valor del `select` fijado con la herramienta de formularios; la **propia página** disparó su envío automático y mostró las 4 citas de la Dra. Méndez.
Reservar cita: elegir paciente	Igual que arriba; la página se recargó sola con los horarios.
Reservar cita: clic en el horario y en "Confirmar cita"	**Clics reales**, hechos en un intento anterior; ahí se descubrió el defecto H4-04. Después de la corrección, los clics fallaron por tiempo agotado del panel (H4-11).
Pantalla WEB-09 (disponibilidad) con captura	Captura; coincide con el diseño.
Lectura del texto de la agenda del médico y del listado de pacientes del médico	Lectura de la página (no cambia datos).

Ejecutado con JavaScript (código que inyecté en la página; no es un usuario haciendo clic)

Comprobación	Qué hice	Qué **no** queda probado
Reserva correcta de la cita de las 09:30	`form.requestSubmit(...)`	El clic en el botón.
Conflicto de horario (10:30 reservado por la API antes del envío)	`requestSubmit` y lectura del mensaje y del estado de los campos	El clic; el aspecto visual del mensaje.
Cancelación de la cita	`form.submit()`	El cuadro de confirmación "¿Confirmas que deseas cancelar esta cita?" **no se mostró ni se probó**; `submit()` lo omite.
Cierre de sesión y nuevo inicio de sesión (Lucía y administrador)	`submit()` / `requestSubmit()` con valores asignados por código	El flujo con teclado y clics.
Disponibilidad: hora final anterior a la inicial, guardado válido y restauración	Valores asignados por código y `requestSubmit`	Los clics y los campos de hora con el selector del navegador.

Verificación con Cypress, sin inyectar JavaScript (entrega 4, repetida y ampliada)

Suite: `tests/cypress/e2e/recepcion-reserva.cy.js` (8 pruebas). Todas las acciones del usuario van por la interfaz (escribir, elegir, pulsar); la API solo prepara y limpia datos y simula a *otra persona* que ocupa el horario en el paso 7. Resultado: **8 de 8 superadas**, en **Electron 146** (4 ejecuciones seguidas) y en **Chrome 155** (2 ejecuciones), todas sin intervención manual y repetibles (los datos de prueba se limpian y reutilizan).

Paso pedido	Prueba	Resultado	Qué se comprobó
1. Iniciar sesión como Recepción	1	✓	Escribir correo y contraseña, pulsar "Iniciar sesión"; aparece "Elena Vargas · Recepción".
2. Elegir médico, fecha, paciente y horario	2 y 3	✓	Elegir médico y fecha en la agenda (la página se actualiza sola), "Reservar cita", cambiar la fecha en el formulario y volver (los horarios se actualizan), elegir paciente y pulsar el horario 09:00.
3. Confirmar con el botón	2 y 3	✓	Botón "Confirmar cita →"; abre el detalle con aviso "Cita confirmada correctamente", paciente, médico, hora 09:00 y estado Confirmada.
4. Abrir "Ver detalle"	4	✓	Desde la agenda, "Ver detalle ↗" de la cita lleva a la misma cita.
5. Cancelar rechazando la confirmación	5a	✓	El cuadro pide exactamente "¿Confirmas que deseas cancelar esta cita?", se responde que no, **no se envía ninguna petición**, el estado sigue Confirmada y la cita sigue en la agenda.
5. Cancelar aceptando	5b	✓	Se envía la petición, aviso "La cita fue cancelada", estado Cancelada y desaparece el botón.
6. El horario vuelve a estar disponible	6	✓	En la agenda 09:00 muestra " + Horario disponible" sin el paciente, y al pulsarlo el formulario trae 09:00 habilitado y elegido.
7. Conflicto de horario conservando lo ingresado	7	✓	Otra persona reserva 10:00 mientras se llena el formulario; al confirmar aparece "Este horario ya está ocupado. Selecciona otro para continuar.", **se conservan paciente, médico y fecha**, el horario 10:00 queda deshabilitado y sin elegir, y se puede reservar 10:30 sin empezar de nuevo.

Paso pedido	Prueba	Resultado	Qué se comprobó
8. Cambiar de mes en el calendario	8	✓	"Mes siguiente" muestra el mes siguiente con el día 1 marcado, otra vez el siguiente, "Mes anterior" dos veces regresa, y elegir el día 15 cambia la fecha de la agenda.

Evidencia: `reports/cypress/resultados-electron.xml`, `reports/cypress/resultados-chrome.xml` y capturas `reports/cypress/screenshots/recepcion-reserva.cy.js/01…08` (de la ejecución en Chrome).

Límites de esta verificación (no se marca como probado):

- Cypress dispara eventos del navegador; **no es una persona con teclado y ratón físicos**. En particular, escribir la fecha con teclado real (H4-12) no se reprodujo.
- El cuadro `confirm()` lo atiende Cypress: se comprobó que la página lo pide con el texto correcto y que respeta la respuesta, no su aspecto.
- Ejecución *headless*; no se midió accesibilidad (Lighthouse/axe), teclado, lector de pantalla ni móvil.
- Solo cubre Recepción. Médico, Administrador, pantallas de pacientes y médicos, smoke general y regresión siguen pendientes.

Con esta suite quedan **sustituidas** las comprobaciones con JavaScript de reserva, conflicto y cancelación de la sección anterior; las de cierre de sesión, inicio de sesión del médico y del administrador y de la pantalla de disponibilidad siguen siendo solo las descritas arriba (JavaScript).

Lo que no se verificó en navegador

- Aún sin verificar con interacción real: cierre de sesión, inicio de sesión de Médico y Administrador, y la pantalla de disponibilidad (WEB-09), que solo se probaron con JavaScript.
- Navegación solo con teclado, lector de pantalla, vista móvil y pantallas pequeñas.
- La web con la sesión de Recepción tras cambiar la zona horaria o cerca de medianoche.

Falta ampliar Cypress (smoke, otros roles, regresión y más escenarios negativos) y la revisión de accesibilidad con Lighthouse o axe.

Entrega 5 · expediente, consultas y recetas

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-01	Entorno / proceso	Se indicó que el PR de la entrega 4 estaba fusionado, pero `origin/main` seguía en la fusión del PR #3 (`00e510d`).	La fusión no llegó a `origin/main` (no se verifica con `gh` ; se comprobó con `git ls-remote`).	La rama `feature/entrega-5-consultas-recetas` se creó **sobre** `feature/entrega-4-agenda-citas` (rama apilada) para no perder la entrega 4.	Al fusionar el PR de la entrega 4, el de la 5 mostrará solo sus cambios. **Pendiente de tu confirmación.**
H5-02	Prueba	Al ejecutar varias suites seguidas, las preparaciones por API fallaban con `429 Too Many Requests` y todo quedaba en 0 pruebas.	El límite de intentos de inicio de sesión (10 por minuto por IP) funcionó como debe; las suites hacen muchos inicios de sesión.	En Development el límite sube a 60 por minuto (`appsettings.Development.json`); en otros ambientes sigue en 10.	15/15 pruebas, 4 ejecuciones seguidas por navegador.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-03	Herramienta	La captura de pantalla de Electron se colgaba de forma intermitente (cerca de 2 de cada 6 ejecuciones) y hacía fallar pruebas funcionales que sí pasaban.	Electron está obsoleto en Cypress 16; el problema es de la captura, no de la aplicación.	Las capturas de evidencia se toman solo en navegadores reales (Chrome); en Electron se ejecutan todas las aserciones.	Sin fallos en 4 ejecuciones seguidas en Electron.
H5-04	Producto (web)	Abrir el expediente de un paciente sin relación mostraba una página en blanco (404 sin cuerpo).	Las respuestas vacías no tenían página.	Página `/Estado` con `UseStatusCodePagesWithReExecute`: mensaje en español y se conserva el código HTTP.	Cypress paso 8: estado 404 y "No encontramos lo que buscas", sin datos del paciente.
H5-05	Prueba	Riesgo de falso negativo en la prueba de bitácoras: si el capturador no recibiera nada, "no contiene el dato" pasaría igual.	—	Se agregó un control: se escribe un texto conocido en la bitácora y se exige que el capturador lo vea antes de comprobar que el contenido clínico no aparece.	Pasa y es significativa.
H5-06	Herramienta	Una edición de archivo fue rechazada ("modified since read") tras cambiar de rama.	El archivo cambió al cambiar de rama.	Se volvió a leer y a escribir.	Sin efecto en el resultado.

Decisiones y desviaciones del diseño en esta entrega (no son errores):

- **Varios medicamentos (WEB-14):** el diseño muestra un solo medicamento; se agregó " + Agregar medicamento" y "Quitar este medicamento" (hasta 10), cada uno con dosis, frecuencia, duración e indicaciones, como se pidió. Funciona sin JavaScript.
- **Cita atendida (WEB-13):** se agregó la lista "Cita atendida" para asociar la consulta a la cita (requisito); se propone la cita desde la que se abrió el expediente.
- **"Crear receta ↗"** en cada consulta del expediente (el diseño solo ofrece "Guardar y crear receta" al registrar).
- **Laboratorio:** el diseño dice "Sin resultados registrados"; como el módulo no existe, se indica que está **pendiente** y no se muestra nada.
- **Qué ve el médico:** en el expediente solo lo que él escribió, no las notas de otros médicos del mismo paciente (mínimo privilegio). Es una decisión de seguridad que puedes cambiar.

Verificación de la entrega 5

- **Cypress `medico-expediente.cy.js` (10 pruebas tras el ajuste del expediente compartido), sin inyectar JavaScript:** iniciar sesión como Médico; abrir el expediente desde la agenda; recorrer Consultas, Recetas y Laboratorio (pendiente); guardar una consulta incompleta (errores); registrar la consulta; crear una receta con dos medicamentos (agregar y quitar filas conservando lo escrito); ver el detalle; volver al expediente y abrir la receta desde su tarjeta; comprobar que la cita ya no admite otra consulta; y que un paciente sin relación devuelve 404. **10/10 y la regresión de Recepción 8/8, en Chrome 155 y Electron 146** (una ejecución por navegador tras el ajuste; en la primera ejecución la prueba 10 falló porque `cy.visit` rechaza el 403 de Acceso denegado: se corrigió la prueba con `failOnStatusCode: false`, no la aplicación). No se repitió a distintas horas del día; la independencia horaria se debe al diseño (cita pasada insertada directamente).

- **Regresión de Recepción `recepcion-reserva.cy.js` (8 pruebas):** 8/8 en ambos navegadores, tras los cambios de esta entrega.
- **Límites:** Cypress dispara eventos del navegador (no es una persona con teclado físico); `confirm()` no interviene en este recorrido; no se midió accesibilidad ni móvil; cada ejecución agrega una consulta y una receta de prueba (la información clínica no se borra) con un texto único.

Entrega 5 · ajustes posteriores (expediente compartido y pruebas independientes de la hora)

Decisión de producto (reemplaza la de la primera versión de la entrega 5): el expediente es **compartido entre médicos autorizados**.

- Un médico con relación de atención válida con el paciente (una cita no cancelada o una consulta registrada) **lee** las consultas y recetas escritas por otros médicos; cada registro indica su autor (`IsOwn`, `DoctorName`) y la lectura de lo ajeno se audita como `other_author`, sin contenido.
- **No puede modificar** registros de otro profesional: solo el autor crea recetas sobre su consulta, solo se registra la consulta de una cita propia y no existe operación de edición ni de borrado de consultas ni de recetas (PUT, PATCH y DELETE devuelven 405, también para el autor).
- Sin relación válida no hay acceso (404 y `clinical.access_denied`); al cancelarse la última cita sin consulta se pierde la lectura. Recepción, Administrador y Laboratorio mantienen sus restricciones; el paciente lee solo sus recetas, de cualquiera de sus médicos.
- Motivo y riesgo: la relación de atención es una barrera baja (basta una cita no cancelada, incluso futura, con ese médico). Con el expediente compartido, esa cita da lectura de todo el historial clínico del paciente. Se acepta por indicación del equipo; si se quisiera endurecer, bastaría exigir una cita ya atendida o una consulta, cambiando `PatientAccess.HasCareRelationship` (una sola definición).

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-07	Prueba	El spec de Cypress del médico dependía de la hora del día: necesitaba un horario libre **más tarde hoy** y fallaba en los últimos minutos del día.	Para registrar una consulta hace falta una cita de hoy o anterior; la API no permite reservar en el pasado, así que la prueba reservaba "hoy, más tarde".	Cita ya pasada creada con una tarea de Cypress directamente en la base de datos **local** (`db:createPastAppointment`; se niega a conectar con cualquier servidor que no sea local). No se debilita ninguna validación de la API.	La prueba no mira el reloj.
H5-08	Prueba	La prueba del médico **cambiaba temporalmente el horario de la Dra. Méndez** (un médico de uso normal) y lo restauraba al final.	Necesitaba que atendiera todo el día.	Dos médicos dedicados a Cypress, sembrados solo en Development (`e2e.medico` y `e2e.colega`, ver `E2eFixtureSeeder`). Ningún médico de uso normal se toca.	Sin cambios en agendas reales.
H5-09	Prueba	Cada ejecución del spec del médico dejaba una consulta y una receta nuevas.	La información clínica no se borra por la API.	Tarea `db:cleanE2eClinical` al empezar y al terminar: borra consultas, recetas y citas **solo** de los pacientes `CYP-E2E-*`.	Sin acumulación; el estado vacío del expediente ahora se comprueba.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H5-10	Producto (diseño de código)	Con el expediente compartido, `PrescriptionAuthorizationHandler` necesitaba la relación de atención y la consulta estaba escrita dentro de otro manejador.	Duplicación.	Servicio `ICareRelationship` (usado por ambos manejadores; mismo criterio que el listado de pacientes). Las pruebas unitarias lo simulan sin base de datos.	31 pruebas unitarias.

Entrega 6 · laboratorio y notificaciones asíncronas

Qué se implementó, qué se ejecutó y qué no

Implementado: resultados de laboratorio (dominio, migración `LaboratorioYNotificaciones`, `LabsController`, `LabResultAuthorizationHandler`), pantallas WEB-16/17/18 y la pestaña Laboratorio del expediente, outbox transaccional con publicador en segundo plano, consumidor de `LabResultReady` con reintentos, DLQ e idempotencia, API de notificaciones del paciente (para la futura app móvil), scripts de ejecución en contenedor Linux y de evidencia.

Ejecutado (todo con resultado real):

Qué	Dónde	Resultado
`dotnet test MediCore.slnx` (`scripts/dotnet-linux.sh test MediCore.slnx`)	contenedor Linux `dotnet/sdk:10.0`, Testcontainers con PostgreSQL 17, Redis 7 y RabbitMQ 4	**226/226**: 44 unitarias y 182 de integración (43 de `LaboratoryApiTests`, 8 de `MessagingTests`)
Cypress `laboratorio-resultados`, `medico-expediente`, `recepcion-reserva`	Chrome 155 y Electron 146, contra la API y la web en contenedores Linux conectados al compose real	**26/26** en cada navegador (8 + 10 + 8)
`scripts/evidencia_laboratorio.py`	RabbitMQ real de compose (se detiene y se vuelve a iniciar)	Escenarios A–D correctos; informe en `documentacion/evidencias/entrega-6-laboratorio.md`
Migración	generada en el contenedor y aplicada a la base de desarrollo al arrancar la API (solo agrega tablas y una columna; no borra datos)	aplicada sin errores

No ejecutado o no verificado: - Nada se ejecutó de forma nativa en Windows: **Smart App Control** (activo) bloquea los `.dll` sin firma recién compilados; no se desactivó (ver H6-01). El código se compila y prueba en Linux. - `dotnet format --verify-no-changes` en Linux reporta diferencias de espacios en archivos que esta entrega no tocó (por ejemplo `AgendaDay.cs`), así que no es concluyente; **no se afirma** que el formato esté verificado. - No se midió rendimiento (k6/JMeter pendiente), accesibilidad ni uso móvil; no hay Insomnia ni Maestro. - El caso "RabbitMQ detenido" no se automatiza en Cypress (no controla Docker); se cubre con `MessagingTests` y con el script de evidencia. - No se probó la pérdida definitiva de la conexión con el broker más allá de detenerlo y volver a iniciarlo, ni varias instancias de la API publicando a la vez (la consulta usa `FOR UPDATE SKIP LOCKED`, pero no hay prueba con dos procesos). - Cambio de `docker-compose.yml` (nombre de nodo fijo `medicore-rabbitmq`) **sin aplicar**: el contenedor de RabbitMQ en ejecución no

se recreó para no tocar los datos existentes; al recrearlo, RabbitMQ empieza con un almacén de colas vacío dentro del mismo volumen (los datos anteriores no se borran).

Decisiones y adaptaciones al diseño

- **Quién ve qué en laboratorio:** Laboratorio ve todos los resultados (equipo compartido; decisión revisable hacia "solo los propios"); el Médico, solo con relación válida de atención; el Paciente, solo los suyos; Recepción y Administrador, ninguno. Acceso ajeno → 404 (no confirma existencia) + `clinical.access_denied`; rol sin permiso → 403.
- **Resultados inmutables:** no hay edición ni borrado (PUT/PATCH/DELETE → 405); una corrección es un resultado nuevo, como las recetas.
- **WEB-17:** el diseño tiene un solo parámetro; se permiten de 1 a 10 (filas que se agregan y quitan sin perder lo escrito, también sin JavaScript). Unidad y rango son opcionales (hay resultados cualitativos). Se agrega un filtro para elegir al paciente. **No se copió** la casilla "Simular envío de notificación pendiente" del prototipo: la notificación pendiente es un estado real (broker caído o consumidor con fallos), no una simulación.
- **WEB-18:** "Reintentar notificación" aparece solo para Laboratorio y solo si la notificación sigue pendiente. El médico ve el detalle y vuelve a su expediente.
- **Estado de la notificación:** "Enviada" = existe la notificación interna del paciente; "Pendiente" = todavía no (el resultado ya está guardado).
- **El evento lleva solo identificadores**; el título y el cuerpo de la notificación (que incluye el tipo de examen) se arman al consumirlo y se guardan en la tabla `notifications`, que solo lee el propio paciente.
- **Alcance de mensajería:** solo `LabResultReady`; los demás eventos siguen en la entrega 7 (ver plan).

Hallazgos

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H 6-01	Entorno	Migraciones, pruebas y API dejaron de cargar los `.dll` del proyecto: "Una directiva de Control de aplicaciones bloqueó este archivo" (0x800711C7).	Smart App Control (estado activo) de Windows bloquea binarios sin firma recién compilados; antes de esta sesión no bloqueaba.	**No se desactivó** (es un ajuste de seguridad del equipo). Se añadieron `scripts/dotnet-linux.sh` y `scripts/dev-stack-linux.sh`: build, migraciones, pruebas y la pila de desarrollo corren en contenedores Linux del SDK oficial, sin tocar volúmenes ni redes de compose.	Todo lo ejecutado en esta entrega usó ese camino; falta repetirlo de forma nativa en un equipo sin el bloqueo.
H 6-02	Defecto de producto (detectado por `MessagingTests`)	Tras detener y volver a iniciar RabbitMQ, el consumidor dejaba de recibir mensajes y las notificaciones no se generaban.	El publicador, al ver la conexión cerrada mientras el cliente la recuperaba, creaba una conexión nueva y cerraba la anterior, que era la del consumidor.	`RabbitMqConnection` ya no reemplaza una conexión existente (se recupera sola, incluido el consumidor).	`Con_el_broker_detenido…` y las demás pruebas de mensajería pasan; la evidencia B lo repite contra el RabbitMQ real.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H 6-03	Prueba	Tres pruebas fallaban con `operator does not exist: jsonb ~~ jsonb`.	`Payload.Contains(...)` sobre una columna `jsonb` se traduce a `LIKE`.	Ayudante `OutboxIdsForLabResultAsync` con `Payload->>'labResultId'`.	43/43 de laboratorio.
H 6-04	Prueba	`BrokerFixture` fallaba con "ports are not available ... Intento de acceso a un socket no permitido".	El puerto fijo elegido caía en un rango reservado por Windows (Hyper-V).	Puerto fijo aleatorio con reintento ante ese error (hace falta puerto fijo para detener y reiniciar el broker con la misma dirección).	8/8 de mensajería.
H 6-05	Prueba	La validación de los atributos de un `record` posicional no se detectaba con `Validator.TryValidateObject`.	Los atributos están en los parámetros del constructor, no en propiedades; ASP.NET Core sí los aplica.	Ayudante que valida el valor de cada parámetro contra sus atributos.	12 pruebas unitarias de validación.
H 6-06	Prueba existente	El recorrido 2 de Cypress del médico esperaba el aviso "laboratorio: pendiente".	La pestaña Laboratorio ahora lista resultados.	Espera "Sin resultados registrados." y el spec y la limpieza (`db:cleanE2eClinical`) incluyen resultados, notificaciones y eventos de laboratorio de los pacientes `CYP-E2E-*`.	10/10 y regresión de Recepción 8/8.
H 6-07	Riesgo abierto	`dotnet format` no es concluyente en Linux; el consumidor se probó con un solo proceso; el almacén de colas de RabbitMQ cambia al recrear el contenedor con el nuevo nombre de nodo.	Ver "No ejecutado o no verificado".	Documentado; sin corrección en esta entrega.	Abierto.

Evidencia reproducible: `documentacion/evidencias/entrega-6-laboratorio.md` (generada por `python scripts/evidencia_laboratorio.py`) y `documentacion/evidencias/entrega-6-pruebas.md` (comandos y resultados de las ejecuciones).

Entrega 7 · app móvil del paciente

Qué se implementó, compiló, probó y qué no

Estado	Detalle
Implementado	MOB-01 a MOB-12 en `src/MediCore.Mobile` (XAML + Shell) sobre `src/MediCore.Mobile.Core` (cliente de la API con renovación de tokens, ViewModels con estados de carga/vacío/error/sesión vencida, reserva con clave de idempotencia y recuperación ante horario ocupado); sesión en `SecureStorage`; configuración de red solo-HTTPS con excepción para desarrollo; 8 flujos Maestro; scripts de preparación y ejecución; documentación en `documentacion/05-app-movil.md`.
Compilado	`MediCore.Mobile.Core` y las pruebas móviles (SDK de .NET en contenedor Linux). La app MAUI se compiló a APK de desarrollo (arm64-v8a y x86_64) en un contenedor Linux con el workload `maui-android` y la plataforma Android 36: **Build succeeded**. Compilar en Linux **no equivale** a probarla en Android.

Estado	Detalle
Probado (verificado)	50 pruebas unitarias de la lógica móvil (xUnit, API simulada solo para aislar el ViewModel) y 17 pruebas del cliente contra la **API real** (PostgreSQL y Redis reales): login y rechazo del personal, renovación y rotación de tokens, cierre de sesión que revoca, reserva y cancelación, idempotencia (5 reintentos simultáneos → 1 cita), horario ocupado (409), IDOR en citas/resultados/recetas/notificaciones, notificaciones. Los 6 escenarios de datos de Maestro se prepararon con éxito contra la API real.
Probado en Android	**Sí, en el emulador `medicore_pixel` (Android 35, WHPX) con Maestro 2.12.0: 8 de 8 flujos aprobados** contra la API real en Docker (`documentacion/evidencias/entrega-7-maestro.md`). No se probó en un dispositivo físico, ni accesibilidad, rendimiento o rotación.

Herramientas (instaladas con autorización)

SDK de Android (command-line tools, platform-tools, emulator y la imagen `android-35;google_apis;x86_64` desde dl.google.com, licencias aceptadas con tu autorización) y Maestro 2.12.0 (release oficial de GitHub). No había un SDK reutilizable (Visual Studio 18 Insiders no trae uno). WHPX estaba disponible: no hizo falta reiniciar ni habilitar virtualización. El workload `maui-android` no se instaló en Windows (se compila en contenedor Linux). Smart App Control sigue activo.

Hallazgos y decisiones

ID	Tipo	Qué ocurrió	Causa / decisión	Resultado
H7-01	Entorno	No hay workload, SDK, emulador ni Maestro.	No se instaló nada en el equipo sin tu aprobación; se separó la lógica en `Mobile.Core` para probarla ahora; la compilación Android se intentó en un contenedor Linux.	Ver el reporte.
H7-02	Prueba	El token de acceso no cambia si se renueva en el mismo segundo (la prueba exigía que fuera distinto).	El JWT es determinista para la misma sesión y segundo; lo que debe rotar es el refresco.	La prueba comprueba la rotación del refresco y que reutilizar el refresco viejo revoca la sesión.
H7-03	Decisión	Las notificaciones del diseño incluyen "Cita confirmada" y "Nueva receta".	El sistema solo genera `LabResultReady` (los demás eventos están pendientes en el plan); no se inventó backend nuevo.	La app muestra cualquier notificación y abre el contenido solo cuando existe (hoy, resultados).
H7-04	Riesgo (cerrado)	Los identificadores de Maestro dependían de cómo MAUI expone `AutomationId` y `SemanticProperties`.	Los flujos usan texto visible y descripciones de accesibilidad.	**Confirmado en emulador**: `Fecha de la cita` y `Notificaciones` funcionan como selectores.
H7-06	Defecto de código (detectado al compilar)	La primera compilación real de la app falló: `DatePicker.DateSelected` entrega `DateTime?` en MAUI 10 (CS1503); además `DisplayAlert` está obsoleto.	El código MAUI se escribió sin poder compilarlo hasta tener el workload en un contenedor.	`SlotsPage` usa `e.NewDate is { } date`; el diálogo usa `DisplayAlertAsync`.
H7-07	Entorno	La compilación fallaba con NETSDK1047 (sin destino `android-x64` / `android-arm64`) y el instalador del SDK con "UserName must have a valid value".	Falta `RuntimeIdentifiers` en el restore; el contenedor no define `USER`.	`RuntimeIdentifiers` en el `.csproj` y `USER` / `USERNAME` en `android-build-linux.sh`.

ID	Tipo	Qué ocurrió	Causa / decisión	Resultado
H7-05	Riesgo abierto	Smart App Control sigue activo; no se probó compilar la app MAUI de forma nativa en Windows.	No se desactivó; la compilación se hace en contenedor Linux y la prueba en el emulador de Windows (binarios de Google firmados).	Abierto solo para la compilación nativa.
H7-08	Defecto (solo visible en Android)	La app se cerraba al abrir: `No assemblies found ... Fast Deployment`.	El APK Debug no incluía los ensamblados.	`EmbedAssembliesIntoApk=true`, `AndroidUseSharedRuntime=false` (APK ≈84 MB).
H7-09	Defecto (solo visible en Android)	Tras lo anterior: `lib/x86_64 ... MUST be STORED`.	Bibliotecas nativas comprimidas en el APK.	`android:extractNativeLibs="true"` en el manifiesto.
H7-10	Entorno	`adb install -r` fallaba con `INSTALL_FAILED_UPDATE_INCOMPATIBLE` y se seguía probando una versión vieja (parecía que los arreglos no funcionaban).	Cada compilación en contenedor genera otra clave de depuración.	Desinstalar antes de instalar (documentado).
H7-11	Prueba	En la primera pasada fallaron los flujos 01, 02 y 07.	01: el texto de la tarjeta lleva un ícono delante; 02: se intentaba tocar un campo ya escrito; 07: el emulador quedó `offline` un instante (no la app).	Selectores con expresión regular (01), formulario reiniciado (02), repetición (07).
H7-12	Herramienta	`maestro_run.py` no encontraba `maestro` en Windows y los subflujos exigían cabecera de configuración.	`maestro` es un `.bat`; Maestro exige `appId` y `---` también en subflujos.	`shutil.which` y cabecera en `subflows/login.yaml`.

Mejora pendiente (H7-13): clave de firma de depuración estable. Cada compilación en contenedor crea otra clave, por lo que hay que desinstalar la app antes de instalar un APK nuevo y se pierden sus datos locales (la sesión guardada). Pendiente: generar una vez un `debug.keystore` propio **fuera del repositorio** (por ejemplo en un volumen de Docker o en la carpeta de usuario), montarlo en `scripts/android-build-linux.sh` y configurar la firma (`AndroidSigningKeyStore`) para actualizar con `adb install -r`. No se versiona nunca la clave.

Evidencia: `documentacion/evidencias/entrega-7-pruebas.md`.

Web de extremo a extremo (2026-11-02) · Acceso, Recepción, Administrador, Médico y Laboratorio

Prioridad cambiada: se pausó Azure/Terraform/Ansible/Jenkins y la app móvil (sin borrar nada) para revisar la web del personal. Rama `feature/web-extremo-a-extremo` (desde `main`; la entrega 8 sigue sin fusionar).

Qué se ejecutó (resultados reales)

- **Cypress en Chrome 155: 102/102** (10 specs, ver `tests/cypress/README.md`) contra la API y la web en contenedores (`scripts/dev-stack-linux.sh`). No se re-ejecutó en Electron.
- **xUnit: 293/293** (44 + 50 + 199) con `scripts/dotnet-linux.sh test MediCore.slnx`.
- **Rastreo por rol** (`scripts/web_crawl.py`, solo GET): las 18 pantallas del diseño existen y responden; permisos por rol correctos; sin textos de "pendiente".
- **Caída de la API** (`scripts/web_conexion.py`, a mano, no Cypress): todas las pantallas principales responden 200 con un mensaje en español ("No se pudo cargar…"), sin trazas; la API se volvió a levantar.

Hallazgos

ID	Tipo	Qué ocurrió	Resultado
HW-01	Defecto	La página de inicio decía que las pantallas llegarían en "próximas entregas".	Accesos directos por rol. Corregido.
HW-02	Defecto	`max` de la fecha de nacimiento tenía un formato inválido y el navegador no limitaba fechas futuras.	Corregido (`yyyy-MM-dd`).
HW-03	Defecto (visible en celular)	Con el menú del Administrador (más enlaces) la página se ensanchaba a 427 px en una pantalla de 360 px.	`grid-template-columns: minmax(0,1fr)` en `app.css`; el menú se desplaza en su franja. Corregido.
HW-04	Defecto (visible en celular)	Las tablas anchas (médicos, usuarios, pacientes, resultados) podían ensanchar la página.	Se desplazan dentro de su panel. Corregido.
HW-05	Prueba	Tras elegir paciente en Reservar la página se recarga sola; la prueba actuaba antes de la recarga.	Las pruebas esperan cada recarga. No era defecto de la web.
HW-06	Prueba	La notificación del laboratorio es asíncrona: el estado "Enviada" no es inmediato.	La prueba acepta Enviada o Pendiente en el listado.
HW-07	Prueba	La tarea `db:createPendingLabResult` buscaba el correo en mayúsculas; el normalizado está en minúsculas.	Corregida.

Pendiente / no comprobado

- Auditoría de accesibilidad automatizada (axe/Lighthouse) y lector de pantalla: solo se comprobaron etiquetas, orden de teclado y desbordes con pruebas propias.
- La concurrencia real de doble reserva se cubre con xUnit/API (la prueba Cypress reserva dos veces seguidas, no en paralelo).
- Electron tras la ampliación; Firefox/Safari; navegadores móviles reales (solo viewport de 375 px).
- Insomnia, k6/JMeter, Maestro adicional, Jenkins/DevSecOps y Azure siguen pendientes (pausados por decisión).

Entrega 8 · contenedores e infraestructura

Qué se implementó, ejecutó y qué no

- **Implementado:** 4 Dockerfile (api, web, worker, migrator), proyecto `MediCore.Worker`, `docker-compose.qa.yml` sobre el Compose existente, proxy Caddy, scripts `qa.sh` / `qa-smoke.sh` / `qa-persistence.sh` / `validate-iac.sh`, Terraform (bootstrap del estado + VM de QA), Ansible (base, docker, aplicación), `documentacion/06-infraestructura.md`.
- **Ejecutado:** imágenes construidas; ambiente QA levantado desde cero; humo de web y API; persistencia tras `restart` y `down` + `up` (13/13); puertos de datos no publicados; Cypress de Recepción 8/8 a través del proxy; APK contra el ambiente en contenedores y 8/8 flujos Maestro; xUnit 293/293 tras los cambios; validación de Terraform, Ansible, hadolint y compose. Detalle: `documentacion/evidencias/entrega-8-verificacion.md`.
- **No ejecutado:** `terraform plan/apply`, `ansible-playbook` contra un servidor, HTTPS real, prueba en Azure. **La validación de archivos no se presenta como un despliegue.**

Hallazgos

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H8-01	Defecto (solo visible en contenedor)	La web quedó `unhealthy`: `UnauthorizedAccessException` al escribir en `/keys`.	El volumen nuevo es propiedad de root y la imagen corre sin privilegios.	`Dockerfile` de la web crea `/keys` con el dueño del usuario de la aplicación.	Web saludable; las claves persisten.
H8-02	Defecto (solo visible en una base nueva)	`--seed-qa` fallaba: `ClinicalDemoSeeder` no encontraba a la Dra. Méndez.	`DevelopmentBootstrap` sembraba primero los médicos de las pruebas E2E y el sembrado de demostración solo crea médicos si no hay ninguno; en desarrollo no se notaba porque ya había datos.	Se siembra primero la demostración y después los datos E2E.	Base nueva con 8 usuarios, 5 médicos y 5 pacientes.
H8-03	Prueba	Con el contenedor `-p medicore` el `name:` del archivo base podía colisionar con los volúmenes de desarrollo.	Dos archivos Compose definen `name`.	`docker-compose.qa.yml` fija `name: medicore-qa` y `scripts/qa.sh` siempre usa `-p medicore-qa`; se comprobó que los volúmenes de desarrollo no cambian.	Sin efectos en el desarrollo.
H8-04	Herramienta	`ansible-lint` marcó 2 faltas (nombre en minúscula, variable de rol sin prefijo) y `apt_repository` está obsoleto.	Primer borrador del playbook.	Corregidas; repositorio con `deb822_repository`.	Perfil *production* sin fallos.
H8-05	Prueba	En la primera pasada de Maestro contra QA fallaron 01 y 03 en 8-9 s.	`DeviceServerDiedException` del driver de Maestro en el emulador; no la app.	Se repitió la suite completa.	8/8.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
H8-06	Seguridad (decisión)	Detrás del proxy, el límite de intentos de login por IP veía siempre la IP del proxy.	Faltaban cabeceras reenviadas.	`ForwardedHeaders__Enabled` (solo en el Compose de QA, donde la API no publica puertos).	Documentado; IP real del cliente.
H8-07	Riesgo abierto	La construcción de imágenes ocurre en la VM (no hay registro) y requiere acceso al repositorio.	Evitar el costo de un registro en esta etapa.	Anotado: GHCR/ACR con el pipeline de Jenkins.	Abierto.
H8-08	Riesgo abierto	Copias de seguridad de PostgreSQL y monitoreo no existen; el disco de datos es la única protección.	Fuera del alcance de esta entrega.	Resuelto: copias cifradas automáticas y restauración probada, ver `evidencias/respaldos.md` . El monitoreo sigue pendiente.	Cerrado.
H8-09	Riesgo abierto	No hay prueba xUnit para el worker ni para `--seed-qa` (solo verificación con el ambiente).	Se verificó con el ambiente en contenedores.	Anotado.	Abierto.

Evidencia: `documentacion/evidencias/entrega-8-verificacion.md` y `entrega-8-maestro-qa.md`.

Cierre · rendimiento, seguridad y pipeline (2026-11-06)

El registro formal, con pasos, severidad y prioridad, está en 12-registro-de-defectos.md.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
HP-01	Prueba	Stress y Spike aprobaron casi sin reservas.	Load había llenado las agendas.	Cada corrida crea sus médicos, y falla si no reserva.	Corridas válidas.
HP-02	Producto	Quedaron 14 405 eventos pendientes después del pico.	El outbox publicaba 20 eventos cada 2 s.	Lote de 200 y drenado continuo.	0 pendientes.
HS-01	Producto	SonarQube encontró bugs S1751.	Bucles que salían en la primera vuelta.	Lectura corregida.	0 bugs.
HS-02	Producto	La puerta rechazó el código nuevo.	Duplicación entre web y móvil, sin pruebas.	`ProblemDetailsReader` compartido y 4 pruebas.	Puerta aprobada.
HS-03	Prueba	La puerta por defecto aprobaba sin condiciones.	Era un proyecto sin puerta asignada.	Puerta propia "MediCore".	La puerta exige condiciones reales.
HA-01	Producto	axe encontró contraste insuficiente en el calendario.	Colores del diseño.	Colores en `app.css` .	0 infracciones.

ID	Tipo	Qué ocurrió	Causa	Corrección	Resultado
HZ-01	Producto	Un carácter nulo provocaba un error 500.	PostgreSQL rechaza NUL en texto.	`InvalidTextExceptionHandler` responde 400.	Prueba de regresión.
HZ-02	Producto	ZAP reportó cabeceras faltantes.	No estaban en Caddy.	Agregadas.	0 fallas en ZAP.
HZ-04	Prueba	ZAP API recibía casi solo 401.	Probaba cerrar sesión y perdía la sesión.	Rutas de sesión excluidas.	Análisis con sesión; se completó en HZ-05.
HZ-05	Prueba	ZAP no llegaba a la lógica clínica.	Usaba solo Administrador, identificadores inventados y cuerpos que la API rechazaba.	Análisis por rol, con identificadores y cuerpos válidos. El orden importa: el análisis activo altera datos.	40 de 45 operaciones alcanzadas, 0 errores 500.
HC-01	Producto	7 archivos fallaban `dotnet format` en Jenkins.	La validación nunca se había ejecutado.	Formato aplicado.	Etapa 3 aprobada.
HJ-01	Entorno	Jenkins no tenía `docker`.	Debian separa el cliente en `docker-cli`.	Paquete correcto.	Corre etapas.
HJ-02	Entorno	El smoke fallaba en Jenkins.	`127.0.0.1` era el propio contenedor.	Jenkins entra a la red del proxy.	Etapa 10 aprobada.
HJ-03	Prueba	Insomnia falló en Jenkins sin horarios libres.	QA acumuló 761 médicos y 34 655 citas de k6 y ZAP.	QA se recrea en cada corrida y la fecha varía.	65 de 65.
HD-01	Herramienta	Dependency-Check reportó 4 críticas.	Confundió bibliotecas cliente con servidores.	Revisadas y aceptadas con motivo.	Ver reporte de seguridad.
H9-01	Producto	QA no arrancaba tras reiniciar Docker.	Jenkins crea los contenedores con rutas de su workspace.	Archivos dentro de las imágenes.	`qa.sh smoke` correcto.
H9-02	Producto	Ansible fallaba en un Ubuntu mínimo.	Faltaba `python3-debian`.	Paquete agregado.	Dos pasadas sin fallas.
H9-03	Entorno	Los 8 flujos de Maestro fallaron a la vez.	La cortina de notificaciones del emulador tapaba la app.	Cortina cerrada antes de ejecutar.	8 de 8.
HJ-04	Entorno	Jenkins perdió su historial al reiniciar Docker.	`/var/jenkins_home` vive en la VM de Docker Desktop.	Aceptado; las consolas están en `reports/jenkins/`.	Numeración reiniciada.
HE-01	Error propio	Se borraron capturas antiguas de Cypress que no estaban en Git (unos 9,7 MB).	Una limpieza con `rm -rf`.	No se pueden recuperar. Cypress genera capturas nuevas si una prueba falla.	Sin efecto en las pruebas.

Entregas anteriores

Los hallazgos de las entregas 0 a 3 se trasladarán al registro formal de defectos en la entrega 14. Los principales: login con 503 por usar `localhost` para Redis (el cliente probaba IPv6 y Docker publica solo IPv4; corregido con `127.0.0.1`), una prueba que comparaba el `traceId` de dos respuestas de error, una prueba del catálogo de médicos afectada por la caché compartida con otras pruebas (se descarta la clave antes de insertar datos directos) y la interfaz de `IAsyncLifetime` de xUnit v2.

MediCore · App móvil del paciente (.NET MAUI, Android)

Entrega 7. Pantallas MOB-01 a MOB-12 del diseño aprobado (`Referencias/medicore/diseno`), con los textos en español, sobre la **API real**. No hay respuestas simuladas en la app ni portal web del paciente.

Qué hay y dónde

Pieza	Ruta	Se compila con
Lógica de la app sin MAUI: cliente de la API, sesión, ViewModels, estados	`src/MediCore.Mobile.Core`	SDK de .NET (en `MediCore.slnx`)
App Android (vistas XAML, almacenamiento seguro, navegación)	`src/MediCore.Mobile`	workload `maui-android` + SDK de Android (**fuera** de `MediCore.slnx`)
Pruebas unitarias de la lógica (50)	`tests/MediCore.Mobile.Tests`	SDK de .NET
Pruebas del cliente contra la API real (17)	`tests/MediCore.Api.Tests/MobileApiIntegrationTests.cs`	SDK de .NET + Docker
Flujos Maestro (8)	`tests/maestro`	Maestro + dispositivo Android
Preparación de datos y ejecución de Maestro	`scripts/maestro_prepare.py`, `scripts/maestro_run.py`	Python 3

La separación `Core` / `Mobile` permite compilar y probar casi toda la lógica sin las herramientas de Android; la capa MAUI solo contiene vistas, navegación Shell y el almacenamiento seguro.

Pantallas

ID	Pantalla	Vista	ViewModel
MOB-01	Inicio de sesión	`LoginPage`	`LoginViewModel`
MOB-02	Inicio (próxima cita, accesos, cerrar sesión)	`HomePage`	`HomeViewModel`
MOB-03	Mis citas	`AppointmentsPage`	`AppointmentsViewModel`
MOB-04	Seleccionar médico (paso 1/3)	`DoctorsPage`	`BookingViewModel`
MOB-05	Fecha y horario (paso 2/3)	`SlotsPage`	`BookingViewModel`
MOB-06	Confirmar cita (paso 3/3)	`ConfirmPage`	`BookingViewModel`
MOB-07	Detalle de cita (cancelar)	`AppointmentDetailPage`	`AppointmentDetailViewModel`
MOB-08 / MOB-09	Mis resultados / detalle	`LabResultsPage`, `LabResultDetailPage`	`LabResultsViewModel`, `LabResultDetailViewModel`
MOB-10 / MOB-11	Mis recetas / detalle	`PrescriptionsPage`, `PrescriptionDetailPage`	`PrescriptionsViewModel`, `PrescriptionDetailViewModel`
MOB-12	Notificaciones (abrir lleva al resultado)	`NotificationsPage`	`NotificationsViewModel`

La barra inferior (Inicio, Citas, Resultados, Recetas) y la campana de notificaciones siguen el diseño.

Seguridad

- **Sesión en almacenamiento seguro:** `SecureSessionStore` guarda solo los tokens y el nombre con `SecureStorage` (Keystore de Android). **Nunca** la contraseña (la prueba `Login_valido_guarda_los_tokens_pero_nunca_la_contrasena` lo comprueba); el campo se vacía tras cada intento. `android:allowBackup="false"` evita copiar la sesión a copias de seguridad.
- **Autorización en la API:** la app no decide permisos. Un paciente solo recibe lo suyo (resultados, recetas, notificaciones y citas ajenas → 404); el personal que intenta entrar a la app es rechazado (la API devuelve el rol y la app descarta y revoca ese token). Lo comprueban `MobileApiIntegrationTests` contra la API real.
- **Renovación de sesión:** el acceso dura 15 minutos y el refresco es de un solo uso; el cliente renueva antes de que venza o ante un 401, con un candado para que solo una llamada renueve. Si falla, se borra la sesión local y se vuelve a MOB-01 con "Tu sesión venció. Inicia sesión de nuevo para continuar.".
- **Red:** solo HTTPS, salvo `network_security_config.xml`, que permite HTTP únicamente hacia `10.0.2.2`, `localhost` y `127.0.0.1` (desarrollo). En QA y producción la API debe estar en HTTPS.
- **Sin secretos en el repositorio:** la dirección de la API se fija al compilar (`-p:ApiBaseUrl=...`); las contraseñas de pruebas se pasan por variable de entorno.

Estados de interfaz

Cada pantalla que carga datos muestra **cargando**, **error de conexión con "Reintentar"**, **lista vacía** (textos del diseño: "Aún no tienes citas…", etc.) y, si la sesión venció, vuelve al inicio de sesión. Formularios: el inicio de sesión valida correo y contraseña antes de llamar a la API (mensajes bajo cada campo); la reserva no deja continuar sin horario ("Selecciona un horario para continuar.") y limita la fecha entre hoy y el máximo reservable.

Reserva sin duplicados ni pérdida de datos

- El botón **Confirmar** ignora pulsaciones mientras hay una solicitud en curso (`AsyncCommand`).
- Cada solicitud lleva un `Idempotency-Key` que se **reutiliza en los reintentos** (por ejemplo, tras perder la conexión): si la primera llegó y la respuesta no, la API devuelve la cita ya creada. Cambiar de horario genera otra clave.
- Si otra persona ocupa el horario antes de confirmar (409), la app muestra "Este horario ya está ocupado. Selecciona otro para continuar.", **conserva médico y fecha**, vuelve a consultar los horarios y permite elegir otro.

Compilar, instalar y ejecutar

Herramientas necesarias (estado de este equipo al 2026-10-27, tras instalarlas)

Herramienta	Estado en este equipo	Para qué
.NET SDK 10.0.401	instalado	todo
JDK 17 (Microsoft OpenJDK 17.0.21)	instalado	compilar Android y Maestro

Herramienta	Estado en este equipo	Para qué
Workload `maui-android`	**no instalado en Windows a propósito** (la app se compila en el contenedor Linux; no hace falta para probar)	compilar `src/MediCore.Mobile` de forma nativa
SDK de Android en `%LOCALAPPDATA%\Android\Sdk` (command-line tools, platform-tools y emulator, instalados de dl.google.com)	**instalado** (no había un SDK reutilizable: Visual Studio 18 Insiders no trae uno)	instalar y ejecutar
Emulador `medicore_pixel` (Pixel 6, `system-images;android-35;google_apis;x86_64`)	**instalado y en uso**	probar la app
Maestro CLI 2.12.1 en `%LOCALAPPDATA%\Maestro\maestro\bin` (release oficial de GitHub)	**instalado**	ejecutar los flujos
Aceleración del emulador	**WHPX disponible** (`emulator -accel-check`): no hizo falta habilitar nada ni reiniciar	rendimiento

Instalación autorizada por la persona responsable; se aceptaron las licencias del SDK de Android. Smart App Control sigue activo.

Pasos (cuando estén las herramientas)

```
dotnet workload install maui-android                          # como administrador
dotnet build src/MediCore.Mobile -f net10.0-android -t:InstallAndroidDependencies -
p:AndroidSdkDirectory="$env:LOCALAPPDATA\Android\Sdk" -p:AcceptAndroidSDKLicenses=True
# Emulador (una vez): crear un AVD con la imagen instalada, o conectar un teléfono con depuración USB.
dotnet build src/MediCore.Mobile -f net10.0-android -c Debug -p:ApiBaseUrl=http://10.0.2.2:5011/
adb install -r src/MediCore.Mobile/bin/Debug/net10.0-android/com.medicore.paciente-Signed.apk
```

Compilar sin instalar el SDK en el equipo (contenedor Linux, no sustituye la prueba en Android): `scripts/android-build-linux.sh` (ver más abajo).

> **Smart App Control:** sigue activado. Los ensamblados de MAUI de Microsoft están firmados; los de MediCore no, y Windows puede bloquear su carga si alguna herramienta de compilación los ejecuta (no solo los lee). Si ocurre, compila con `scripts/android-build-linux.sh`.

Conexión con los servicios en Docker

La API y las dependencias corren en Docker en tu equipo; la app corre en Android y debe llegar a la API:

1. `docker compose up -d` (PostgreSQL, Redis, RabbitMQ) y la API en Development con `scripts/dev-stack-linux.sh up` (o `dotnet run --project src/MediCore.Api --launch-profile http` donde Smart App Control lo permita). La API queda en `127.0.0.1:5011`.
2. **Emulador de Android Studio:** el equipo anfitrión se ve como `10.0.2.2`. Compila con `-p:ApiBaseUrl=http://10.0.2.2:5011/` (valor por defecto).
3. **Teléfono por USB:** `adb reverse tcp:5011 tcp:5011` y compila con `-p:ApiBaseUrl=http://localhost:5011/`.
4. **Teléfono por Wi-Fi:** publica la API en tu red con `DEV_STACK_BIND=0.0.0.0 scripts/dev-stack-linux.sh up` (solo en una red de confianza), compila con `-p:ApiBaseUrl=http://<IP-del-equipo>:5011/` y agrega esa IP a `Platforms/Android/Resources/xml/network_security_config.xml`.

5. Para las notificaciones, RabbitMQ debe estar en marcha: el publicador y el consumidor corren dentro de la API.
6. Usuarios de prueba: `ana.garcia@example.com` (paciente de desarrollo) y `e2e.paciente@example.com` (paciente aislado de las pruebas), con la contraseña de `Seed:DevPassword`.

Compilación verificada (contenedor Linux, no Android)

`scripts/android-build-linux.sh` (imagen `medicore-android-build:local`: SDK de .NET 10.0.403, JDK 17, workload `maui-android`, Microsoft.Maui.Sdk 10.0.110, Microsoft.Android.Sdk 36.1.118; SDK de Android con la plataforma 36 instalado en el volumen nuevo `medicore-android-sdk`) **compiló la app sin errores** y generó el APK de desarrollo:

Archivo	Contenido
`artifacts/com.medicore.paciente-Signed.apk` (≈15 MB, firmado con la clave de depuración)	paquete `com.medicore.paciente`, ABIs `arm64-v8a` (teléfonos) y `x86_64` (emulador), `network_security_config.xml` incluido, API en `http://10.0.2.2:5011/`
`artifacts/com.medicore.paciente.apk`	la misma compilación sin firmar

`artifacts/` no se versiona (ignorado por Git): se regenera con el script. Qué prueba esto: que el XAML (incluidos los enlaces compilados) y el código C# **compilan** contra MAUI 10 y Android 36. Qué **no** prueba: que las pantallas se vean y funcionen en Android; eso exige un emulador o dispositivo (ver "Herramientas necesarias").

El compilador encontró y se corrigieron: `DatePicker.NewDate` es `DateTime?` en MAUI 10 (CS1503), `DisplayAlert` está obsoleto (`DisplayAlertAsync`), el restore debe incluir `android-arm64;android-x64` (NETSDK1047) y el instalador del SDK de Android exige las variables `USER`/`USERNAME` en el contenedor. Queda la advertencia `MA002` del SDK de MAUI (informativa).

Compilar en un contenedor Linux (APK de desarrollo)

`scripts/android-build-linux.sh` usa la imagen `medicore-android-build:local` (`scripts/android/Dockerfile`: SDK de .NET 10, JDK 17 y el workload `maui-android`) y un volumen nuevo `medicore-android-sdk` para el SDK de Android. Deja el APK en `artifacts/` (ignorado por Git). **Compilar en Linux produce el APK, pero no prueba la app**: eso exige un emulador o dispositivo Android.

Maestro

```
export MEDICORE_PASSWORD=<Seed:DevPassword>
python scripts/maestro_run.py            # todos los flujos; --only 03 para uno
```

`maestro_run.py` verifica antes que existan Maestro, `adb`, un dispositivo y la app instalada; si falta algo, lo lista y **no escribe ninguna evidencia**. Con todo disponible escribe `documentacion/evidencias/entrega-7-maestro.md` con el resultado real de cada flujo. Detalle de los flujos en `tests/maestro/README.md`.

Adaptaciones al diseño (explicadas)

- **Tipografías:** el diseño usa Manrope y DM Sans (Google Fonts); la app usa la tipografía del sistema para no descargar ni empaquetar fuentes. Colores, formas y textos son los del diseño.
- **Íconos de la barra inferior:** solo texto ("Inicio", "Citas", "Resultados", "Recetas").
- **Fecha de la reserva:** el diseño usa `<input type="date">`; en Android es el selector de fecha nativo.

- **Sin simulaciones:** se omiten los datos y textos de demostración del prototipo ("Acceso simulado…", "Datos ficticios de demostración", credenciales precargadas).
- **Notificaciones:** hoy el sistema genera solo `LabResultReady`; otros tipos (por ejemplo "Cita confirmada" o "Nueva receta" del diseño) llegarán con los eventos pendientes y la app los muestra sin enlace hasta entonces.
- **Cerrar sesión** está en el Inicio (como el diseño).
- **Tarjetas completas tocables** (además del enlace "Ver … →") para facilitar el uso táctil y la automatización.

Probar en el emulador (pasos reproducibles)

```
$sdk = "$env:LOCALAPPDATA\Android\Sdk"
& "$sdk\cmdline-tools\latest\bin\avdmanager.bat" create avd -n medicore_pixel -k "system-images;android-35;google_apis;x86_64" -d pixel_6
Start-Process "$sdk\emulator\emulator.exe" -ArgumentList "-avd","medicore_pixel","-no-audio","-gpu","swiftshader_indirect"
scripts/dev-stack-linux.sh up                                          # API real en Docker (127.0.0.1:5011 = 10.0.2.2 desde el emulador)
scripts/android-build-linux.sh                                         # APK en artifacts/
& "$sdk\platform-tools\adb.exe" uninstall com.medicore.paciente     # ¡siempre desinstalar antes!
& "$sdk\platform-tools\adb.exe" install artifacts\com.medicore.paciente-Signed.apk
$env:MEDICORE_PASSWORD = "<Seed:DevPassword>"; $env:MAESTRO_CLI_NO_ANALYTICS = "1"
python scripts/maestro_run.py
```

> **Desinstala antes de instalar:** cada compilación en contenedor genera una clave de depuración nueva; `adb install -r` falla con `INSTALL_FAILED_UPDATE_INCOMPATIBLE` y se seguiría probando la versión anterior. Al desinstalar se pierden los datos locales de la app (la sesión). **Mejora pendiente:** conservar una clave de depuración estable fuera del repositorio (ver H7-13 en `04-registro-de-hallazgos.md`).

Repetición final (2026-11-06): 8/8 con el código final, en emulador Android 35 contra QA (`http://10.0.2.2:8080/`). Evidencia: `documentacion/evidencias/maestro-codigo-final.md`. Antes de ejecutar hay que cerrar la cortina de notificaciones del emulador: `adb shell cmd statusbar collapse`.

Defectos que solo aparecieron al ejecutar en Android (corregidos)

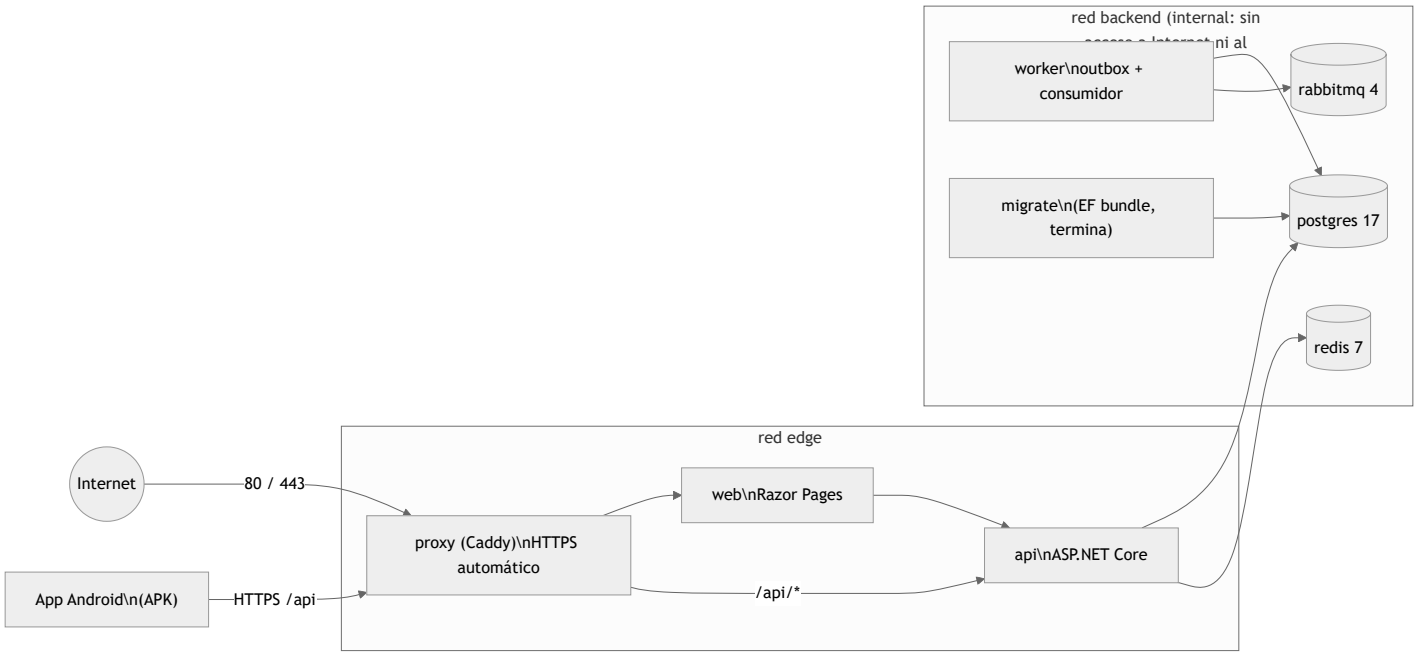
1. **La app se cerraba al abrir** (`No assemblies found ... Fast Deployment`): el APK Debug no incluía los ensamblados. Ahora `EmbedAssembliesIntoApk=true` y `AndroidUseSharedRuntime=false` (el APK pesa ≈84 MB).
2. **Bibliotecas nativas comprimidas** (`lib/x86_64 MUST be STORED`): se agregó `android:extractNativeLibs="true"` al manifiesto.
3. `App` recibía `AppShell` por inyección antes de cargar los recursos; ahora lo crea `CreateWindow` y el Shell usa colores literales en su raíz.

MediCore · Contenedores e infraestructura de QA (entrega 8)

Estado: **ejecutado sin costo; NO desplegado en Azure.** El equipo decidió no usar recursos de pago. Terraform corrió con un proveedor simulado y Ansible sobre un Ubuntu desechable (`documentacion/evidencias/iac-sin-costo.md`). No se creó ningún recurso real. Si se decide pagar, hay que aprobar antes el plan y el costo de la sección 3.

1. Arquitectura

Un monolito modular en contenedores (sin microservicios): la API, la web, un proceso de mensajería (worker) y tres dependencias, detrás de un proxy inverso. La app Android es un APK: **no** es un servicio del Compose.



Servicio	Imagen	Puertos publicados	Salud	Persistencia
`postgres`	`postgres:17-alpine`	ninguno	`pg_isready`	volumen `postgres-data`
`redis`	`redis:7-alpine` (con contraseña)	ninguno	`redis-cli ping`	volumen `redis-data`
`rabbitmq`	`rabbitmq:4-management-alpine` (nodo fijo `medicore-rabbitmq`)	ninguno	`rabbitmq-diagnostics ping`	volumen `rabbitmq-data`
`migrate`	`docker/migrator.Dockerfile` (paquete `efbundle`)	—	termina con código 0	—
`api`	`docker/api.Dockerfile`	ninguno	`GET /api/health/ready` (consulta PostgreSQL)	—

Servicio	Imagen	Puertos publicados	Salud	Persistencia
`worker`	`docker/worker.Dockerfile`	ninguno	latido en `/tmp/medicore-worker.alive`	—
`web`	`docker/web.Dockerfile`	ninguno	`GET /Login`	volumen `web-keys` (claves de la cookie de sesión)
`proxy`	`caddy:2-alpine`	80 y 443 (local: 127.0.0.1:8080 y 8443)	`GET /api/health`	volúmenes `caddy-data`, `caddy-config` (certificados)

Decisiones: - **Se reutilizó el `docker-compose.yml` existente** (dependencias con su salud y volúmenes). `docker-compose.qa.yml` se superpone: quita los puertos publicados (`ports: !reset []`), agrega las redes y los servicios de la aplicación. El nombre de proyecto `medicore-qa` crea volúmenes propios (`medicore-qa_*`): el desarrollo (`medicore_*`) no se toca. - **Proceso de mensajería = `MediCore.Worker`** (proyecto nuevo, sin HTTP): es el mismo código de `MediCore.Infrastructure` (publicador del outbox y consumidor de `LabResultReady`). En QA la API corre con `Messaging__Enabled=false` y solo guarda el evento en el outbox; el worker lo publica. En desarrollo (`dotnet run`) todo sigue dentro de la API. - **Configuración solo por variables de entorno** (`.env.qa`, ignorado por Git; `.env.qa.example` documenta cada variable). Nada de secretos en imágenes ni en el repositorio. Las imágenes corren con usuario sin privilegios. - **Detrás del proxy:** la API y la web leen `X-Forwarded-For/Proto` (`ForwardedHeaders__Enabled`), de modo que el límite de intentos de inicio de sesión y la auditoría ven la IP real del cliente. Es seguro porque solo el proxy llega a ellas. - **Claves de la cookie de sesión de la web** en un volumen: sobreviven a los reinicios (sin esto, cada reinicio cerraba todas las sesiones).

Orden de arranque y migraciones

`postgres`, `redis`, `rabbitmq` (saludables) → `migrate` (aplica las migraciones pendientes y termina; es idempotente y toma un bloqueo exclusivo, así que reintentar o correr dos veces es seguro) → `api` y `worker` (saludables) → `web` → `proxy`. Las migraciones **no** se aplican al arrancar la API fuera de Development: las aplica solo el contenedor `migrate`. Los datos ficticios de QA se siembran aparte y bajo demanda (`scripts/qa.sh seed`, solo en los entornos Development y QA).

Operación

```
scripts/qa.sh init        # crea .env.qa con contraseñas aleatorias (una vez)
scripts/qa.sh up          # construye, migra y levanta
scripts/qa.sh seed        # usuarios y datos FICTICIOS de QA
scripts/qa.sh smoke       # pruebas de humo de web y API a través del proxy
scripts/qa.sh restart     # reinicia sin borrar volúmenes
scripts/qa.sh down        # elimina contenedores y red; CONSERVA los volúmenes
scripts/qa.sh destroy     # además BORRA los volúmenes medicore-qa_* (pide escribir el nombre del
proyecto)
scripts/qa.sh backup      # respaldo inmediato; los automáticos corren cada 6 horas
scripts/qa.sh backups     # lista los respaldos del volumen medicore-qa-respaldos (sobrevive a destroy)
scripts/qa.sh export <carpeta>   # copia los respaldos fuera de Docker
scripts/qa.sh import <archivo>   # trae un respaldo exportado
scripts/qa.sh restore <archivo>  # REEMPLAZA la base (pide confirmación)
```

2. Infraestructura en Azure (propuesta sencilla, uso académico)

Una sola máquina virtual Linux con Docker Compose. Es la opción más barata y simple que cumple el documento (contenedores, ambiente QA reproducible, Terraform + Ansible); se descartaron Container Apps/AKS y PostgreSQL/Redis administrados por costo y complejidad (se anotan como evolución).

```
Grupo de recursos medicore-qa-rg
 ├─ Red virtual 10.20.0.0/16 · subred app 10.20.1.0/24 · NSG (22 solo desde tu IP, 80, 443; todo lo
demás negado)
 ├─ IP pública Standard estática + nombre DNS gratuito  <etiqueta>.<región>.cloudapp.azure.com
 ├─ Máquina virtual Ubuntu 24.04 · Standard_B2s (2 vCPU, 4 GiB) · acceso solo por clave SSH
 │    ├─ Disco del sistema 64 GiB (StandardSSD)
 │    └─ Disco de datos 32 GiB (StandardSSD) montado para /var/lib/docker (los volúmenes sobreviven si
se recrea la VM)
 └─ Apagado automático diario (opcional)
Grupo medicore-tfstate-rg (se crea una vez con infrastructure/terraform/bootstrap)
 └─ Cuenta de almacenamiento (LRS) con el estado de Terraform
```

Reparto de responsabilidades (sin duplicar):

Terraform (`infrastructure/terraform`)	Ansible (`infrastructure/ansible`)
Recursos de Azure: grupo, red, NSG, IP, VM, discos, apagado automático, estado remoto	Dentro de la VM: paquetes, SSH endurecido, UFW, fail2ban, actualizaciones automáticas, disco de datos, Docker, despliegue del Compose (el mismo de local), `.env.qa` con los secretos del Vault, siembra opcional
No guarda secretos de la aplicación ni usa cloud-init	No crea nada en Azure

Seguridad de red: ni PostgreSQL, Redis ni RabbitMQ se exponen: no hay regla de entrada en el NSG para 5432, 6379, 5672 ni 15672, UFW los niega y, además, el Compose de QA no publica sus puertos (viven en la red `internal`). SSH solo desde `admin_cidrs` (la validación de Terraform rechaza `0.0.0.0/0`). El panel de RabbitMQ no se publica.

Estado de Terraform: en Azure Blob Storage (`infrastructure/terraform/bootstrap` lo crea una sola vez): contenedor privado, solo HTTPS/TLS 1.2, **sin claves de acceso** (solo Microsoft Entra ID con el rol *Storage Blob Data Contributor*), versionado y retención de 30 días, bloqueo del blob mientras se ejecuta Terraform (evita dos `apply` simultáneos) y candado `CanNotDelete`. El estado del propio bootstrap es local (pequeño y sin secretos) y está ignorado por Git, igual que `*.tfvars`, `backend.hcl`, el inventario real y el Vault de Ansible. `.terraform.lock.hcl` **sí** se versiona (fija las versiones de los proveedores).

3. Recursos y costo estimado (REQUIERE TU APROBACIÓN antes de crear nada)

Precios aproximados de pago por uso en `eastus` (USD, 730 h/mes), a **verificar en la calculadora de precios de Azure** antes de aprobar; cambian con la región y el tiempo. Cada recurso va por separado:

#	Recurso	Detalle	USD/mes (24/7)	USD/mes (~8 h/día, 22 días)
1	Máquina virtual `Standard_B2s`	2 vCPU, 4 GiB, Linux; se factura por hora encendida	≈ 30	≈ 7
2	Disco del sistema	StandardSSD 64 GiB	≈ 5	≈ 5
3	Disco de datos	StandardSSD 32 GiB	≈ 2,4	≈ 2,4
4	IP pública Standard estática	se cobra aunque la VM esté apagada	≈ 3,7	≈ 3,7
5	Cuenta de almacenamiento del estado	LRS, unos pocos KB	< 0,1	< 0,1
6	Red virtual, NSG, interfaz, nombre DNS, diagnóstico de arranque	sin costo	0	0
7	Tráfico de salida	primeros 100 GiB/mes gratis	0	0
8	HTTPS (Let's Encrypt), Caddy, Docker, Ansible, Terraform	software libre	0	0
	Total aproximado		**≈ 41**	**≈ 18**

- Apagar la VM (el apagado automático diario ya está configurado) detiene el costo del cómputo, **no** el de discos ni IP.
- `terraform destroy` elimina todo (excepto el estado y su cuenta, protegidos por el candado) y deja el costo en ≈ 0 (el estado cuesta centavos).
- Alternativas: `Standard_B2ms` (8 GiB) ≈ 60 USD/mes si hace falta más memoria; `Standard_B1ms` (2 GiB) ≈ 15 USD/mes probablemente insuficiente para 7 contenedores. Con *Azure for Students* o créditos de prueba el costo puede quedar cubierto: confirma tu saldo.
- Antes de aplicar hay que comprobar la cuota de vCPU de la serie B en la región.

4. Crear, modificar y destruir (procedimiento controlado)

Requisitos en la máquina de administración: `az` (Azure CLI), Terraform ≥ 1.9, Ansible-core con las colecciones de `requirements.yml` (o contenedores equivalentes), una clave SSH y una suscripción de Azure aprobada.

Crear 1. `az login` y `az account set --subscription <id>`. 2. Estado remoto, una sola vez: `cd infrastructure/terraform/bootstrap && terraform init && terraform plan && terraform apply`. Copia la salida `backend_config` a `infrastructure/terraform/azure/backend.hcl`. 3. `cd ../azure`, copia `terraform.tfvars.example` a `terraform.tfvars` (tu clave **pública**, tu IP con `/32`), `terraform init -backend-config=backend.hcl`, `terraform plan -out qa.plan`, **revisa el plan**, `terraform apply qa.plan`. 4. `terraform output ansible_inventory` → pégalo en `infrastructure/ansible/inventory/qa.ini`. Crea y cifra `group_vars/qa/vault.yml` (ver `vault.yml.example`). 5. `cd ../../ansible && ansible-galaxy collection install -r requirements.yml && ansible-playbook site.yml --ask-vault-pass` (con `-e medicore_seed=true` la primera vez para sembrar datos ficticios). 6. Verifica: `QA_URL=https://<fqdn> bash scripts/qa-smoke.sh`.

Modificar: cambiar el tamaño, las reglas del NSG o el apagado se hace en `variables.tf` / `terraform.tfvars` y `terraform plan` → `apply` (el plan muestra qué se reemplaza). Cambios de configuración o de versión de la aplicación: `medicore_version` en `group_vars/qa/main.yml` y `ansible-playbook site.yml` (idempotente).

Destruir: `docker compose ... down` no borra datos; para eliminar la infraestructura `cd infrastructure/terraform/azure && terraform plan -destroy` y luego `terraform destroy`. El estado y su cuenta (con candado) se eliminan aparte, a propósito, quitando el candado.

5. Verificado localmente (qué se ejecutó y qué no)

Detalle y resultados en `documentacion/evidencias/entrega-8-verificacion.md`. Resumen: - Se construyeron las 4 imágenes y se levantó el ambiente QA con `scripts/qa.sh up` (todo saludable), se sembró y pasaron las pruebas de humo de web y API a través del proxy. - Persistencia: datos, notificación asíncrona (worker + RabbitMQ) y volúmenes sobreviven a `restart` y a `down` + `up` sin borrar volúmenes. - Los puertos de PostgreSQL, Redis y RabbitMQ del ambiente QA no se publican. - La app Android (APK compilado contra `http://10.0.2.2:8080/`) se conectó al ambiente local en contenedores y los flujos Maestro pasaron (ver la evidencia). - Terraform (`fmt`, `init -backend=false`, `validate`), Ansible (sintaxis y `ansible-lint` perfil *production*), `hadolint` y `docker compose config` pasan. - `scripts/iac-simulacion.sh` ejecuta Terraform con proveedor simulado y aplica Ansible en un Ubuntu desechable: **prueba el código, no un despliegue en Azure.** - **No se ejecutó:** `terraform plan/apply` real (requiere suscripción de Azure), `ansible-playbook` por SSH contra una VM, HTTPS con certificado real, ni una prueba de la app contra un servidor en Internet.

6. Qué falta para probar el despliegue real en Azure

1. Tu **aprobación** del plan y costo (sección 3) y una suscripción con crédito y cuota de la serie B.
2. Instalar en la máquina de administración Azure CLI, Terraform y Ansible (o usar los contenedores de `scripts/validate-iac.sh`) y hacer `az login`.
3. Ejecutar el bootstrap, `plan` y `apply` (sección 4) y revisar el plan antes de aplicar.
4. Una clave SSH y tu IP de administración; el Vault de Ansible con contraseñas nuevas.
5. Acceso al repositorio desde la VM (hoy `medicore_repo_url` es el repositorio de GitHub: si es privado hace falta una llave de despliegue de solo lectura).
6. Compilar el APK con `-p:ApiBaseUrl=https://<fqdn>/` (la app solo permite HTTP hacia el emulador y localhost) y repetir los flujos Maestro contra Azure.

7. Mejoras anotadas

- Registro de imágenes (GHCR o ACR) para no construir en la VM: lo traerá el pipeline de Jenkins.
- Llevar los respaldos a un destino externo (hoy `qa.sh export` es manual, ver `evidencias/respaldos.md`) y agregar monitoreo con alertas.
- Fijar las imágenes base por resumen (*digest*) y escanearlas (Trivy) en el pipeline.

MediCore · Pruebas de rendimiento (k6)

Escenario: mitad de los usuarios virtuales reserva citas y la otra mitad consulta agenda, pacientes, catálogo y expediente (`tests/k6/medicore.js`). Se ejecuta con `scripts/perf-tests.sh <perfil>` contra el ambiente QA.

Umbrales declarados antes de ejecutar

Perfil	Carga	p95 lectura	p95 reserva	Error inesperado
Load	30 usuarios durante 3 min	< 800 ms	< 1000 ms	< 1 %
Stress	sube hasta 150 usuarios	informativo	informativo	< 5 %
Spike	salto de 10 a 200 usuarios	< 2000 ms	< 3000 ms	< 2 %

Una reserva rechazada con 409 es el comportamiento correcto (horario ocupado) y no cuenta como error. Además, al terminar debe cumplirse: ninguna cita activa solapada por médico o paciente, y ningún mensaje en la cola de mensajes fallidos.

Resultados (ambiente QA local en Docker, equipo de 12 núcleos y 16 GB)

Perfil	Peticiones	req/s	p95 lectura	p95 reserva	Error inesperado	Reservas	409	Umbrales
load	20399	75.2	5.3 ms	8.9 ms	0 %	3260	41	cumplidos
stress	41620	196.4	6.2 ms	8.6 ms	0 %	6672	186	cumplidos
spike	43478	328.0	7.4 ms	11.5 ms	0 %	6954	188	cumplidos

Después de cada perfil: 0 solapes por médico, 0 solapes por paciente, 0 eventos pendientes y 0 mensajes en la DLQ (`reports/k6/integridad.txt`, `reports/k6/corrida.txt`).

Reservas simultáneas

`scripts/perf-citas.sh [cantidad] [distintas|mismo]` lanza todas las reservas a la vez, cada una con un paciente distinto, y luego comprueba la integridad.

Prueba	Creadas	409	Errores	p95
100 en horarios distintos	100	0	0	194,6 ms

Prueba	Creadas	409	Errores	p95
100 al mismo horario	1	99	0	382,8 ms
200 en horarios distintos	200	0	0	120,6 ms
200 al mismo horario	1	199	0	634,2 ms

En las cuatro hubo 0 solapes por médico y por paciente, y 0 errores 500 en la API. Antes de corregir HP-03, 1 de cada 100 reservas fallaba. Los JSON de `reports/k6/citas-*.json` son de las pruebas de 200.

Defectos que encontró esta prueba

- **HP-01, escenario de carga:** las primeras corridas de Stress y Spike aparecieron aprobadas casi sin reservas, porque Load había llenado las agendas. Se descartaron. Ahora cada corrida crea sus médicos y falla si no alcanza un mínimo de reservas.
- **HP-02, defecto de producto:** el publicador del outbox enviaba como máximo 20 eventos cada 2 s. La API seguía rápida, pero al terminar quedaron 14 405 notificaciones pendientes. Ahora el lote es de 200 y el publicador continúa sin esperar mientras haya lotes completos. El rezago se vació en menos de un minuto y las corridas finales terminan con 0 pendientes.
- **HP-03, defecto de configuración:** 100 reservas sin pausa agotaron las conexiones de PostgreSQL. Load, Stress y Spike no lo vieron porque cada usuario espera entre peticiones. Se limitó cada proceso a 40 conexiones.

Límites de esta medición

- No se alcanzó el punto de quiebre: 200 usuarios virtuales no degradaron el sistema. Cada usuario espera entre 0,5 y 1,5 s entre iteraciones.
- k6, la API y la base de datos comparten el mismo equipo, sin latencia de red real. Los números no representan una VM de Azure `Standard_B2s`.
- El p95 de lectura se beneficia de la cache de Redis y de un expediente con pocos datos.

MediCore · Prueba de usabilidad

Estado: pendiente de ejecutar. El documento del proyecto exige al menos 5 participantes reales. Este archivo define la prueba; los resultados se llenan al aplicarla y no deben inventarse.

Preparación

- Ambiente: QA local (`scripts/qa.sh up`, `scripts/qa.sh seed`), web en `http://127.0.0.1:8080` y app móvil en emulador o teléfono.
- Participantes: mínimo 5 personas que no hayan desarrollado el sistema. Al menos 1 por cada perfil: recepción, médico, laboratorio y paciente.
- Moderación: una persona guía y otra anota. No se ayuda salvo que el participante quede bloqueado; esa tarea cuenta como no completada.
- Datos: solo cuentas ficticias. No se registra información personal de los participantes.

Tareas

ID	Perfil	Tarea	Éxito	Tiempo objetivo
U1	Recepción	Registrar un paciente nuevo	Aparece en la lista	2 min
U2	Recepción	Reservar una cita para ese paciente	Aparece en la agenda	2 min
U3	Recepción	Cancelar esa cita	Queda cancelada y el horario se libera	1 min
U4	Médico	Abrir la agenda y el expediente de un paciente	Ve consultas y recetas	1 min
U5	Médico	Registrar una consulta con una receta de dos medicamentos	La receta aparece en el expediente	4 min
U6	Laboratorio	Registrar un resultado con dos parámetros	Aparece con su estado de notificación	3 min
U7	Paciente	Reservar una cita en la app	Aparece en "Mis citas"	2 min
U8	Paciente	Consultar un resultado y una notificación	Abre el detalle	2 min

Registro por participante

Participante (código)	Perfil	Tarea	Completó (sí/no)	Tiempo	Errores	Ayuda pedida	Comentario
P1							
P2							
P3							

Participante (código)	Perfil	Tarea	Completó (sí/no)	Tiempo	Errores	Ayuda pedida	Comentario
P4							
P5							

Cuestionario SUS

Al terminar, cada participante responde de 1 (muy en desacuerdo) a 5 (muy de acuerdo):

1. Creo que me gustaría usar este sistema con frecuencia.
2. Encontré el sistema innecesariamente complejo.
3. Pensé que el sistema era fácil de usar.
4. Creo que necesitaría ayuda técnica para poder usarlo.
5. Las funciones del sistema estaban bien integradas.
6. Pensé que había demasiada inconsistencia en el sistema.
7. Imagino que la mayoría aprendería a usarlo rápidamente.
8. Encontré el sistema muy incómodo de usar.
9. Me sentí seguro usando el sistema.
10. Necesité aprender muchas cosas antes de poder usarlo.

Cálculo: a las preguntas impares se les resta 1; las pares se restan de 5. La suma se multiplica por 2,5.

Criterios de aceptación

- Al menos 80 % de las tareas completadas sin ayuda.
- SUS promedio de 68 o más.
- Ningún problema que impida completar U2, U5 o U6.

Resultados

Por llenar después de la sesión: tasa de finalización, tiempo promedio, SUS promedio, problemas encontrados y su severidad. Cada problema se registra como Issue.

MediCore · Matriz de riesgos (confidencialidad, integridad y disponibilidad)

Escala: probabilidad e impacto de 1 (bajo) a 3 (alto). Nivel = probabilidad × impacto: 1–2 bajo, 3–4 medio, 6–9 alto. "Residual" es el nivel después de los controles que ya existen.

ID	Dim.	Riesgo	Prob.	Imp.	Nivel	Control implementado	Evidencia	Residual
R-01	C	Un usuario lee datos clínicos de otro paciente (IDOR)	2	3	6	Autorización por recurso; acceso ajeno responde 404 y se audita	`ClinicalApiTests`, `LaboratoryApiTests`, Insomnia	Bajo
R-02	C	Un rol administrativo accede a información clínica	2	3	6	RBAC: Recepción y Administrador sin acceso clínico	`IdentityApiTests`, Insomnia, Cypress `acceso-permisos`	Bajo
R-03	C	Datos clínicos en bitácoras, auditoría o eventos	2	3	6	Auditoría solo con identificadores; eventos sin contenido	Pruebas de bitácoras, `PatientEventsTests`	Bajo
R-04	C	Sesión robada o token reutilizado	2	3	6	JWT de 15 min, refresco de un solo uso, sesión en Redis, cookie HttpOnly	`IdentityApiTests`	Bajo
R-05	C	Secretos en el repositorio	2	3	6	`.env` ignorados, plantillas de ejemplo, Ansible Vault	Revisión de archivos versionados	Bajo
R-06	C	Vulnerabilidad web explotable en QA	2	3	6	CSP, cabeceras, antiforgery, validación de entrada	OWASP ZAP web y API	Bajo
R-07	C/I	Dependencia con vulnerabilidad conocida	2	3	6	Análisis en el pipeline	Dependency-Check, npm audit, NuGet	Medio: revisar avisos de Redis
R-08	I	Doble reserva de un horario bajo alta demanda	3	3	9	Exclusión en PostgreSQL, candados de asesoría, reintento	`AppointmentsApiTests`, k6, consulta de solapes	Bajo
R-09	I	Reserva duplicada por reintento del cliente	3	2	6	`Idempotency-Key` y detección de la misma reserva	`AppointmentsApiTests`, Insomnia, Maestro	Bajo
R-10	I	Alteración de registros clínicos	1	3	3	Sin edición ni borrado: PUT, PATCH y DELETE responden 405	`ClinicalApiTests`, Insomnia	Bajo

ID	Dim.	Riesgo	Prob.	Imp.	Nivel	Control implementado	Evidencia	Residual
R-11	I	Datos vencidos en cache llevan a reservar lo ocupado	2	2	4	TTL e invalidación; la base de datos decide la reserva	`AvailabilityApiTests`	Bajo
R-12	I/D	RabbitMQ cae después de registrar un resultado	2	3	6	Outbox transaccional con reintentos	`MessagingTests`, `PatientEventsTests`, `evidencias/resiliencia-qa.md`	Bajo
R-13	I	Mensaje repetido o dañado en la cola	2	2	4	Consumidor idempotente, reintentos y DLQ	`MessagingTests`	Bajo
R-14	D	Redis no disponible	2	3	6	Cache falla abierto; sesiones fallan cerrado por seguridad	`AvailabilityApiTests`, `IdentityApiTests`	Medio: sin Redis no hay sesión
R-15	D	Saturación por picos de demanda	2	3	6	Pruebas de carga con umbrales y lote de outbox	Reportes de k6	Bajo en local; sin medir en Azure
R-16	D	Pérdida de datos por falta de respaldo	2	3	6	Copias cifradas cada 6 h, verificadas; restauración probada (`evidencias/respaldos.md`)	H8-08 cerrado	Medio: siguen en el mismo disco salvo exportación
R-17	D	Servidor único en QA	2	2	4	Reconstrucción por Terraform y Ansible	`evidencias/iac-sin-costo.md`, sin Azure	Medio: abierto
R-18	D	Ambiente de nube no aprovisionado	3	2	6	Código ejecutado sin costo; el equipo decidió no pagar Azure	`evidencias/iac-sin-costo.md`	Alto: abierto
R-19	C/I	App móvil probada solo en emulador	2	2	4	Maestro 8/8 con el código final, contra QA	`evidencias/maestro-codigo-final.md`	Bajo: falta dispositivo físico

MediCore · Plan de pruebas

Plan formal del Proyecto 2 (curso 048, UMG). Sigue las 20 secciones de la sección 7 del documento del proyecto. Los números de la sección 18 salen de reportes guardados en `reports/`; lo que no se ejecutó aparece como pendiente.

1. Introducción y objetivo

MediCore centraliza pacientes, agendas, consultas, recetas, resultados de laboratorio y notificaciones de una institución de salud. Incluye una web para el personal, una app Android para el paciente y una API.

El objetivo del plan es comprobar, con evidencia reproducible, cuatro cosas:

1. Las reglas clínicas y de agenda funcionan por cada rol.
2. Ningún usuario ve o cambia información que no le corresponde.
3. Con alta demanda no hay doble reserva, y una falla del broker no pierde datos clínicos.
4. La solución se puede reconstruir desde el repositorio y el pipeline detiene una entrega con fallas.

2. Alcance y fuera de alcance

Dentro del alcance

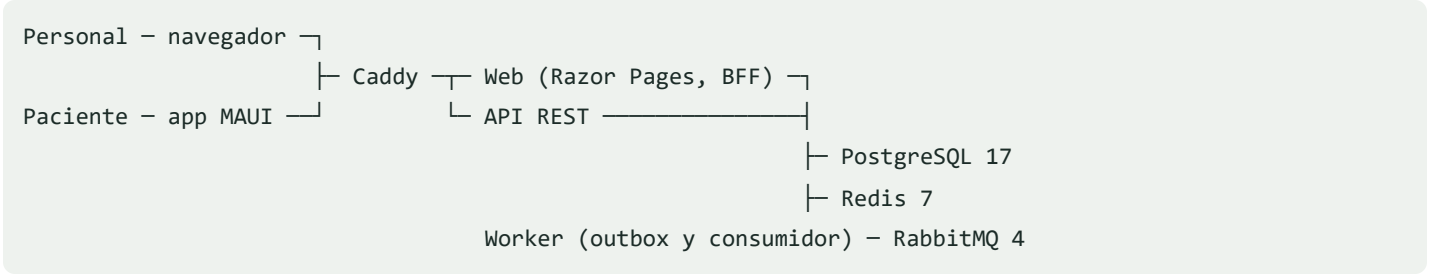
- Módulos: Identidad, Pacientes, Médicos, Agenda, Consultas, Recetas, Laboratorio, Notificaciones y Auditoría.
- Web del personal (WEB-01 a WEB-18) y app del paciente (MOB-01 a MOB-12).
- API de referencia y sus rutas complementarias, con 38 rutas en `documentacion/openapi-v1.json`.
- Redis: disponibilidad, catálogos, sesiones, TTL, invalidación y caída del servicio.
- RabbitMQ: outbox, consumidor, reintentos, DLQ e idempotencia.
- Pipeline de Jenkins, contenedores, Terraform y Ansible.

Fuera del alcance

- Portal web del paciente y módulos no listados en el documento.
- Microservicios.
- Integración con sistemas hospitalarios externos.
- Pagos y facturación.
- Aprovisionar Azure de pago: la decisión de pago sigue abierta. El código se valida, pero no se aplica.

3. Arquitectura de la solución

Monolito modular en .NET 10 con un solo modelo de datos.



- **Datos:** PostgreSQL garantiza las reglas críticas. Dos restricciones `EXCLUDE` impiden solapar citas por médico y por paciente.
- **Cache y sesión:** el cache falla abierto y consulta PostgreSQL. La sesión falla cerrado: sin Redis nadie entra.
- **Eventos:** el dato y su evento se guardan en la misma transacción. El Worker publica después y el consumidor no duplica notificaciones.
- **Infraestructura:** Docker Compose para QA, Terraform para Azure y Ansible para configurar el servidor.

4. Riesgos y prioridades

La matriz completa está en 09-matriz-de-riesgos.md, con 19 riesgos clasificados por confidencialidad, integridad y disponibilidad.

Prioridad	Qué se prueba primero	Riesgos
1	Doble reserva bajo alta demanda	R-08, R-09
1	Acceso a datos clínicos ajenos	R-01, R-02, R-03
1	Pérdida de un resultado si cae RabbitMQ	R-12, R-13
2	Sesiones, secretos y vulnerabilidades	R-04, R-05, R-06, R-07
2	Caída de Redis y saturación	R-14, R-15
3	Respaldos y disponibilidad del servidor	R-16, R-17, R-18

5. Estrategia de pruebas

La mayor parte de las pruebas corre sin interfaz, porque es más rápida y estable. Las pruebas de navegador y móvil cubren los recorridos que usan las personas.

Nivel	Herramienta	Qué demuestra
Unitario	xUnit	Reglas de dominio y lógica móvil
Integración	xUnit y Testcontainers	API con PostgreSQL, Redis y RabbitMQ reales
API	Insomnia (`inso`)	Contrato, errores, autenticación y autorización
Web	Cypress	Recorridos por rol en un navegador real
Móvil	Maestro	Flujos del paciente en Android
SAST	SonarQube, Fortify	Defectos y vulnerabilidades en el código
SCA	Dependency-Check, npm audit	Dependencias vulnerables
DAST	OWASP ZAP	Ataques sobre el ambiente QA en ejecución
Rendimiento	k6	Umbrales de p95, errores y throughput
Accesibilidad	axe-core	Reglas WCAG en 14 pantallas
Usabilidad	Sesión con personas y SUS	Facilidad de uso real

Toda corrección se comprueba repitiendo la prueba que encontró la falla. Los defectos se registran en 12-registro-de-defectos.md.

6. Ambientes y datos de prueba

Ambiente	Uso	Cómo se levanta
Testcontainers	Integración, con contenedores desechables	`scripts/dotnet-linux.sh test MediCore.slnx`
QA local	API, Cypress, ZAP y k6	`scripts/qa.sh init`, `up`, `seed`, `smoke`
CI local	Jenkins (8090) y SonarQube (9000)	`ci/docker-compose.ci.yml`
Android	Maestro en emulador	`scripts/maestro_run.py`
QA en Azure	Último paso, sujeto a la decisión de pago	`documentacion/06-infraestructura.md`

Todos los datos son ficticios. `--seed-qa` crea usuarios y datos solo en Development o QA. Las contraseñas de QA viven en `.env.qa`, que no se versiona. Cada corrida de k6 crea 30 médicos, 300 pacientes y 55 fechas para que las agendas no se llenen.

7. Pruebas unitarias

Proyectos `tests/MediCore.UnitTests` (dominio) y `tests/MediCore.Mobile.Tests` (lógica de la app).

Cubren reglas de horarios, citas de 30 minutos, estados, validación de recetas y parámetros, renovación de sesión móvil y `Idempotency-Key` en reintentos.

8. Pruebas de integración

Proyecto `tests/MediCore.Api.Tests`, con la API real y dependencias reales en contenedores.

Integración	Pruebas	Qué se verifica
API con PostgreSQL	`AppointmentsApiTests`, `ClinicalApiTests`, `PatientsApiTests`	Restricciones, transacciones y concurrencia de agenda
API con Redis	`AvailabilityApiTests`, `IdentityApiTests`	TTL, datos vencidos, invalidación, sesiones y caída de Redis
API con RabbitMQ	`MessagingTests`, `PatientEventsTests`	Outbox y publicación con confirmación del broker
Consumidor con RabbitMQ	`MessagingTests`	Reintentos, DLQ, idempotencia y broker detenido

9. Pruebas de API

`tests/insomnia/medicore.insomnia.json` tiene 61 solicitudes y 65 comprobaciones. Se ejecuta con `scripts/api-tests.sh` contra QA.

Cubre casos positivos, negativos, autenticación, autorización por rol y por recurso, y errores. Incluye la API de referencia completa, reservas repetidas, IDOR y los 405 de edición clínica.

10. Pruebas funcionales y smoke

- **Smoke:** `scripts/qa.sh smoke` comprueba disponibilidad con PostgreSQL, cabeceras de seguridad, que el proxy no exponga OpenAPI e inicio de sesión de Recepción.

- **Funcionales:** Cypress, en `tests/cypress/e2e`, recorre Recepción, Médico, Laboratorio y Administrador.
- **Negativos:** credenciales inválidas, acceso sin permiso, horario ocupado y formularios incompletos.

Los casos de prueba y su resultado están en 11-casos-de-prueba.md.

11. Pruebas de regresión

El pipeline repite todas las suites en cada ejecución. Además:

- Cada defecto de producto corregido deja una prueba que lo reproduce, como el carácter nulo en `PatientsApiTests`.
- Cypress y xUnit se repiten después de cada cambio de código antes de declarar un resultado.

12. Pruebas móviles

`tests/maestro` tiene 8 flujos: login válido, login inválido, reserva, consulta y cancelación, resultados, recetas, notificaciones y cierre de sesión. Usa el paciente aislado `e2e.paciente`.

La app usa solo la API real. El APK se compila con `scripts/android-build-linux.sh`.

13. Pruebas de seguridad

Tema del documento	Cómo se prueba
RBAC por rol	`IdentityApiTests`, Insomnia y Cypress `acceso-permisos`
Autorización por recurso	`ClinicalApiTests`, `LaboratoryApiTests` e Insomnia
Datos sensibles	Pruebas de bitácoras, `PatientEventsTests` y revisión de auditoría
Sesiones y autenticación	JWT, refresco de un solo uso, bloqueo y límite por IP
API Security	Validación, 405, cabeceras, ZAP API con sesión real
Trazabilidad	`audit_entries`, `GET /api/audit` y sus pruebas
Código y dependencias	SonarQube, Dependency-Check, npm audit y Fortify si hay licencia

Las excepciones aceptadas viven en `ci/riesgos-aceptados.json`, con su motivo.

14. Pruebas de rendimiento

`tests/k6/medicore.js` simula pacientes que reservan mientras profesionales consultan agenda y expedientes. Los umbrales se declararon antes de ejecutar y están en 07-rendimiento.md.

Perfil	Carga	p95 lectura	p95 reserva	Error inesperado
Load	30 usuarios por 3 min	< 800 ms	< 1000 ms	< 1 %
Stress	Hasta 150 usuarios	Informativo	Informativo	< 5 %
Spike	De 10 a 200 usuarios	< 2000 ms	< 3000 ms	< 2 %

Al terminar cada perfil debe haber 0 citas solapadas, 0 eventos pendientes y 0 mensajes en la DLQ.

15. Pruebas de usabilidad y accesibilidad

- **Accesibilidad:** `accesibilidad-axe.cy.js` analiza 14 pantallas con axe-core 4.10.3 y reglas WCAG A y AA.
- **Usabilidad:** 08-usabilidad.md define 8 tareas, el cuestionario SUS y los criterios. Exige al menos 5 personas reales.

16. Criterios de entrada y salida

Entrada

- El código compila y `dotnet format` no reporta cambios.
- Las migraciones se aplican en un ambiente limpio.
- QA responde al smoke y tiene datos sembrados.

Salida, comprobada por `scripts/quality-gate.py`

- Quality Gate de SonarQube aprobado.
- Ninguna vulnerabilidad crítica sin aceptar en npm audit o Dependency-Check.
- Ninguna alerta alta sin aceptar en ZAP web o ZAP API.
- Colección de Insomnia sin fallas.
- Umbrales de k6 cumplidos.

Una prueba crítica fallida detiene el pipeline en su etapa, y por eso el despliegue final no se alcanza.

17. Gestión de defectos

Cada defecto registra pasos, resultado esperado, resultado obtenido, severidad, prioridad y evidencia.

Severidad	Significado
Crítica	Pérdida o exposición de datos clínicos, o sistema inutilizable
Alta	Una función principal falla, sin alternativa
Media	Falla con alternativa, o error incorrecto sin daño de datos
Baja	Efecto menor o de presentación

Prioridad: P1 corregir antes de la entrega, P2 corregir en esta versión, P3 planificar.

Ciclo: abierto, en corrección, verificado y cerrado. El defecto se cierra solo después de repetir la prueba. En GitHub se reporta con `.github/ISSUE_TEMPLATE/defecto.yml`.

18. Métricas y resultados

Los resultados de cada herramienta, con su fecha y reporte, están en 13-informe-final.md.

19. Riesgos residuales

Se detallan en el informe final y en la matriz de riesgos. Los principales son las copias en el mismo disco (R-16), el servidor único (R-17) y el ambiente de Azure sin aprovisionar (R-18).

20. Decisión GO / NO-GO

La decisión y su justificación están en 13-informe-final.md. Se basa solo en los criterios de la sección 16 y en lo que realmente se ejecutó.

MediCore · Casos de prueba y evidencias

Selección de casos de los temas exigidos por el documento: roles, citas, laboratorio, APIs, Redis, RabbitMQ, seguridad y móvil. Cada caso apunta a una prueba automatizada que se puede repetir. La lista completa son las propias suites.

Prioridad: A crítica, M media. El resultado global de cada suite está en 13-informe-final.md.

Citas y concurrencia

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-01	A	Reservas simultáneas al mismo horario	Una sola cita activa, las demás 409	`Solicitudes_simultaneas_al_mismo_horario_producen_una_unica_reserva_activa`	Aprobado
CP-02	A	Alta demanda en varios horarios	Nunca más de una cita por horario	`Alta_demanda_en_varios_horarios_nunca_supera_una_cita_por_horario`	Aprobado
CP-03	A	Mismo pedido reenviado con la misma clave	Devuelve la cita original	`Reenviar_la_misma_solicitud_con_la_misma_clave_devuelve_la_cita_original_sin_duplicar`	Aprobado
CP-04	A	Reenvíos simultáneos con la misma clave	Se crea una sola cita	`Reenvios_simultaneos_con_la_misma_clave_crean_una_sola_cita`	Aprobado
CP-05	M	Misma clave con otra solicitud	409	`Reutilizar_una_clave_con_otra_solicitud_devuelve_409`	Aprobado
CP-06	A	Cancelar libera el horario	Otro paciente puede reservar	`Cancelar_libera_el_horario_para_otro_paciente_y_es_idempotente`	Aprobado
CP-07	M	Cancelaciones simultáneas	Sin errores ni dobles cambios	`Cancelaciones_simultaneas_de_la_misma_cita_son_seguras`	Aprobado
CP-08	A	Reservas con 200 usuarios mientras se consulta	0 solapes y 0 % de error inesperado	k6 `spike`, `reports/k6/integridad.txt`	Aprobado
CP-09	A	Dos reservas al mismo horario, una tras otra, en la web	La segunda se rechaza	Cypress, `Prevención de doble reserva`	Aprobado

Redis

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-10	M	Primer acceso y segundo acceso	MISS con TTL, luego HIT	`Cache_MISS_puebla_Redis_con_TTL_y_el_siguiente_acceso_es_HIT`	Aprobado
CP-11	A	Reservar o cancelar	La disponibilidad cacheada se invalida	`Reservar_y_cancelar_invalidan_la_disponibilidad_cacheada`	Aprobado
CP-12	M	Cambio de horario del médico	Se invalidan todas sus fechas	`Cambiar_el_horario_de_atencion_invalida_todas_las_fechas_cacheadas_del_medico`	Aprobado
CP-13	M	Dato vencido	Se recarga desde PostgreSQL	`Dato_vencido_el_cache_expira_por_TTL_y_se_recarga_desde_la_base_de_datos`	Aprobado
CP-14	A	Redis caído en disponibilidad y reserva	Siguen funcionando	`Si_Redis_no_esta_disponible_la_disponibilidad_y_la_reserva_siguen_funcionando`	Aprobado
CP-15	A	Redis caído en autenticación	Login 503 y token rechazado	`Con_redis_caido_el_login_devuelve_503_en_lugar_de_un_500`, `Con_redis_caido_un_token_valido_deja_de_aceptarse`	Aprobado

RabbitMQ y laboratorio

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-16	A	Broker detenido al registrar un resultado	Se guarda y el evento se publica al volver	`Con_el_broker_detenido_el_resultado_se_guarda_y_el_evento_se_publica_al_volver_el_broker`	Aprobado
CP-17	A	Broker detenido al reservar	Se guarda y la notificación llega al volver	`Con_el_broker_detenido_la_cita_se_guarda_y_la_notificacion_llega_al_volver_el_broker`	Aprobado
CP-18	A	Mensaje repetido	No duplica la notificación	`Un_mensaje_reentregado_no_duplica_la_notificacion`, `Un_evento_repetido_no_duplica_la_notificacion`	Aprobado
CP-19	M	El consumidor falla dos veces	Se reintenta y termina procesado	`Si_el_consumidor_falla_dos_veces_el_mensaje_se_reintenta_y_termina_procesado`	Aprobado
CP-20	A	Registrar, consultar y notificar	Laboratorio y el médico lo ven, con su estado de notificación	Cypress, `Laboratorio · registrar, consultar y notificar un resultado`	Aprobado
CP-21	M	Reintento de notificación	No duplica	`Reintentar_si_la_notificacion_ya_existe_no_hace_nada`	Aprobado

Roles, recursos y sesión

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-22	A	Cada rol entra solo a lo suyo	Sin permiso, respuesta 403 o redirección	Cypress, `Permisos por rol (web y API)`	Aprobado
CP-23	A	Paciente pide datos ajenos (IDOR)	404 y auditoría	Insomnia, carpetas 3, 5 y 6	Aprobado
CP-24	A	Registros clínicos no se editan	PUT, PATCH y DELETE devuelven 405	Insomnia, `ClinicalApiTests`	Aprobado
CP-25	A	Bloqueo tras 5 intentos	Cuenta bloqueada aun con la clave correcta	`Cuenta_se_bloquea_tras_cinco_intentos_fallidos_incluso_con_la_contrasena_correcta`	Aprobado
CP-26	A	Reutilizar un refresh rotado	Se revoca la sesión	`Reutilizar_un_refresh_token_ya_rotado_revoca_toda_la_sesion`	Aprobado
CP-27	M	Token vencido	401	`Token_vencido_devuelve_401`	Aprobado
CP-28	A	Datos clínicos en auditoría y bitácoras	No aparecen	`Ni_la_auditoria_ni_las_bitacoras_contienen_el_contenido_clinico`	Aprobado
CP-29	M	Carácter nulo en búsqueda y datos	400, no 500	`Un_caracter_nulo_en_busqueda_o_en_datos_devuelve_400_y_no_500`	Aprobado

App móvil

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-30	A	Login válido e inválido	Entra o muestra el error	Maestro 01 y 02	8/8 antes de los eventos, repetir
CP-31	A	Reservar, consultar y cancelar	La cita aparece y se cancela	Maestro 03 y 04	8/8 antes de los eventos, repetir
CP-32	A	Resultados, recetas y notificaciones	Se abre el detalle	Maestro 05, 06 y 07	8/8 antes de los eventos, repetir

Accesibilidad

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-33	M	WCAG A y AA en 14 pantallas	0 violaciones	`accesibilidad-axe.cy.js`, `reports/accesibilidad/axe.json`	Aprobado

ID	Prioridad	Caso	Resultado esperado	Prueba	Estado
CP-34	M	Pantalla de 375 px	Sin desplazamiento horizontal	`pantalla-pequena.cy.js`	Aprobado

MediCore · Registro de defectos

Defectos reales encontrados durante el proyecto. La causa detallada de cada uno, y los errores de pruebas o de entorno, están en 04-registro-de-hallazgos.md.

La severidad y la prioridad siguen la sección 17 del plan de pruebas. Se asignaron al consolidar este registro, a partir del efecto observado.

Resumen

Severidad	Cerrados	Abiertos
Crítica	0	0
Alta	7	0
Media	14	0
Baja	3	3

Defectos cerrados

ID	Módulo	Severidad	Prioridad	Pasos	Esperado	Obtenido	Corrección	Evidencia
H4-01	Agenda	Alta	P1	Enviar 12 reservas simultáneas al mismo horario	Una 201 y las demás 409	11 respuestas 500 por deadlock	`pg_advisory_xact_lock` y reintento ante 40P01	`AppointmentsApiTests`
H4-03	Web	Alta	P1	Abrir Agenda o Mi agenda	La página carga	Error 500	`_Shell` reenvía la sección de scripts	Cypress `recepcion-agenda`, `medico-agenda`
H4-04	Web	Alta	P1	Cambiar fecha en Reservar y pulsar Confirmar	Se crea la cita	La página queda igual	`action` explícito	Cypress `recepcion-reserva`
H4-05	Web	Alta	P1	Confirmar la reserva después de H4-04	Se crea la cita	Página vacía con 400	Token antifalsificación explícito	Cypress `recepcion-reserva`
H4-12	Web	Media	P2	Escribir una fecha en el filtro de agenda	La agenda cambia de fecha	Conservaba la fecha anterior	Envío automático al completar la fecha	Cypress `recepcion-agenda`

ID	Módulo	Severidad	Prioridad	Pasos	Esperado	Obtenido	Corrección	Evidencia
H5-04	Web	Baja	P2	Abrir el expediente de un paciente sin relación	Mensaje con 404	Página en blanco	Página `/Estado`	Cypress `medico-expediente`
H6-02	Mensajería	Alta	P1	Detener y reiniciar RabbitMQ, registrar un resultado	Se crea la notificación	El consumidor dejaba de recibir	La conexión se recupera sin reemplazarla	`MessagingTests`
H7-08	Móvil	Media	P1	Instalar el APK y abrir la app	La app abre	Se cierra al abrir	Ensamblados dentro del APK	`documentacion/evidencias/entrega-7-maestro.md`
H7-09	Móvil	Media	P1	Abrir la app tras H7-08	La app abre	Error de bibliotecas nativas	`extractNativeLibs`	`documentacion/evidencias/entrega-7-maestro.md`
HW-03	Web	Media	P2	Entrar como Administrador en 360 px	Sin desplazamiento horizontal	La página medía 427 px	Grid con `minmax(0,1fr)`	Cypress `pantalla-pequena`
H8-01	Contenedores	Media	P1	Levantar QA en volúmenes nuevos	Web saludable	`unhealthy` al escribir en `/keys`	Dueño correcto en el Dockerfile	`documentacion/evidencias/entrega-8-verificacion.md`
H8-02	Datos QA	Media	P1	Ejecutar `--seed-qa` en una base nueva	Siembra completa	Fallaba sin la Dra. Méndez	Orden del sembrado corregido	`documentacion/evidencias/entrega-8-verificacion.md`
HP-02	Mensajería	Alta	P1	Ejecutar el perfil Spike de k6 y contar pendientes	Eventos procesados	14 405 eventos pendientes por un tope de 20 cada 2 s	Lote de 200 y drenado continuo	`reports/k6/corrida.txt`
HS-01	Código	Media	P2	Analizar con SonarQube	0 bugs	S1751 en `MediCoreApi.cs` y `MobileApiClient.cs`	Bucles corregidos	`reports/sonarqube/medidas.json`
HA-01	Accesibilidad	Media	P2	Ejecutar axe en el calendario	Contraste AA	Contraste insuficiente	Colores en `app.css`	`reports/accesibilidad/axe.json`

ID	Módulo	Severidad	Prioridad	Pasos	Esperado	Obtenido	Corrección	Evidencia
HZ-01	API	Media	P2	Buscar o guardar texto con un carácter nulo (`%00`)	Respuesta 400	Error 500 de PostgreSQL	`InvalidTextExceptionHandler`	`PatientsApiTests`, `reports/zap/zap-api.html`
HC-01	Código	Baja	P3	Ejecutar `dotnet format --verify-no-changes` en Jenkins	Sin cambios	7 archivos con formato incorrecto	Formato aplicado	Etapa 3 de Jenkins
HS-02	Código	Media	P2	Ejecutar SonarQube sobre código nuevo	Duplicación de 3 % o menos	9,3 % de duplicación y 49 % de cobertura	Lectura de errores compartida y 4 pruebas	Corrida 2 de Jenkins, `reports/sonarqube/quality-gate.json`
H8-08	Datos	Alta	P1 antes de producción	Revisar la infraestructura	Copias automáticas	No hay respaldos	Copias cifradas cada 6 horas, verificadas, y restauración probada con pérdida total	`documentacion/evidencias/respaldos.md`, `reports/respaldos/simulacro.txt`
HP-03	Agenda	Media	P1	Enviar 100 reservas simultáneas (`scripts/perf-citas.sh 100`)	Todas responden 201 o 409	1 de cada 100 falló con error 500: `53300 too many clients`. API y worker podían abrir más conexiones que `max_connections`	Tope de 40 conexiones por proceso en `docker-compose.qa.yml`. Repetido con 100 y 200 reservas: 0 errores	`reports/k6/citas-mismo.json`, `reports/k6/citas-distintas.json`
HS-03	Código	Media	P2	Ejecutar la etapa 16 con los scripts nuevos	Puerta de SonarQube aprobada	Corridas 7 y 8 rechazadas: 8 y luego 1 incidencia nueva, y puntos de seguridad sin revisar; las etapas 17 y 18 no se ejecutaron	Incidencias corregidas sin bajar la puerta; una expresión regular reemplazada y un punto de seguridad revisado con justificación	`reports/jenkins/corrida-7.txt`, `reports/jenkins/corrida-8.txt`, `reports/sonarqube/quality-gate.json`
H9-01	Contenedores	Media	P1	Crear QA desde Jenkins y reiniciar Docker	QA vuelve a arrancar	API, web y proxy terminan con código 127: montaban archivos del workspace de Jenkins	Script de salud y Caddyfile dentro de las imágenes	`docker/proxy.Dockerfile`, `scripts/qa.sh smoke`
H9-02	Infraestructura	Media	P2	Aplicar `site.yml` en un Ubuntu mínimo	Docker instalado	Falla: faltaba `python3-debian`	Paquete agregado al rol `base`	`documentacion/evidencias/iac-sin-costo.md`

ID	Módulo	Severidad	Prioridad	Pasos	Esperado	Obtenido	Corrección	Evidencia
HZ-02	Web	Baja	P3	Ejecutar ZAP	Cabeceras presentes	Faltaban `Permissions-Policy` y `Cross-Origin-Resource-Policy`	Agregadas en Caddy	`reports/zap/zap.html`

Defectos abiertos

ID	Módulo	Severidad	Prioridad	Pasos	Esperado	Obtenido	Decisión	Evidencia
HQ-01	Datos QA	Baja	P2	Revisar `.env.qa`	Contraseña sembrada fuerte	Contraseña de demostración de 5 letras	Ficticia y solo local. Regenerar con `qa.sh init` antes de exponer QA	`.env.qa` (no versionado)
HZ-03	Web	Baja	P3	Ejecutar ZAP	Cabeceras COEP y COOP	No están	Aceptado en QA; revisar con HTTPS	`reports/zap/zap.html`
HJ-04	Pipeline	Baja	P3	Reiniciar Docker Desktop	Jenkins conserva su historial	Se pierde `/var/jenkins_home` y la numeración vuelve a 1	Aceptado: las consolas se guardan en `reports/jenkins/`	`reports/jenkins/corridas.txt`

Falsos positivos

Herramienta	Alerta	Justificación
ZAP	HTTP Only Site	QA local por HTTP. En Azure se usa HTTPS
ZAP	Unexpected Content-Type	Ocurre en rutas que no son de la API
Dependency-Check	4 críticas en bibliotecas cliente	Son CVE de servidores PostgreSQL y Redis. Ver 15-seguridad-y-accesibilidad.md
ZAP	Private IP Disclosure	Es la IP interna registrada en la auditoría

MediCore · Informe final

Corte: 2026-11-06, rama `Dev_Diego`. Los resultados son de la corrida 9 de Jenkins, salvo donde se indica otra fuente.

Conclusión

Decisión propuesta: NO-GO para producción y GO condicionado para QA.

Todos los criterios automáticos se cumplieron: la agenda resistió la carga sin doble reserva y una caída de RabbitMQ no perdió datos clínicos. Aun así, MediCore no debe pasar a producción: faltan entregables exigidos y nunca se desplegó en Azure.

La decisión final corresponde al equipo.

Métricas

Área	Herramienta	Resultado	Criterio	Estado
Unitarias e integración	xUnit	304 de 304	100 %	Cumple
API	Insomnia	61 solicitudes, 65 de 65 comprobaciones	100 %	Cumple
Web	Cypress, Chrome	116 de 116	100 %	Cumple
Móvil	Maestro	8 de 8 con el código final, en emulador contra QA	5 a 8 flujos	Cumple
Calidad	SonarQube	Puerta aprobada, 0 bugs, 74,2 % de cobertura	Puerta aprobada	Cumple
SAST	Fortify	No ejecutado, sin licencia	Exigido	**Pendiente**
SCA	npm audit y NuGet	0 vulnerables	0 críticas	Cumple
SCA	Dependency-Check	4 críticas, todas falsos positivos justificados	0 sin aceptar	Cumple
DAST	ZAP web y API con 5 roles	0 alertas altas, 0 errores 500, 40 de 45 operaciones alcanzadas	0 altas sin aceptar	Cumple
Rendimiento	k6	Umbrales cumplidos	Declarados antes	Cumple
Accesibilidad	axe-core	14 pantallas, 0 infracciones	0	Cumple
Usabilidad	Prueba con personas	No aplicada	5 participantes	**Pendiente**

Área	Herramienta	Resultado	Criterio	Estado
Pipeline	Jenkins	Corrida 9: 18 etapas completas, con criterios de salida cumplidos y aprobación humana en la etapa 17	Bloquea si falla	Cumple
Infraestructura	Terraform y Ansible sin Azure	Terraform 3 de 3 con proveedor simulado; Ansible sin fallas en Ubuntu desechable	Reproducible	Cumple en simulación
Nube	Azure	Sin desplegar: el equipo decidió no usar recursos de pago	Exigido	**Pendiente**

Rendimiento

La prueba de reservas simultáneas (`scripts/perf-citas.sh`) se ejecutó después de la corrida 9 y su corrección no pasó por Jenkins. Ver 07-rendimiento.md.

Perfil	Peticiones	req/s	p95 lectura	p95 reserva	Error inesperado	Reservas	409
Load, corrida 5	20 399	75,2	5,3 ms	8,9 ms	0 %	3260	41
Stress, corrida manual	41 620	196,4	6,2 ms	8,6 ms	0 %	6672	186
Spike, corrida manual	43 478	328,0	7,4 ms	11,5 ms	0 %	6954	188

Después de Stress y Spike hubo 0 solapes, 0 eventos pendientes y 0 mensajes en la DLQ. Se ejecutaron antes de los últimos cambios, que no tocaron agenda ni mensajería.

No se alcanzó el punto de quiebre. k6, la API y la base de datos compartieron el equipo, así que estas cifras no representan Azure.

Defectos

Se registraron 27 defectos: 24 cerrados y 3 abiertos, ninguno crítico ni de severidad alta. Todos están en GitHub Issues. Las pruebas de esta etapa encontraron defectos reales que las suites anteriores no veían:

- **k6:** el outbox acumuló 14 405 eventos pendientes después del pico.
- **ZAP:** un carácter nulo provocaba un error 500.
- **SonarQube:** rechazó código nuevo por duplicación y cobertura.
- **axe:** el calendario no cumplía el contraste.
- **Reinicio de Docker:** QA no volvía a arrancar si Jenkins lo había creado.
- **Ansible:** el playbook no instalaba un paquete que necesitaba.

Escenario crítico

Pregunta del documento	Resultado	Evidencia
¿Hay doble reserva con alta demanda?	No: 0 solapes por médico y por paciente	`reports/k6/integridad.txt`, `AppointmentsApiTests`
¿Hay consistencia mientras médicos consultan?	Sí: lecturas y reservas simultáneas sin errores inesperados	`reports/k6/`
¿Se pierde un resultado si cae RabbitMQ?	No: responde 201 y la notificación queda pendiente	`documentacion/evidencias/resiliencia-qa.md`
¿Se recupera y evita duplicados?	Sí: se envía al volver y el paciente recibe una sola	`documentacion/evidencias/resiliencia-qa.md`

Flujo asíncrono de notificaciones

1. El resultado y su evento `LabResultReady` se guardan en la misma transacción de PostgreSQL.
2. El Worker publica el evento en RabbitMQ con confirmación del broker.
3. El consumidor crea la notificación y confirma el mensaje.
4. Cada notificación emite `PatientNotification`, que la marca como despachada.

Con el Worker o RabbitMQ detenidos, en QA se comprobó lo siguiente:

- La API respondió 201 y la notificación quedó `Pending`.
- Al volver el servicio pasó a `Sent`.
- El paciente recibió exactamente una notificación.

Con Redis detenido, el inicio de sesión respondió 503 y el token activo 401. Volvió a funcionar al iniciar Redis.

Respaldos

PostgreSQL se respalda cada 6 horas, con cifrado y verificación. En un simulacro se destruyeron la base y el volumen de respaldos, y se restauró desde una copia exportada: 14 tablas idénticas, 36 849 filas y 72 s de recuperación en este equipo. La pérdida máxima esperada es de 6 horas. Las copias siguen en el mismo disco salvo exportación manual. Detalle: evidencias/respaldos.md.

Pipeline

Corrida	Resultado	Qué demostró
2	Falló	SonarQube rechazó el código nuevo y Dependency-Check no pudo abrir su base
3	Falló	El smoke no llegaba a QA desde el contenedor de Jenkins

Corrida	Resultado	Qué demostró
4	Falló	La colección de API falló porque QA arrastraba datos de corridas anteriores
5	Inestable	Las 18 etapas llegaron al final, con Fortify y Maestro sin ejecutar
6	Falló	`SIMULAR_FALLO` detuvo el pipeline: no se ejecutaron Aprobación ni Deploy final
7	Falló	SonarQube rechazó 8 incidencias nuevas y un punto de seguridad sin revisar. La puerta no se bajó
8	Falló	SonarQube rechazó 1 incidencia y otro punto de seguridad. Lo demás de la etapa 16 cumplió
9	Inestable	Puerta de SonarQube y criterios de salida cumplidos, con los respaldos incluidos. Fortify y Maestro quedan como aviso. Diego aprobó la etapa 17 y la etapa 18 registró que no hay servidor de Azure

La corrida 5 terminó inestable a propósito: Fortify y Maestro no se ejecutaron y el pipeline no los oculta. La aprobación de la etapa 17 se dio de forma automatizada, y la etapa 18 solo registró que no hay servidor de Azure.

En la corrida 9 la aprobación la dio una persona desde Jenkins, sin automatizarla. Jenkins numera esa corrida como 3 porque perdió su historial al reiniciar Docker (HJ-04).

Las consolas están en `reports/jenkins/`.

Por qué no es GO para producción

Bloqueo	Por qué importa	Quién lo resuelve
No hay despliegue en Azure (R-18)	La ejecución sin costo no demuestra la nube real	Equipo, si decide pagar
Usabilidad sin aplicar	Es un entregable exigido	Equipo, con 5 personas
Fortify sin ejecutar	Es un entregable exigido	Equipo, con licencia

Riesgos residuales

Riesgo	Nivel	Tratamiento
Copias en el mismo disco (R-16)	Medio	Respaldos probados. Exportarlos o llevarlos a otro lugar con `qa.sh export`
Ambiente de nube sin aprovisionar (R-18)	Alto	Decidir el pago y seguir `06-infraestructura.md`
Sin Redis no hay sesión (R-14)	Medio	Aceptado por seguridad. Vigilar el servicio
Servidor único (R-17)	Medio	Aceptado para QA

Riesgo	Nivel	Tratamiento
ZAP sin HTTPS real	Bajo	Repetir sobre HTTPS si se despliega
Puntos sensibles de Terraform	Medio	Revisar antes de aplicar
Avisos de Redis 2026	Medio	Revisar y actualizar la imagen
Rendimiento medido en un solo equipo	Medio	Repetir en Azure
Contraseña de semilla débil en QA local	Bajo	`qa.sh init` genera una fuerte. Usarla antes de exponer QA

Criterios para cambiar la decisión

MediCore puede pasar a GO cuando se cumpla todo esto:

1. Usabilidad con 5 personas, SUS de 68 o más y 80 % de tareas sin ayuda.
2. Fortify ejecutado y sus hallazgos clasificados.
3. QA desplegado en Azure, con ZAP y k6 repetidos allí.

MediCore · Guion de defensa práctica

Pasos para responder en vivo lo que pide el documento: ejecutar el pipeline, provocar una falla, detener Redis o el consumidor, mostrar un hallazgo de ZAP, explicar el p95, recrear infraestructura y justificar la decisión GO / NO-GO.

Presentación

Diapositivas con notas por sección: https://claude.ai/artifact/1bNnrBdRYNPbqvP6RzfK5c

Antes de presentar

Hazlo con 30 minutos de anticipación. Docker Desktop debe estar encendido.

```
bash scripts/qa.sh up
```

```
bash scripts/qa.sh seed
```

```
bash scripts/qa.sh smoke
```

```
docker compose -p medicore-ci --env-file ci/.env.ci -f ci/docker-compose.ci.yml up -d
```

Deja abiertas estas páginas:

- Web: `http://127.0.0.1:8080`
- Jenkins: `http://127.0.0.1:8090`
- SonarQube: `http://127.0.0.1:9000`
- `reports/zap/zap-api.html`

El pipeline completo tarda demasiado para la exposición. Lleva abierta una corrida terminada y muestra en vivo solo la que falla.

1. Ejecutar el pipeline

1. Abre Jenkins, entra a **MediCore** y pulsa **Build with Parameters**.
2. Deja `SIMULAR_FALLO` sin marcar y pulsa **Build**.
3. Muestra las 18 etapas en la vista del trabajo.
4. Si te piden una corrida completa, muestra la última que terminó y sus reportes en **Artifacts**.

Qué decir: cada herramienta corre en contenedor, por eso el mismo pipeline funciona en cualquier equipo con Docker.

2. Provocar una prueba fallida

1. Pulsa **Build with Parameters**.

2. Marca `SIMULAR_FALLO` y pulsa **Build**.
3. La etapa 16 falla con el mensaje `Fallo simulado`.
4. Muestra que las etapas 17 y 18 quedaron omitidas.

Qué decir: una prueba crítica fallida, una vulnerabilidad crítica o un Quality Gate rechazado impiden el despliegue final.

3. Detener Redis

Inicia sesión en la web y detén Redis:

```
docker stop medicore-qa-redis-1
```

Qué ocurre: el inicio de sesión responde 503 y un token activo responde 401. No aparece un error 500.

Qué decir: el cache falla abierto y la sesión falla cerrado. Si no podemos comprobar una sesión, no entregamos datos clínicos.

Recupera el servicio:

```
docker start medicore-qa-redis-1
```

4. Detener el consumidor o RabbitMQ

Detén el Worker, que publica y consume eventos:

```
docker stop medicore-qa-worker-1
```

Registra un resultado de laboratorio en la web. Sigue guardado y su notificación queda en `Pending`.

Vuelve a iniciar el Worker:

```
docker start medicore-qa-worker-1
```

La notificación aparece como enviada. En el ensayo tardó segundos: ver `documentacion/evidencias/resiliencia-qa.md`.

Para detener el broker, usa `medicore-qa-rabbitmq-1` en lugar del Worker. El resultado también queda guardado.

Qué decir: el dato y su evento se guardan en la misma transacción. La entrega es al menos una vez, y el consumidor no duplica.

5. Mostrar un hallazgo de ZAP

1. Abre `reports/zap/zap-api-medico.html` y `reports/zap/cobertura-api.json`.
2. Muestra que no hay alertas altas.
3. Explica el defecto HZ-01: un carácter nulo en la búsqueda provocaba un error 500.
4. Muestra la corrección en `src/MediCore.Api/Security/InvalidTextExceptionHandler.cs`.
5. Muestra la prueba `Un_caracter_nulo_en_busqueda_o_en_datos_devuelve_400_y_no_500`.

Qué decir: ZAP se ejecutó con los 5 roles, con sesiones reales y datos válidos. Alcanzó 40 de las 45 operaciones del contrato, sin alertas altas ni errores 500. Las otras 5 están fuera a propósito. Antes el análisis era parcial porque usaba solo Administrador; lo corregimos y lo medimos.

6. Explicar el p95

El p95 es el tiempo que cumplen 95 de cada 100 peticiones. Refleja mejor la experiencia que el promedio, porque un promedio esconde a los usuarios más lentos.

Ejemplo: en Load, el p95 de reserva fue 8,9 ms. El 95 % de las reservas respondió en ese tiempo o menos, frente a un umbral de 1000 ms.

Límite: k6, la API y la base de datos corrieron en el mismo equipo, sin red real. Los números no representan una VM de Azure.

Hallazgo: la API respondía rápido, pero el perfil Spike dejó 14 405 eventos pendientes. Por eso también se mide la integridad después de la carga.

7. Recrear la infraestructura

Local, conservando los datos:

```
bash scripts/qa.sh down
```

```
bash scripts/qa.sh up
```

Ejecución de Terraform y Ansible sin Azure y sin costo (Ansible construye la aplicación, así que tarda varios minutos):

```
bash scripts/iac-simulacion.sh all
```

Qué decir: decidimos no usar recursos de pago, así que no hay despliegue real en Azure. Terraform recorrió aprovisionar, modificar y destruir con un proveedor simulado, y Ansible se aplicó en un Ubuntu desechable hasta dejar la aplicación funcionando. Eso prueba nuestro código, no la nube. La evidencia está en `documentacion/evidencias/iac-sin-costo.md`.

8. Restaurar un respaldo

Destruye QA y lo recupera desde una copia exportada. Tarda cerca de un minuto y medio:

```
QA_CONFIRM=medicore-qa bash scripts/simulacro-respaldo.sh
```

Qué decir: se borró la base y también el volumen de respaldos. Las 14 tablas volvieron idénticas. La pérdida máxima esperada es de 6 horas y las copias siguen en el mismo disco salvo que se exporten.

9. Justificar la decisión

La decisión está en la sección "Decisión GO / NO-GO" de `documentacion/13-informe-final.md`.

Separa lo que está probado de lo que falta. Si te preguntan por algo no ejecutado, dilo sin rodeos y muestra en qué lista de pendientes aparece.

MediCore · Reporte de seguridad y accesibilidad

Resultados de la corrida 5 de Jenkins, del 2026-11-06. Todas las pruebas se hicieron sobre el ambiente QA local, con datos ficticios.

Resumen

Herramienta	Resultado	Reporte
SonarQube	Quality Gate aprobado, 0 bugs, 0 vulnerabilidades	`reports/sonarqube/`
npm audit	0 vulnerabilidades	`reports/npm-audit/audit.json`
NuGet	0 paquetes vulnerables	`reports/dependency-check/nuget-vulnerable.txt`
Dependency-Check	58 coincidencias, 4 críticas: todas apuntan a servidores, no a las bibliotecas	`reports/dependency-check/`
OWASP ZAP web	0 fallas, 141 reglas aprobadas, 2 advertencias	`reports/zap/zap.html`
OWASP ZAP API	0 fallas, 119 reglas aprobadas, 2 advertencias	`reports/zap/zap-api.html`
Fortify	**No ejecutado**: no hay licencia	`scripts/sec-fortify.sh`
axe-core	14 pantallas, 0 infracciones	`reports/accesibilidad/axe.json`

SonarQube

El Quality Gate "MediCore" exige: 0 vulnerabilidades, seguridad A, confiabilidad B o mejor, mantenibilidad A, cobertura de 60 % o más y duplicación de 3 % o menos. En el código nuevo exige cobertura de 80 % y duplicación de 3 % o menos.

Métrica	Valor
Bugs	0
Vulnerabilidades	0
Code smells	124
Cobertura	74,2 %
Cobertura del código nuevo	80,5 %
Duplicación	0,2 %
Líneas de código	10 637

La puerta sí rechazó una corrida. En la corrida 2 falló por 49 % de cobertura y 9,3 % de duplicación en el código nuevo. Se compartió la lectura de errores entre web y móvil, se agregaron 4 pruebas y los sembradores de datos ficticios salieron del cálculo de cobertura.

Revisión de los 29 puntos sensibles

Sonar los marca para revisión humana. Se revisaron uno por uno, pero el estado en el servidor sigue como "por revisar".

Cantidad	Ubicación	Conclusión
12	SQL en las pruebas de Cypress	Solo pruebas: preparan datos ficticios en QA
4	Generadores aleatorios en k6, Cypress y scripts	Solo datos de prueba; no se usa para seguridad
2	`IgnoreAntiforgeryToken` en `Error` y `Estado`	Aceptado: páginas de solo lectura, sin datos ni cambios
3	Imágenes de CI y Android corren como root	Aceptado: herramientas locales, nunca en producción
5	Terraform: IP pública, acceso público, cifrado e identidad	**Revisar antes de aplicar en Azure**
3	HTTP en k6, redirecciones en la imagen de Jenkins y contraseña en la colección	Red interna de QA y variable de entorno, sin secreto real

Dependencias

Las 58 coincidencias de Dependency-Check salen de `Npgsql.EntityFrameworkCore.PostgreSQL.dll` y `StackExchange.Redis.dll`. La herramienta confundió estas bibliotecas cliente con los servidores PostgreSQL y Redis.

Las 4 críticas se aceptaron en `ci/riesgos-aceptados.json`, con motivo:

CVE	Afecta a	Por qué no aplica
CVE-2018-1115	PostgreSQL servidor	QA usa PostgreSQL 17.11
CVE-2018-16850	PostgreSQL 10 y 11	QA usa PostgreSQL 17.11
CVE-2019-10211	Instalador de PostgreSQL en Windows	QA usa imagen Linux
CVE-2025-49844	Redis servidor, corregido en 7.4.6	QA usa Redis 7.4.12

Lo que no se descartó: el servidor sí es software que corremos. Las coincidencias CVE-2026-25243 y CVE-2026-23631 figuran hasta Redis 8.6.3, y QA usa 7.4.12. Ambas requieren un atacante ya autenticado en Redis. Redis corre en una red interna con contraseña. Queda como riesgo residual: hay que revisar el aviso oficial de Redis y actualizar la imagen antes de producción.

OWASP ZAP

Se analizó la web con un escaneo activo acotado. La API se analizó cinco veces con su contrato OpenAPI, una por rol: Laboratorio, Recepción, Médico, Paciente y Administrador. Cada análisis usa una sesión real, identificadores reales de QA y cuerpos válidos, para pasar la validación y llegar a la lógica clínica. Se excluyen cerrar sesión, renovar sesión y cambiar roles, para no perder la sesión ni alterar las cuentas de prueba.

Cobertura medida: `reports/zap/cobertura-api.json` cuenta, con el registro del proxy, las respuestas por ruta y rol. 40 de las 45 operaciones del contrato respondieron con éxito al menos una vez, y ninguna petición produjo un error 500. Las 5 restantes son las 3 excluidas y las 2 rutas de salud, que no se cuentan en la medición.

Rol	Rutas probadas	Con respuesta exitosa	Errores 500
Laboratorio	87	13	0
Recepción	84	15	0
Médico	99	28	0

Rol	Rutas probadas	Con respuesta exitosa	Errores 500
Paciente	99	30	0
Administrador	85	26	0

Las rutas probadas incluyen variantes que genera ZAP, por eso superan a las del contrato.

Límite: ZAP busca fallas técnicas conocidas. No demuestra que un usuario no pueda leer datos ajenos; eso lo cubren `ClinicalApiTests`, `LaboratoryApiTests` e Insomnia. QA corre sin HTTPS.

Alerta	Dónde	Decisión
HTTP Only Site	Web	QA local sin HTTPS. En Azure se usa HTTPS
COEP faltante	Web	Aceptada en QA. Revisar con HTTPS
Unexpected Content-Type	API	Falso positivo: rutas fuera de la API
Private IP Disclosure	API	Es la IP interna registrada en la auditoría

Defectos corregidos:

- **HZ-01:** un carácter nulo provocaba un error 500. Ahora responde 400, y hay una prueba de regresión.
- **HZ-02:** se agregaron las cabeceras `Permissions-Policy` y `Cross-Origin-Resource-Policy`.

Controles de la aplicación

Tema del documento	Evidencia	Resultado
RBAC	`IdentityApiTests`, Insomnia, Cypress `acceso-permisos`	Aprobado
Autorización por recurso	`ClinicalApiTests`, `LaboratoryApiTests`, Insomnia	Aprobado
Datos sensibles	Pruebas de auditoría y bitácoras	Aprobado
Sesiones	JWT de 15 minutos, refresco de un solo uso, bloqueo tras 5 intentos	Aprobado
API Security	Validación, límite por IP, cabeceras, ZAP con 5 roles	Aprobado
Trazabilidad	`audit_entries`, `GET /api/audit`	Aprobado

Accesibilidad

`accesibilidad-axe.cy.js` analiza 14 pantallas con axe-core 4.10.3 y reglas WCAG 2.0 y 2.1, niveles A y AA.

- **Resultado:** 0 infracciones.
- **Defecto corregido (HA-01):** el calendario tenía contraste insuficiente.
- **Límite:** axe no sustituye la revisión con lector de pantalla ni a las personas de la prueba de usabilidad. Solo cubre lo que se puede detectar de forma automática.

Pendientes

1. Ejecutar Fortify si se consigue licencia. Es un entregable exigido.
2. Revisar los 5 puntos de Terraform antes de aplicar en Azure.
3. Revisar los avisos de Redis y actualizar la imagen antes de producción.

Evidencia de la entrega 6: laboratorio y notificaciones asíncronas

Ejecutada el 2026-10-27 12:39:44 -0600 con `scripts/evidencia_laboratorio.py` contra la pila de desarrollo (PostgreSQL, Redis y RabbitMQ de docker compose; API en Development). Solo datos ficticios `CYP-E2E-*`. El informe contiene identificadores y estados, no contenido clínico.

A. Broker disponible: registro, notificación asíncrona y lectura del paciente

- `POST /api/labs/results` → **201** (resultado `67b037c0-5ae3-465a-8347-6f590b79450c`, estado inicial de la notificación: `Pending`).
- La notificación pasó a `Sent` de forma asíncrona: **True**; notificaciones guardadas para el resultado: **1**.
- Paciente `GET /api/notifications?unreadOnly=true` → **200**, notificaciones del resultado: **1** (tipo `LabResultReady`, sin leer: `True`).
- `GET /api/notifications/unread-count` → **200**, sin leer: **1**.
- `POST /api/notifications/{id}/read` → **204**.
- El paciente lee su propio resultado: `GET /api/labs/results/{id}` → **200**.

B. RabbitMQ detenido: el resultado no se pierde y el evento pendiente se recupera

- `docker compose stop rabbitmq` → código 0.
- Con el broker detenido, `POST /api/labs/results` → **201** (resultado `eb2e4876-eb71-422b-83ce-ccfa7b7e7da8` guardado).
- `GET /api/labs/results/{id}` → **200**, estado de la notificación: `Pending`.
- Evento en el outbox: intentos de publicación fallidos **1**, procesado: `pendiente` (sigue pendiente, con espera creciente).
- Notificaciones generadas mientras el broker está caído: **0**.
- `docker compose start rabbitmq` → código 0.
- Tras volver el broker, el evento pendiente se publicó y se consumió: notificación generada **True**; intentos fallidos acumulados 3; evento procesado a las `18:40:23`.
- Estado final de la notificación: `Sent`; notificaciones del resultado: **1**.

C. Reentrega del mismo evento: sin notificaciones duplicadas

- Se marcó de nuevo como pendiente el evento ya consumido (simula la caída entre "publicado" y "marcado") y el publicador lo volvió a publicar con el mismo `MessageId`.
- Evento republicado: **True**. Notificaciones del resultado después de la reentrega: **1** (debe ser 1).

D. Autorización y validación

Comprobación	Esperado	Obtenido	Resultado
Recepción registra un resultado	403	403	OK

Comprobación	Esperado	Obtenido	Resultado
Recepción lista resultados	403	403	OK
Recepción lee un resultado	403	403	OK
Médico sin relación lee el resultado	404	404	OK
Médico sin relación lista los del paciente	404	404	OK
Paciente intenta registrar un resultado	403	403	OK
Personal lee notificaciones de pacientes	403	403	OK
Sin sesión	401	401	OK
Resultado sin parámetros	400	400	OK
Fecha futura	400	400	OK
Paciente inexistente	400	400	OK
Modificar un resultado (PUT)	405	405	OK
Borrar un resultado (DELETE)	405	405	OK

Limpieza

- Se borraron los resultados, notificaciones y eventos de los pacientes `CYP-E2E-*` creados por esta ejecución. No se tocó ningún otro dato ni volumen.

Resultado global: CORRECTO (13/13 comprobaciones de la sección D).

Entrega 6 · comandos y resultados de las pruebas

Todas las ejecuciones se hicieron en equipos Windows con Smart App Control activo, por lo que el código se compiló y probó en contenedores Linux (`mcr.microsoft.com/dotnet/sdk:10.0`). Docker Desktop con los servicios de `docker compose` (PostgreSQL, Redis y RabbitMQ) en marcha. Datos de prueba aislados: Testcontainers crea bases desechables y Cypress/evidencia solo usan pacientes `CYP-E2E-*`.

Pruebas xUnit (unitarias + integración con PostgreSQL, Redis y RabbitMQ reales)

```
scripts/dotnet-linux.sh test MediCore.slnx
```

Proyecto	Resultado
`MediCore.UnitTests`	44 superadas, 0 fallidas, 0 omitidas
`MediCore.Api.Tests`	182 superadas, 0 fallidas, 0 omitidas (incluye `LaboratoryApiTests` y `MessagingTests`)

Cypress (Chrome 155 y Electron 146)

```
scripts/dev-stack-linux.sh up        # API y web en contenedores Linux, sobre las dependencias de compose
cd tests/cypress && CYPRESS_MEDICORE_PASSWORD=<Seed:DevPassword> npx cypress run --browser chrome
cd tests/cypress && CYPRESS_MEDICORE_PASSWORD=<Seed:DevPassword> npx cypress run --browser electron
scripts/dev-stack-linux.sh down
```

Spec	Chrome	Electron
`laboratorio-resultados.cy.js` (nuevo)	8/8	8/8
`medico-expediente.cy.js`	10/10	10/10
`recepcion-reserva.cy.js` (regresión)	8/8	8/8

Evidencia con el broker real

```
MEDICORE_PASSWORD=<Seed:DevPassword> python scripts/evidencia_laboratorio.py
```

Informe: `documentacion/evidencias/entrega-6-laboratorio.md` (broker disponible; broker detenido y recuperado; reentrega sin duplicados; 13/13 comprobaciones de autorización y validación).

Pruebas de integración de mensajería (`MessagingTests`, RabbitMQ en Testcontainers)

1. Registro → publicación → consumo → notificación interna del paciente.

2. RabbitMQ detenido: el resultado se guarda y queda "Pendiente"; al volver el broker el evento se recupera y se notifica.
3. Mensaje reentregado con el mismo `MessageId` y mensaje nuevo para el mismo resultado: una sola notificación.
4. Evento del outbox publicado dos veces: una sola notificación.
5. El consumidor falla dos veces: reintenta con espera y termina procesando (3 entregas).
6. El consumidor falla siempre: el mensaje va a la DLQ con el tipo de error (`InvalidOperationException`) y sin notificación; tras corregir la causa, "reintentar" encola un evento nuevo y la notificación se crea una sola vez.
7. Mensaje ilegible o de un resultado inexistente: directo a la DLQ, sin reintentos.
8. Publicador y consumidor activos con broker y apagados sin él.

Entrega 7 · resultado de los flujos Maestro

Ejecutado el 2026-10-27 15:25:13 -0600 con `scripts/maestro_run.py` en el emulador `medicore_pixel` (Pixel 6, Android 35 con Google APIs, x86_64, WHPX), contra la API y RabbitMQ de desarrollo.

Flujo	Escenario	Resultado	Duración
`01-login-valido.yaml`	base	APROBADO	33 s
`02-login-invalido.yaml`	base	APROBADO	58 s
`03-reserva-cita.yaml`	cita	APROBADO	62 s
`04-consulta-y-cancelacion.yaml`	cancelar	APROBADO	53 s
`05-resultados.yaml`	resultados	APROBADO	43 s
`06-recetas.yaml`	recetas	APROBADO	43 s
`07-notificaciones.yaml`	notificaciones	APROBADO	44 s
`08-cierre-de-sesion.yaml`	base	APROBADO	55 s

Entorno: Maestro 2.12.0, APK de desarrollo compilado en contenedor Linux (`scripts/android-build-linux.sh`) e instalado con `adb`, API/web en contenedores Linux sobre PostgreSQL, Redis y RabbitMQ de `docker compose` (la app llega por `10.0.2.2:5011`). Datos del paciente aislado `e2e.paciente@example.com` (CYP-E2E-004), preparados antes de cada flujo con `scripts/maestro_prepare.py`.

Historial honesto de esta sesión (antes de este resultado): la app no abría hasta corregir 3 defectos solo visibles en Android (ver `documentacion/05-app-movil.md`); en la primera pasada completa fallaron 3 flujos: 01 y 02 por selectores de los propios flujos (texto con ícono; campo de texto ya escrito) y 07 por un corte momentáneo del emulador (`device offline`), no de la app. Se corrigieron los flujos 01 y 02 y se repitió la suite completa: **8 de 8 aprobados en una sola ejecución**.

Límites: un emulador (Android 35), no dispositivos físicos; sin pruebas de accesibilidad, rendimiento ni rotación; Maestro valida lo visible, no la apariencia pixel a pixel.

Entrega 7 · comandos y resultados verificados

Equipo Windows con Smart App Control activo: el código se compila y prueba en contenedores Linux (`mcr.microsoft.com/dotnet/sdk:10.0`). Docker Desktop con los servicios de `docker compose`.

xUnit

```
scripts/dotnet-linux.sh test MediCore.slnx
```

Proyecto	Resultado
`MediCore.UnitTests`	44 superadas
`MediCore.Mobile.Tests` (cliente HTTP con respuestas programadas y ViewModels con API simulada para aislarlos)	50 superadas
`MediCore.Api.Tests` (incluye `MobileApiIntegrationTests`: 17 pruebas del cliente móvil contra la API REAL con PostgreSQL y Redis en Testcontainers)	199 superadas

Preparación de datos de Maestro contra la API real

```
MEDICORE_PASSWORD=<Seed:DevPassword> python scripts/maestro_prepare.py
<base|cita|cancelar|resultados|recetas|notificaciones>
```

Los 6 escenarios se prepararon sin errores (cita futura, cita para cancelar, resultado con su notificación asíncrona real, receta con consulta, notificación sin leer).

Compilación de la app Android (contenedor Linux)

```
scripts/android-build-linux.sh       # imagen medicore-android-build:local + volumen medicore-android-
sdk
```

Resultado: **Build succeeded**; `artifacts/com.medicore.paciente-Signed.apk` (SHA-256 `ef9f513cdd4264df855a314a13291624e305369409af6752b95de46f0d999a66`) y `com.medicore.paciente.apk` (`f003bdace2a2ab9802c0fd06e57176053d71ad57429001519e035f0a61b44e9f`). Contenido comprobado: `AndroidManifest.xml`, `res/xml/network_security_config.xml`, bibliotecas `arm64-v8a` y `x86_64`. Esto es **compilación**, no prueba en Android.

Android (ejecución)

Ejecutado en el emulador `medicore_pixel`: los 8 flujos Maestro pasan (8/8, una sola ejecución). Resultado completo en `entrega-7-maestro.md`.

(Registro previo, antes de instalar las herramientas: `python scripts/maestro_run.py` sin herramientas responde, sin escribir evidencia:

`NO se ejecutó ningún flujo. Falta: - Maestro CLI (comando "maestro" en el PATH) - adb (platform-tools del SDK de Android) - la variable MEDICORE_PASSWORD (Seed:DevPassword)`)

xUnit: qué se ejecutó y cuándo (sin presentar resultados viejos como nuevos)

Momento	Qué se ejecutó	Resultado
Antes de las correcciones de Android (estado del commit `9f7d464`)	`scripts/dotnet-linux.sh test MediCore.slnx` (suite completa)	293 superadas: 44 unitarias, 50 de la lógica móvil, 199 de integración
Después (commit `f269f0c`)	**No se volvió a ejecutar xUnit**	—

Entre ambos commits cambiaron únicamente: `src/MediCore.Mobile/` (`App.xaml.cs`, `AppShell.xaml`, `MediCore.Mobile.csproj`, `AndroidManifest.xml`), `scripts/maestro_run.py`, `tests/maestro/` (flujos 01 y 02 y el subflujo) y documentación. **No cambió** `MediCore.Mobile.Core`, la API ni ningún proyecto de pruebas xUnit (`git diff --name-only 9f7d464 f269f0c`), y ninguno de esos proyectos compila el proyecto MAUI. Por eso los 293 resultados corresponden al mismo código xUnit, pero **no son una ejecución posterior a las últimas correcciones**. Lo que sí se ejecutó después es la compilación de la app (Build succeeded) y los 8 flujos Maestro en el emulador.

Ubicación de los artefactos

- APK de desarrollo: `artifacts/com.medicore.paciente-Signed.apk` (≈84 MB, firmado con una clave de depuración; `com.medicore.paciente.apk` es el mismo sin firmar). **No está en Git** (carpeta `artifacts/` ignorada): se genera con `scripts/android-build-linux.sh`.
- Resultado de Maestro: `documentacion/evidencias/entrega-7-maestro.md`. Registros y capturas de cada ejecución de Maestro: `%USERPROFILE%\.maestro\tests\<fecha>\` en el equipo donde se ejecutó (no versionados).

Entrega 8 · flujos Maestro contra el ambiente QA en contenedores

Ejecutado el 2026-10-27 16:16:11 -0600 con `scripts/maestro_run.py` en un dispositivo Android real o emulado, contra la API y RabbitMQ de desarrollo.

Flujo	Escenario	Resultado	Duración
`01-login-valido.yaml`	base	APROBADO	39 s
`02-login-invalido.yaml`	base	APROBADO	61 s
`03-reserva-cita.yaml`	cita	APROBADO	65 s
`04-consulta-y-cancelacion.yaml`	cancelar	APROBADO	59 s
`05-resultados.yaml`	resultados	APROBADO	48 s
`06-recetas.yaml`	recetas	APROBADO	47 s
`07-notificaciones.yaml`	notificaciones	APROBADO	49 s
`08-cierre-de-sesion.yaml`	base	APROBADO	61 s

Entrega 8 · verificación local (contenedores e infraestructura)

Equipo Windows con Docker Desktop y Smart App Control activo. Todo lo siguiente se **ejecutó** salvo lo indicado en "No ejecutado". Ningún recurso en la nube.

1. Imágenes y ambiente QA en contenedores

```
docker build -f docker/{api,web,worker,migrator}.Dockerfile -t medicore/<servicio>:local .   # 4
imágenes, sin errores
scripts/qa.sh init && scripts/qa.sh up && scripts/qa.sh seed
```

- Todos los servicios quedaron `healthy` (migrate terminó con código 0). La base **nueva** se migró desde cero con el paquete de EF (`LaboratorioYNotificaciones` fue la última migración aplicada).
- Defectos encontrados al ejecutarlo y corregidos (ver hallazgos H8-01 y H8-02): permisos del volumen de claves de la web y orden de los sembradores en una base nueva.

1b. Clon limpio

Se clonó la rama desde GitHub en una carpeta temporal y, usando solo lo versionado, se ejecutaron `scripts/qa.sh init`, `docker compose config` y la construcción de las 4 imágenes: todo correcto. (Esta comprobación detectó que `.env.qa.example` estaba ignorado por `.gitignore` y no se versionaba; se corrigió.)

2. Pruebas de humo de web y API (`scripts/qa.sh smoke`, a través del proxy)

11 comprobaciones, todas OK: liveness y readiness de la API (incluye PostgreSQL), la API exige autenticación (401), la web sirve `/Login` en español con CSP, la raíz redirige al inicio de sesión (302), la documentación OpenAPI no se expone en QA, inicio de sesión de Recepción por la API con los usuarios ficticios sembrados y listado de médicos autenticado.

Además, el spec de Cypress de Recepción (8 pruebas, Chrome) pasó **contra el ambiente QA en contenedores a través del proxy** (`MEDICORE_WEB_URL=http://localhost:8080`): 8/8. Cubre el inicio de sesión real de la web con la cookie cifrada cuyas claves viven en el volumen `web-keys`.

3. Persistencia (`scripts/qa-persistence.sh`)

Se crean por la API un paciente nuevo y un resultado de laboratorio (que dispara el evento por el outbox → worker → RabbitMQ → notificación del paciente) y se comprueban tres veces (inicial, tras `restart`, tras `down` + `up`): el paciente, el resultado y **una sola** notificación existen siempre. Se comprobó que los 6 volúmenes `medicore-qa_*` se conservan tras `down`. Resultado: **13/13 comprobaciones OK**. No se borró ningún volumen del desarrollo (`medicore_*`); el único `destroy` fue sobre los volúmenes `medicore-qa_*` que esta misma entrega había creado, para probar una instalación desde cero.

4. Puertos expuestos

`docker ps` del proyecto `medicore-qa`: solo el proxy publica puertos (127.0.0.1:8080 y 127.0.0.1:8443). PostgreSQL, Redis, RabbitMQ, API, web y worker no publican nada; la red `backend` es `internal` (sin salida a Internet ni al anfitrión).

5. App Android contra el ambiente local en contenedores

APK compilado con `scripts/android-build-linux.sh http://10.0.2.2:8080/` (el emulador ve el proxy del anfitrión), instalado en el emulador `medicore_pixel` (Android 35). Los 8 flujos Maestro se ejecutaron contra el ambiente QA (la preparación de datos usa la API del proxy y la base del Compose de QA):

- **Segunda ejecución completa: 8 de 8 aprobados** (`documentacion/evidencias/entrega-8-maestro-qa.md`).
- Primera ejecución: 6 de 8; los flujos 01 y 03 fallaron en 8-9 s por `DeviceServerDiedException` del driver de Maestro en el emulador (`device server died during deviceInfo`), no por la app ni por el ambiente (02 y 04-08 usaron el mismo inicio de sesión y pasaron). Se repitió la suite completa y pasó.
- Límite: un emulador; solo el tráfico por HTTP a `10.0.2.2` (la app no permite HTTP hacia otros hosts). Con Azure se necesitará HTTPS.

6. Pruebas xUnit después de los cambios de código

Cambió código (proyecto `MediCore.Worker`, reordenamiento de sembradores, cabeceras reenviadas y claves de protección de datos en API y web, modo `--seed-qa`). Se **volvió a ejecutar** `scripts/dotnet-linux.sh test MediCore.slnx`: **293 superadas** (44 unitarias, 50 de la lógica móvil, 199 de integración), 0 fallidas. No se agregaron pruebas xUnit nuevas para el worker ni para `--seed-qa`: se verificaron con el ambiente en contenedores (secciones 1 a 3).

7. Validación de archivos de Terraform, Ansible y Docker (`scripts/validate-iac.sh`, en contenedores oficiales)

Qué	Resultado
Terraform 1.12.2: `fmt -check`, `init -backend=false`, `validate` en `azure` y `bootstrap`	correcto
Ansible-core 2.21: `--syntax-check` de `site.yml`	correcto
`ansible-lint` (perfil *production*)	0 fallos (se corrigieron 2 al inicio)
`hadolint` en los 4 Dockerfile (umbral *warning*)	sin advertencias; solo avisos informativos DL3066 (UID numérico)
`docker compose config` del ambiente QA	válido

Esto valida archivos. NO es un despliegue en Azure.

8. No ejecutado

- `terraform plan` / `apply` / `destroy` (requieren suscripción y credenciales de Azure; no se solicitó ejecutarlos).
- `ansible-playbook` contra un servidor real (no hay servidor), HTTPS con certificado real (Let's Encrypt necesita un nombre DNS público), el clon del repositorio desde la VM y el disco de datos de Azure.

- Cypress de Médico y Laboratorio contra QA (sus tareas escriben en la base y la base de QA no se publica); Maestro/Cypress contra Azure.
- Escaneo de seguridad de las imágenes y del código (DevSecOps pendiente).

Terraform y Ansible ejecutados sin Azure

Ejecutado el 2026-11-06 con `scripts/iac-simulacion.sh`. El equipo decidió no usar recursos de pago, así que **esto no es un despliegue en Azure**: no hubo suscripción, IP pública, certificado ni costo.

Terraform

`terraform test` ejecuta el código de `infrastructure/terraform/azure` con el proveedor de Azure simulado. Resultado: 3 de 3 etapas aprobadas (`reports/iac/terraform-test.txt`).

Etapa	Qué se comprobó
Aprovisionar	Crea todos los recursos. VM `Standard_B2s` , sin contraseñas, con disco de datos y apagado automático. Solo abre 22, 80 y 443, y SSH solo a los rangos de administración
Modificar	Cambia la VM a `Standard_B2ms` , el disco a 64 GB y retira el apagado automático
Rechazar SSH abierto	Con `0.0.0.0/0` , Terraform se niega a continuar
Destruir	Al terminar, Terraform destruye todo lo creado en la simulación

Qué no prueba: cuotas, precios, disponibilidad de la región ni que Azure acepte cada valor.

Ansible

`infrastructure/ansible/site.yml` se aplicó sobre un Ubuntu 24.04 desechable (contenedor con systemd y Docker propio). El detalle está en `reports/iac/ansible.txt` .

Pasada	Tareas correctas	Con cambios	Fallidas
Primera	23	18	0
Segunda	21	2	0

- **Paquetes y sistema:** se aplicaron los paquetes base, SSH solo con clave, actualizaciones automáticas, fail2ban y cortafuegos.
- **Docker:** se instaló desde el repositorio oficial.
- **Aplicación:** subió con los 8 servicios saludables, incluido `backup` , desde el commit `3fe9d40` . `GET /api/health` y `GET /Login` respondieron 200.
- **Cortafuegos:** quedó activo y solo permite 22, 80 y 443.
- **Segunda pasada:** los 2 cambios son la siembra de datos, que siempre se marca como cambio, y la tarea que levanta Compose. Esa tarea no reinició ni dañó la aplicación, pero tampoco es idempotente de forma estricta.

Diferencias con Azure: conexión local en vez de SSH, sin disco de datos, repositorio local y secretos aleatorios de un solo uso. El servidor se elimina al terminar.

Defecto encontrado: el rol de Docker necesitaba `python3-debian` y nadie lo instalaba (H9-02). Sin esta ejecución habría fallado el primer día en una VM mínima.

Cómo repetirlo

```
bash scripts/iac-simulacion.sh all
```

Maestro · repetición con el código final

Ejecutado el 2026-11-06 14:15:08 -0600 con `scripts/maestro_run.py` en un dispositivo Android real o emulado, contra la API de http://127.0.0.1:8080 y su RabbitMQ.

Flujo	Escenario	Resultado	Duración
`01-login-valido.yaml`	base	APROBADO	29 s
`02-login-invalido.yaml`	base	APROBADO	52 s
`03-reserva-cita.yaml`	cita	APROBADO	53 s
`04-consulta-y-cancelacion.yaml`	cancelar	APROBADO	47 s
`05-resultados.yaml`	resultados	APROBADO	36 s
`06-recetas.yaml`	recetas	APROBADO	38 s
`07-notificaciones.yaml`	notificaciones	APROBADO	37 s
`08-cierre-de-sesion.yaml`	base	APROBADO	43 s

Evidencia de resiliencia en QA

Ejecutada el 2026-11-06 02:16 con `scripts/evidencia_resiliencia.py` sobre el ambiente QA en contenedores. Solo datos ficticios.

1. Redis detenido

- Antes, `GET /api/doctors` con sesión: **200**
- `docker stop` de Redis. Login: **503**
- El mismo token: **401**
- Redis iniciado de nuevo. Login: **200**, `GET /api/doctors`: **200**

2. Worker detenido

- `docker stop` de `medicore-qa-worker-1`. Registrar resultado: **201**
- 5 segundos después, lectura: **200**, notificación: **Pending**
- Servicio iniciado de nuevo. Notificación enviada en **0.0 s**, estado: **Sent**
- Notificaciones del paciente para ese resultado: **1**

3. RabbitMQ detenido

- `docker stop` de `medicore-qa-rabbitmq-1`. Registrar resultado: **201**
- 5 segundos después, lectura: **200**, notificación: **Pending**
- Servicio iniciado de nuevo. Notificación enviada en **1.0 s**, estado: **Sent**
- Notificaciones del paciente para ese resultado: **1**

Respaldos automáticos y simulacro de restauración

Ejecutado el 2026-11-06 en el ambiente QA local, con `scripts/simulacro-respaldo.sh`. Resultado completo: `reports/respaldos/simulacro.txt`. Cierra H8-08 y reduce R-16.

Cómo funciona

El servicio `backup` de `docker-compose.qa.yml` ejecuta `deploy/respaldos/respaldo.sh`:

- Crea una copia al arrancar y luego cada 6 horas (`BACKUP_INTERVAL_HOURS`).
- Conserva las 28 más recientes (`BACKUP_KEEP`), o sea 7 días.
- Cifra con AES-256 y `BACKUP_PASSPHRASE`. Sin esa clave la copia no se puede leer.
- Descifra y lee cada copia completa antes de aceptarla. Si falla, la borra y lo registra.
- Guarda una huella SHA-256 y la verifica antes de restaurar.
- Marca el servicio como no saludable si no hay una copia reciente.
- Registra solo nombres y tamaños, nunca datos ni la clave.

Las copias van en el volumen externo `medicore-qa-respaldos`, que sobrevive a `qa.sh destroy`.

```
scripts/qa.sh backup                  # copia inmediata
scripts/qa.sh backups                 # lista las copias
scripts/qa.sh export artifacts/copia  # las saca de Docker
scripts/qa.sh import <archivo>        # devuelve una copia exportada
scripts/qa.sh restore <archivo>       # reemplaza la base, pide confirmación
```

Simulacro

1. Se tomó la huella md5 y el conteo de filas de cada tabla.
2. Se creó una copia y se exportó fuera de Docker.
3. **Desastre:** se destruyeron la base y también el volumen de respaldos.
4. Se reconstruyó QA. La base quedó con 0 usuarios.
5. Se importó la copia exportada y se restauró.
6. Se compararon las tablas y se ejecutaron las pruebas de humo.

Medida	Resultado
Tablas comparadas	14
Filas antes y después	36 849 y 36 849
Tablas idénticas	Sí, en filas y md5
Pruebas de humo	Aprobadas
Servicios después de restaurar	8 saludables
Tiempo de recuperación medido	72 s
Datos perdidos en el simulacro	0, porque la copia se tomó justo antes

Otras comprobaciones

Prueba	Resultado
Ciclo y retención, con intervalo 0 y límite 3	Quedan 3 copias y sus 3 huellas; las antiguas se retiran
Base caída durante el ciclo	Registra el fallo, sigue en marcha y no deja archivos temporales
Archivo cortado	`pg_restore` lo rechaza
Clave incorrecta	El descifrado falla
Ansible en Ubuntu desechable, commit `3fe9d40`	8 servicios saludables, incluido `backup`

Qué encontró el simulacro

- `.gitignore` ignoraba `deploy/backup/` por la regla `Backup*/`: Jenkins no habría recibido el script. Se movió a `deploy/respaldos/`.
- La primera comprobación solo leía el índice de la copia. Ahora la lee completa y detecta un `pg_dump` interrumpido.
- La primera comparación dio una fila de más en `audit_entries`: el script comparaba después del inicio de sesión de las pruebas de humo. Se corrigió el orden y se repitió el simulacro completo.

Límites

- **Pérdida máxima esperada: 6 horas.** Es el intervalo entre copias; no hay recuperación a un punto exacto en el tiempo.
- **Las copias siguen en el mismo disco.** Sobreviven a `qa.sh destroy`, pero no a una falla del disco. Sacarlas a otro lugar es manual, con `qa.sh export`. Un destino automático externo requiere un servicio de pago o una decisión del equipo.
- **La clave debe guardarse aparte.** Si se pierde `BACKUP_PASSPHRASE`, las copias no sirven.
- **El tiempo de 72 s es de este equipo y de una base pequeña** (1,7 MB cifrada). En un servidor real debe medirse otra vez.
- **El ciclo de 6 horas no se observó en tiempo real.** Se probó con intervalo 0.
- **Solo se respalda PostgreSQL.** Redis y RabbitMQ no. Las sesiones se cierran al restaurar y los eventos pendientes del outbox sí están en la copia.
- **No hay alertas.** Un respaldo fallido queda en las bitácoras y en el estado del contenedor; nadie recibe un aviso.
- **No se probó en Azure.**